

Frankenstein Transformer: Unified Encoder-Decoder Library, CLI, and Research-Grounded Design Notes

Erick F. Merino M.
This work is not affiliated
`erickfmm@gmail.com`

February 2026

Abstract

Frankenstein Transformer presents a unified configuration-driven toolkit for systematic experimentation with modern transformer architectures, spanning seventeen sequence mixer variants and twenty-three optimizer families. The system supports both encoder-style masked language modeling (MLM) and decoder-style autoregressive (AR) next-token prediction through flexible model class and mode configuration, with specialized fine-tuning workflows for both architectures. The research contributions are threefold: (i) a strict schema-based configuration contract that enables reproducible experimentation across diverse attention mechanisms, including standard softmax attention, sigmoid attention, retentive networks, selective state-space models, continuous-depth transformers, adaptive depth routing (Mixture-of-Depths) [68], conditional memory augmentation (Engram) [10], sparse attention patterns, and gated mechanisms; (ii) a comprehensive optimizer routing framework supporting variance-reduction methods (MARS, Adan, AdEMAMix), memory-efficient variants (Adafactor, GaLore, Lion), schedule-free approaches, second-order preconditioners (Shampoo, SOAP, Sophia), and low-rank APOLLO-family optimizers (Apollo, Apollo-Mini, Q-Apollo) [66]; and (iii) end-to-end workflows spanning quantized deployment via ternary weight packing and sentence-embedding training inspired by SBERT, backed by an expanded quality-assurance stack with broad unit test coverage, YAML example validation, and continuous integration execution. The toolkit implements a web-based configuration interface that provides schema-driven form rendering with inline documentation and real-time validation. This technical reference document includes architectural diagrams, execution-flow visualizations, decision tables, and comprehensive appendices synthesizing literature on transformer architectures, sparse attention mechanisms, gated attention variants, and optimization algorithms. The system enables rapid iteration while maintaining reproducible experimental conditions through its schema-first design philosophy.

Contents

1	Introduction	7
1.1	Motivation and Problem Statement	7
1.2	Contributions	8
1.3	Reading Guide	9
1.4	Web-Based Configuration Builder	9
2	Related Work	10
2.1	Sequence Mixer Architectures	10
2.1.1	Dense Attention Baselines	10
2.1.2	Recurrent and Retentive Architectures	10

2.1.3	Sparse Attention Mechanisms	10
2.1.4	Gated Attention Mechanisms	11
2.2	Optimization Algorithms	11
3	System Design and Architecture	11
3.1	Configuration-Centric Architecture	11
3.2	Schema Scope and Validation Rules	13
3.3	Complete Model Feature Inventory	13
3.4	Training Task Types	14
3.5	Complete Training Feature Inventory	15
3.6	Optimizer Prefix Contract (Full)	16
3.7	Training Safety and Runtime Semantics	17
3.8	Normalization Variants	17
4	Architecture Taxonomy and Implementation	18
4.1	Attention and Sequence-Mixer Families	18
4.2	Dense and Recurrent Families	18
4.3	Latent and Low-Rank Attention Family	19
4.4	Sparse Attention Extensions	19
4.5	Gated Attention Extensions	19
5	Optimizer Families and Training Dynamics	20
6	Quantization and Deployment	20
6.1	Ternary Quantization and Activation Scaling	20
7	SBERT Downstream Tasks	21
8	Summary Tables	21
8.1	Attention and Sequence-Mixer Summary	21
8.2	Optimizer Summary	22
9	Discussion	23
9.1	Schema-Driven Design Trade-offs	23
9.2	Architectural Coverage and Gaps	23
9.3	Optimizer Landscape Fragmentation	23
9.4	Deployment and Production Considerations	24
9.5	Integration and Extensibility Challenges	24
10	Conclusion	24
10.1	Limitations and Future Directions	25
	Bibliography	26
A	Annex A: Optimizer Families	30
A.1	Memory and Complexity Comparison	30
A.2	The Evolution of Optimization in Neural Networks	31
A.3	Standard Baseline and Adaptive Optimizers	31
A.4	Advanced Momentum and Variance Reduction (2024–2025)	33
A.5	Large-Batch, Memory-Efficient, and Parameter-Free Optimizers	37
A.6	Second-Order, Geometric, and Orthogonality Optimizers	43

B	Annex B: Dense, Recurrent, and Memory-Augmented Transformers	47
B.1	Dense Attention Baselines: Standard and Sigmoid	47
B.1.1	Standard Softmax Attention	47
B.1.2	Sigmoid Attention	48
B.1.3	Grouped-Query Attention (GQA)	48
B.2	Recurrent and Retentive Architectures	49
B.2.1	Retentive Networks (RetNet)	49
B.2.2	Mamba: Selective State-Space Models	49
B.3	Continuous-Depth Transformers: ODE Integration	50
B.3.1	ODE Transformer	50
B.4	Test-Time Memory: Titans	50
B.5	Pattern-Driven Mixer Dispatch	51
B.6	Architectural Comparison and Synthesis	51
C	Annex B: Latent and Low-Rank Attention Families	52
C.1	Multi-Head Latent Attention (MLA)	52
C.1.1	Mathematical Formulation	52
C.1.2	Algorithmic Pseudocode	52
C.1.3	Key Characteristics	52
C.2	Group-Query Latent Attention (GQLA)	53
C.2.1	Mathematical Formulation	53
C.2.2	Algorithmic Pseudocode	54
C.2.3	Key Characteristics	54
C.3	Multi-Head Low-Rank Attention (MLRA)	54
C.3.1	Mathematical Formulation	54
C.3.2	Algorithmic Pseudocode	54
C.3.3	Key Characteristics	55
C.4	Tucker Attention	55
C.4.1	Mathematical Formulation	55
C.4.2	Algorithmic Pseudocode	55
C.4.3	Key Characteristics	55
C.5	Interleaved Head Attention (IHA)	56
C.5.1	Mathematical Formulation	56
C.5.2	Algorithmic Pseudocode	56
C.5.3	Key Characteristics	56
C.6	Grouped-head laTenT Attention (GTA)	56
C.6.1	Mathematical Formulation	57
C.6.2	Algorithmic Pseudocode	57
C.6.3	Key Characteristics	57
C.7	Multi-head Temporal Latent Attention (MTLA)	57
C.7.1	Mathematical Formulation	57
C.7.2	Algorithmic Pseudocode	58
C.7.3	Key Characteristics	58
C.8	Architectural Comparison and Synthesis	58
D	Annex C: Comprehensive Sparse Attention Mechanisms	59
D.1	Executive Summary	59
D.2	Sparse Transformer: Factorized Strided and Fixed Patterns	59
D.2.1	Mathematical Formulation	59
D.2.2	Algorithmic Pseudocode	60
D.2.3	Key Characteristics	60
D.3	Longformer: Sliding Window, Dilation, and Global Tokens	60

D.3.1	Mathematical Formulation	60
D.3.2	Algorithmic Pseudocode	61
D.3.3	Key Characteristics	61
D.4	BigBird: Random, Local, and Global Sparse Graph	61
D.4.1	Mathematical Formulation	61
D.4.2	Theoretical Guarantee	62
D.4.3	Algorithmic Pseudocode	62
D.4.4	Key Characteristics	62
D.5	SparseK Attention: Differentiable Top- k Selection	63
D.5.1	Mathematical Formulation	63
D.5.2	Algorithmic Pseudocode	63
D.5.3	Key Characteristics	63
D.6	NSA (Native Sparse Attention): Hardware-Aligned Hierarchical Branches	64
D.6.1	Mathematical Formulation	64
D.6.2	Algorithmic Pseudocode	64
D.6.3	Key Characteristics	65
D.7	FASA: Frequency-Aware Sparse Attention	65
D.7.1	Mathematical Formulation	65
D.7.2	Algorithmic Pseudocode	66
D.7.3	Key Characteristics	66
D.8	SparseAttn: Two-Stage Block-Level Filtering	66
D.8.1	Mathematical Formulation	66
D.8.2	Algorithmic Pseudocode	67
D.8.3	Key Characteristics	67
D.9	MSA (MiniMax Sparse Attention): Blockwise Sparse Attention with Lightweight Index Branch	68
D.9.1	Mathematical Formulation	68
D.9.2	Algorithmic Pseudocode	68
D.9.3	Key Characteristics	69
D.10	SparDA (Sparse Decoupled Attention): Forecast-Guided Lookahead Block Selection	69
D.10.1	Mathematical Formulation	69
D.10.2	Algorithmic Pseudocode	70
D.10.3	Key Characteristics	70
D.11	Comparative Summary	70
D.12	Design Space and Selection Criteria	71
E	Annex D: Gated Attention Families—Complete Literature Analysis	71
E.1	Executive Summary: Gating for Memory Control	71
E.2	1. Gated Linear Attention (GLA)	72
E.2.1	Mathematical Foundation	72
E.2.2	Hardware-Efficient Training	72
E.2.3	Strengths and Limitations	73
E.3	2. DeltaNet: Error-Correcting Linear Attention	73
E.3.1	Mathematical Foundation	73
E.3.2	Efficient Parallel Training	73
E.3.3	Strengths and Limitations	74
E.4	3. Gated DeltaNet: Synthesis of Gating and Error Correction	74
E.4.1	Mathematical Foundation	74
E.4.2	Empirical Performance and Trade-offs	75
E.5	4. HGRN2: Hierarchical Gating with Outer-Product Expansion	75
E.5.1	Mathematical Foundation	75

E.5.2	Scaling and Empirical Results	76
E.6	5. Forgetting Transformer (FoX): Gating in Softmax Logit Space	76
E.6.1	Mathematical Foundation	76
E.6.2	Integration with FlashAttention	77
E.6.3	Strengths and Limitations	77
E.7	6. Gated Attention (Post-SDPA Sigmoid Gating)	77
E.7.1	Mathematical Foundation	77
E.7.2	Key Findings	78
E.8	7. Gated DeltaNet-2: Decoupled Channel-wise Erase and Write	78
E.8.1	Mathematical Foundation	78
E.8.2	Empirical Performance and Trade-offs	80
E.9	8. Kimi Delta Attention (KDA): Channel-wise Decay Gating for the Delta Rule	80
E.9.1	Mathematical Foundation	80
E.9.2	Empirical Performance and Trade-offs	81
E.10	Comparative Analysis: Taxonomy of Gated Mechanisms	81
E.11	Unified Gating Principle	83
E.12	Implementation and Practical Guidance	83
E.12.1	When to Use Each Architecture	83
E.12.2	Hardware Considerations	84
F	Annex E: Conceptual Introduction—Transformers and Attention for Beginners	84
F.1	What is a Transformer?	84
F.2	The Attention Mechanism: An Analogy	84
F.3	Practical Example Step-by-Step	85
F.3.1	Context 1: “The cat eats fish”	85
F.3.2	Context 2: “The restaurant eats into profit margins”	85
F.4	How Attention Works Mathematically (Simple Version)	85
F.4.1	Step 1: Queries, Keys, and Values	85
F.4.2	Step 2: Compatibility	86
F.4.3	Step 3: Focus	86
F.4.4	Step 4: Combine	86
F.5	Multiple Attention Heads: Multiple Perspectives	86
F.6	Stacking Layers	86
F.7	Why Does It Matter?	86
F.8	Practical Challenges	87
F.8.1	Computational Complexity	87
F.8.2	Memory Requirements	87
F.8.3	Device Efficiency	87
F.9	Modern Solutions	87
F.10	Key Takeaways	87
G	Annex F: Normalization, Quantization, Depth Routing, and Runtime Algorithms	88
G.1	Normalization Variants: LayerNorm, RMSNorm, Dynamic Tanh, and Dynamic Erf	88
G.1.1	LayerNorm (Baseline)	88
G.1.2	RMSNorm (Not Accepted by Schema)	88
G.1.3	Dynamic Tanh (DyT)	89
G.1.4	Dynamic Erf (Derf)	89
G.1.5	Comparison	89
G.2	BitNet Ternary Quantization Path	89
G.3	Mixture-of-Depths (MoD) Token Routing	90

G.3.1	Configuration	90
G.4	Engram Conditional Memory	90
G.4.1	Configuration	90
G.5	Factorized Embeddings and Embedding Convolution	91
G.5.1	Factorized Embeddings	91
G.5.2	Embedding Convolution	91
G.6	Training-Step Stability Controls	91
G.7	SBERT Inference Mode Router	92

1 Introduction

1.1 Motivation and Problem Statement

Transformer architectures have fundamentally transformed the landscape of sequence modeling across natural language processing [50], computer vision, and computational biology. The success of models such as BERT [14], GPT, and their variants has established the transformer as the de facto standard for representation learning in deep learning. However, the rapid proliferation of architectural innovations presents significant practical challenges for researchers and practitioners.

Recent years have witnessed an explosion of alternative attention and sequence-mixing mechanisms, each addressing specific limitations of standard softmax attention: quadratic computational complexity [11], memory-efficient inference requirements [48, 17], content-based selective state management [4], and hardware-aware optimizations [41]. Simultaneously, the optimization literature has diversified beyond classical AdamW, introducing variance-reduction techniques [54, 37, 60], memory-efficient variants [44, 65], schedule-free approaches [12], and second-order preconditioning methods [19, 51].

The practical consequence is a fragmented research ecosystem where experimental comparison across architectures and optimizers requires significant engineering effort. Researchers must implement and debug multiple variants from scratch, ensure consistent training pipelines, and manage complex hyperparameter spaces. This fragmentation hampers reproducibility, slows scientific progress, and increases the barrier to entry for new researchers.

This work addresses these challenges through a unified, configuration-driven experimentation toolkit available at <https://github.com/erickfmm/frankestein-transformer> that provides:

1. **Schema-First Design:** A strict, validated configuration contract that enforces reproducibility while supporting seventeen distinct sequence mixer architectures and twenty-three optimizer families.
2. **Architecture Agnostic Training:** Common training infrastructure supporting dense attention baselines (standard, sigmoid), recurrent alternatives (RetNet, Mamba, ODE-style blocks), adaptive depth routing (Mixture-of-Depths) [68], conditional memory blocks (Engram) [10], sparse attention patterns (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA), and gated mechanisms (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).
3. **Optimizer Routing Framework:** Prefixed hyperparameter groups enabling fine-grained control over embeddings, normalization layers, recurrent blocks, attention blocks, and other parameter subsets across diverse optimizers, including the APOLLO family (Apollo, Apollo-Mini, Q-Apollo) [66].
4. **End-to-End Workflows:** Integrated deployment via quantization (ternary weight packing, INT8 activations) and sentence-embedding capabilities inspired by SBERT [42].
5. **Interactive Configuration:** Web-based interface providing schema-driven form rendering, real-time validation, and CLI command generation.
6. **Reliability Tooling:** Comprehensive automated testing with unit-test suites, YAML preset validation, and CI automation across supported Python versions.

The project command surface is:

`frankestein-transformer`

with subcommands for encoder and decoder workflows: `train`, `finetune`, `deploy`, `quantize`, `infer`, `sbert-train`, `sbert-infer`, `web-server`.

The `web-server` command launches a Streamlit-based configuration builder that provides:

- Schema-driven form fields with parameter titles and detailed descriptions
- Real-time tooltips and help text for each configuration option
- Live YAML preview and download functionality
- Generated CLI commands for training, deployment, inference, and SBERT workflows

This interactive interface serves as an alternative to manual YAML editing, improving usability for users exploring available configuration options and understanding their impact on model behavior and training dynamics.

1.2 Contributions

This work makes the following primary contributions:

1. **Unified Configuration Schema:** A YAML-based schema contract with strict validation that supports seventeen distinct sequence mixer architectures across four categories (dense baselines, recurrent/retentive blocks, sparse attention patterns, and gated mechanisms), alongside adaptive depth and conditional memory controls, while enforcing reproducibility through `additionalProperties: false` constraints.
2. **Comprehensive Architecture Support:** Implementation of modern transformer variants including standard softmax attention [50], sigmoid attention [41], RetNet [48], Mamba [17], ODE-style continuous transformers [63], Titans memory-augmented attention [4], Engram conditional memory layers [10], Mixture-of-Depths token routing [68], sparse attention mechanisms [11, 5, 61, 32, 59, 64, 53], and gated attention architectures [55, 56, 57, 20, 39, 27, 40]. System supports both encoder-mode training with bidirectional attention for masked language modeling (MLM) and decoder-mode training with causal masking for autoregressive next-token prediction.
3. **Optimizer Routing Framework:** Prefixed hyperparameter system enabling per-parameter-group control across twenty-three optimizers, including the adaptivity-tunable Anon [2], variance-reduction methods (MARS, Adan, AdEMAMix, Cautious AdamW), memory-efficient variants (Adafactor, GaLore, Lion), schedule-free approaches (Schedule-Free AdamW), curvature-aware methods (Sophia), second-order preconditioners (Shampoo, SOAP), orthogonality-oriented optimizers (Muon, Turbo-Muon), large-batch scaling (LAMB), and the APOLLO family (Apollo, Apollo-Mini, Q-Apollo) [66].
4. **Quantization and Deployment:** Integrated deployment pipeline supporting ternary weight packing and INT8 activation quantization with size estimates following 1.58-bit storage approximation.
5. **Sentence-Embedding Workflows:** SBERT-inspired training and inference pipelines supporting similarity scoring, retrieval, clustering, and persistent embedding export.
6. **Interactive Configuration Interface:** Streamlit-based web server providing schema-driven form generation, real-time validation, inline documentation, and CLI command generation.
7. **Automated Validation Pipeline:** Continuous integration and expanded unit testing that verify model training paths, optimizer/schema integration, and YAML example compatibility.

1.3 Reading Guide

This document is organized as a technical reference addressing four operational concerns:

1. **Prerequisites:** Appendix [F](#) provides an accessible introduction to transformers and attention mechanisms for readers new to the field.
2. **Configuration Contract:** Section [3.1](#) describes the YAML schema that enforces valid experiments and Section [3.2](#) explains validation rules.
3. **Architecture Selection:** Section [4.1](#) provides comprehensive comparison of sequence mixer families; Appendices [B](#), [C](#), [D](#), and [E](#) synthesize supporting literature; Appendix [G](#) details normalization variants, BitNet quantization, Mixture-of-Depths routing, and Engram conditional memory.
4. **Optimization Dynamics:** Section [5](#) details optimizer routing and training dynamics; Appendix [A](#) provides comprehensive optimizer family analysis.
5. **Deployment and Inference:** Sections [6](#) and [7](#) describe quantized deployment and sentence-embedding workflows.

1.4 Web-Based Configuration Builder

In addition to direct YAML editing, this project provides a Streamlit-based web interface (accessed via `web-server` command) that improves configuration accessibility and discoverability. The interface presents schema fields with:

- **Schema-driven form rendering** — All fields are dynamically generated from the authoritative schema, ensuring consistency and validation.
- **Inline parameter documentation** — Each form field displays a *title* from the schema as its label, with the *description* shown as a help tooltip on hover.
- **Real-time configuration preview** — Users see live YAML output as they modify form fields, enabling immediate validation feedback.
- **CLI command generation** — The interface generates complete CLI commands for training, deployment, inference, and SBERT workflows based on current configuration.
- **Accessibility improvements** — Tooltips and structured forms make configuration options easier to understand, especially for users new to the project or exploring novel architectures.

This web-based approach addresses common usability barriers in configuration-driven experimentation:

- Reduces need to memorize YAML structure and field names
- Prevents typos through schema validation
- Provides educational context through inline documentation
- Enables rapid experimentation with guided parameter tuning
- Serves as both a configuration tool and a learning resource

The web interface implementation uses Streamlit’s form widgets with:

- `st.checkbox()` for binary toggles with help text

- `st.number_input()` for numeric fields with step size and format
- `st.selectbox()` for enum choices with options display
- `st.multiselect()` for array selections from defined options
- `st.text_input()` for string fields
- `st.info()`, `st.caption()` for supplementary information

Schema metadata (title and description fields) are extracted and rendered systematically across all form sections, including:

- Model architecture parameters (hidden size, layers, attention heads, etc.)
- Training runtime settings (batch size, accumulation, scheduler)
- Optimizer configuration with per-parameter-group hyperparameters
- Deployment and quantization options
- SBERT-specific training and inference parameters

2 Related Work

This work sits at the intersection of three active research areas: alternative attention architectures, sparse attention mechanisms, and advanced optimization algorithms. This section provides a concise survey; detailed mathematical formulations and algorithmic descriptions are deferred to the appendices.

2.1 Sequence Mixer Architectures

2.1.1 Dense Attention Baselines

Standard softmax attention [50] achieves full global context through n^2 pairwise interactions but incurs prohibitive memory and compute costs for long sequences. Sigmoid attention [41] replaces row-wise softmax with element-wise sigmoid, yielding faster convergence and up to 17% kernel speedup, though requiring hybrid-norm stabilization at scale. Both serve as dense baselines in this toolkit.

2.1.2 Recurrent and Retentive Architectures

RetNet [48] resolves the “impossible triangle” of parallel training, $\mathcal{O}(1)$ inference, and strong performance via a dual attention–retention formulation. Mamba [17] introduces input-dependent selectivity into state-space models, achieving linear complexity with hardware-aware scan algorithms. ODE-style transformers [63] treat depth as numerical integration over a continuous dynamical system. Titans [4] augments inference with test-time neural memorization for extremely long contexts. Detailed formulations are provided in Appendix B.

2.1.3 Sparse Attention Mechanisms

Sparse attention methods reduce the quadratic bottleneck by restricting token interactions: Sparse Transformer [11] uses factorized strided and fixed patterns ($\mathcal{O}(n\sqrt{n})$); Longformer [5] employs sliding windows with global tokens ($\mathcal{O}(n \cdot w)$); BigBird [61] combines local, random, and global paths; SparseK [32] applies differentiable top- k selection; NSA [59] uses a three-branch hierarchical design; SpargeAttn [64] performs two-stage block-level pruning; and FASA [53] leverages RoPE frequency features for token selection. Complete descriptions are in Appendix D.

2.1.4 Gated Attention Mechanisms

Gated architectures control information flow through learnable gates: GLA [55] adds data-dependent diagonal gating to linear attention; DeltaNet [56] applies the delta learning rule to recurrent state updates; Gated DeltaNet [57] synthesizes both mechanisms; Gated DeltaNet-2 [20] decouples the scalar delta gate into channel-wise erase (key axis) and write (value axis) gates; HGRN2 [39] introduces hierarchical forget gates with outer-product state expansion; FoX [27] embeds forget gates directly into softmax attention; and Gated Softmax [40] applies post-SDPA channel gating. Full mathematical derivations appear in Appendix E.

2.2 Optimization Algorithms

Optimization of highly parameterized transformer architectures requires navigating non-convex loss landscapes with heterogeneous Hessian spectra. The literature has diversified into several families beyond the classical AdamW baseline [31]:

- **Variance reduction and momentum:** RAdam [29], Adan [54], ADOPT [49], AdEMAMix [37], MARS [60], and Cautious optimizers [26] address early-step instability, convergence guarantees, and dual-EMA history mixing.
- **Memory-efficient:** Adafactor [44], GaLore [65], Lion [9], and APOLLO [66] reduce optimizer-state memory through factorization, low-rank projection, or sign-based updates.
- **Schedule-free and parameter-free:** Schedule-Free AdamW [12] and Prodigy [35] absorb scheduler or learning-rate tuning into the optimizer dynamics.
- **Second-order and curvature-aware:** Shampoo [19], SOAP [51], and Sophia [28] incorporate approximate second-order information.
- **Geometry-oriented:** Muon [45] and Turbo-Muon [6] reshape update geometry through orthogonalization.

Detailed algorithmic descriptions, pseudocode, and a comprehensive complexity comparison are provided in Appendix A.

3 System Design and Architecture

3.1 Configuration-Centric Architecture

The core design choice in this repository is that experimentation is *schema first*. Instead of exposing a large number of loosely checked flags, the project forces model topology, optimizer family, training limits, and telemetry options through a single validated configuration document. This reduces ambiguity when reproducing results and makes it possible to compare many architectures under a consistent operational interface.

The authoritative contract is `configs/schema.yaml`. It enforces three top-level objects:

- `model_class`
- `model`
- `training`

Model Class Selection. The `model_class` field determines the architectural variant instantiated by the training pipeline. Three options are supported:

- **frankenstein:** Mixed-architecture encoder models supporting diverse attention mechanisms (standard, sigmoid, retentive, state-space, sparse, and gated mixers) with MoE (Mixture of Experts) and advanced features. Optimized for bidirectional encoder-style training with masked language modeling (MLM) objectives.
- **mini:** Simplified encoder variant designed for smaller-scale training scenarios. Provides reduced parameter overhead and faster iteration for experimentation and prototyping.
- **frankesteindecoder:** Autoregressive causal decoder for LLM-style next-token generation. Enables causal attention masking for sequential text generation tasks. When this class is selected, runtime enforces `mode='decoder'`.

Training Mode Selection. The `model.mode` field controls attention masking behavior across the model:

- **encoder:** Uses bidirectional attention where all tokens attend to all other tokens in the sequence. Suitable for masked language modeling (MLM) pre-training tasks where the model learns to predict randomly masked tokens based on full context.
- **decoder:** Uses causal masking where each token can only attend to previous tokens in the sequence. Required for autoregressive (AR) next-token prediction tasks such as language modeling and text generation. When `model_class='frankesteindecoder'`, the system automatically forces `mode='decoder'` at runtime.

This dual architecture support enables the system to handle both encoder-style pre-training (MLM on bidirectional contexts) and decoder-style generation (causal autoregressive prediction) through a unified configuration interface.

The `model.layer_pattern` supports legacy, sparse, and gated blocks:

- **Retentive Network (RetNet)** — internal reference: `sun_retentive_2023` — code name: `retnet, retnet_attn`
- **Mamba (Selective State Space Model)** — internal reference: `gu_mamba_2023` — code name: `mamba`
- **ODE-style Continuous Depth Block** — internal reference: `zhang_continuous_2021` — code name: `ode`
- **Titans Memory-Augmented Attention** — internal reference: `behrouz_titans_2025` — code name: `titan_attn`
- **Standard Softmax Attention** — internal reference: `vaswani_attention_2017` — code name: `standard_attn`
- **Sigmoid Self-Attention** — internal reference: `ramapuram_theory_2024` — code name: `sigmoid_attn`
- **Sparse Transformer** — internal reference: `child_sparse_transformer_2019` — code name: `sparse_transformer_attn`
- **Longformer** — internal reference: `beltagy_longformer_2020` — code name: `longformer_attn`
- **BigBird** — internal reference: `zaheer_bigbird_2020` — code name: `bigbird_attn`

- **SparseK Attention** — internal reference: `lou_sparsek_2024` — code name: `sparsek_attn`
- **Native Sparse Attention (NSA)** — internal reference: `yuan_nsa_2025` — code name: `nsa_attn`
- **SpargeAttn** — internal reference: `zhang_spargeattn_2025` — code name: `sparge_attn`
- **FASA (Frequency-aware Sparse Attention)** — internal reference: `wang_fasa_2026` — code name: `fasa_attn`
- **Gated Linear Attention (GLA)** — internal reference: `yang_gla_2023` — code name: `gla_attn`
- **DeltaNet** — internal reference: `yang_deltanet_2024` — code name: `deltanet_attn`
- **Gated DeltaNet** — internal reference: `yang_gated_deltanet_2024` — code name: `gated_deltanet_attn`
- **Gated DeltaNet-2** — internal reference: `hatamizadeh_gated_deltanet2_2026` — code name: `gated_deltanet2_attn`
- **HGRN2** — internal reference: `qin_hgrn2_2024` — code name: `hgrn2_attn`
- **Forgetting Transformer (FoX)** — internal reference: `lin_forgetting_transformer_2025` — code name: `fox_attn`
- **Gated Softmax Attention** — internal reference: `qiu_gated_attention_2025` — code name: `gated_softmax_attn`

which corresponds to current attention and sequence-mixer literature [48, 17, 63, 4, 41, 50, 11, 5, 61, 32, 59, 64, 53, 55, 56, 57, 20, 39, 27, 40]

The `training.optimizer.optimizer_class` supports a broad optimizer family: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `anon`, `apollo`, `apollo_mini`, and `q_apollo`.

3.2 Schema Scope and Validation Rules

The schema is strict: top-level and nested objects set `additionalProperties: false`. This guarantees that unknown keys fail fast instead of being silently ignored. The `training.optimizer.parameters` object is additionally constrained by optimizer-specific prefix rules through `allOf+if/then` pattern checks.

Normalization values currently accepted by schema are:

$$\text{norm_type} \in \{\text{layer_norm}, \text{dynamic_tanh}, \text{derf}\}$$

Thus, `rms_norm` is **not** a valid schema value in the current contract.

3.3 Complete Model Feature Inventory

Field	Type/Range	Meaning
<code>model_class</code>	enum	Architecture variant: <code>frankenstein</code> (mixed-architecture encoder), <code>mini</code> (simplified encoder), <code>frankesteindecoder</code> (autoregressive decoder).

Field	Type/Range	Meaning
<code>model.mode</code>	enum	Attention mode: <code>encoder</code> (bidirectional, for MLM) or <code>decoder</code> (causal, for AR). When <code>model_class='frankesteindecoder'</code> , forces <code>mode='decoder'</code> .
<code>vocab_size</code>	$\text{int} \geq 1$	Vocabulary size.
<code>hidden_size</code>	$\text{int} \geq 1$	Hidden dimension.
<code>num_layers</code>	$\text{int} \geq 1$	Physical layer count.
<code>num_loops</code>	$\text{int} \geq 1$	Logical loop count (looped blocks).
<code>num_heads</code>	$\text{int} \geq 1$	Attention heads.
<code>retention_heads</code>	$\text{int} \geq 1$	Retention heads for RetNet-style mixers.
<code>num_experts</code>	$\text{int} \geq 1$	MoE expert count.
<code>top_k_experts</code>	$\text{int} \geq 1$	Top- k expert routing in MoE.
<code>dropout</code>	float $[0, 1]$	Global dropout.
<code>layer_pattern</code>	array enum	Ordered block list: legacy (<code>retnet</code> , <code>retnet_attn</code> , <code>mamba</code> , <code>ode</code> , <code>titan_attn</code> , <code>standard_attn</code> , <code>sigmoid_attn</code>), sparse (<code>sparse_transformer_attn</code> , <code>longformer_attn</code> , <code>bigbird_attn</code> , <code>sparsek_attn</code> , <code>nsa_attn</code> , <code>sparge_attn</code> , <code>fasa_attn</code>), and gated (<code>gla_attn</code> , <code>deltanet_attn</code> , <code>gated_deltanet_attn</code> , <code>gated_deltanet2_attn</code> , <code>hgrn2_attn</code> , <code>fox_attn</code> , <code>gated_softmax_attn</code>).
<code>ode_solver</code>	enum	<code>rk4</code> or <code>euler</code> .
<code>ode_steps</code>	$\text{int} \geq 1$	ODE integration steps.
<code>use_bitnet</code>	bool	Enable low-bit BitLinear path.
<code>norm_type</code>	enum	<code>layer_norm</code> , <code>dynamic_tanh</code> , <code>derf</code> .
<code>use_factorized_embedding</code>	bool	Enable factorized embeddings.
<code>factorized_embedding_dim</code>	$\text{int} \geq 1$	Reduced embedding dimension for factorization.
<code>use_embedding_conv</code>	bool	Enable Conv1d over embedding stream.
<code>embedding_conv_kernel</code>	$\text{int} \geq 1$	Conv1d kernel size.
<code>hope_base</code>	float ≥ 0	HoPE base value (optional in schema).
<code>hope_damping</code>	float ≥ 0	HoPE damping (optional in schema).
<code>use_hope</code>	bool	Apply HoPE in <code>titan_attn</code> .
<code>use_moe</code>	bool	Enable MoE FFN routing path.
<code>ffn_hidden_size</code>	$\text{int} \geq 1$	FFN intermediate width.
<code>ffn_activation</code>	enum	<code>silu</code> or <code>gelu</code> .

Looped depth induced by schema is:

$$L_{\text{logical}} = \text{num_layers} \times \text{num_loops}$$

which is the configuration-level definition of looped blocks.

3.4 Training Task Types

The `training.task` field determines the training objective, working in conjunction with `model.mode` to define how the model learns:

Masked Language Modeling (MLM).

- **Mode:** Requires `mode='encoder'` for bidirectional attention.
- **Objective:** Randomly mask tokens in input sequence (typically 15%) and train model to predict masked tokens based on full bidirectional context.

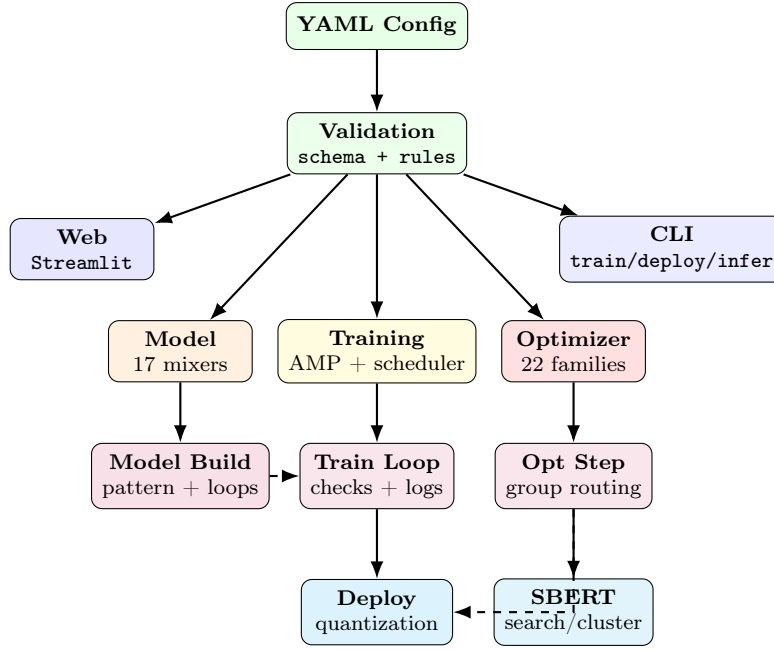


Figure 1: System architecture: configuration flows through validation, partitions into model/training/optimizer, executes runtime, produces deployment/SBERT artifacts.

- **Use Case:** Pre-training encoders for representation learning, following BERT-style methodology [14]. Model learns bidirectional representations capturing context from both left and right.
- **Configuration:** Uses `mlm_probability` parameter to control masking fraction.

Autoregressive (AR) Next-Token Prediction.

- **Mode:** Requires `mode='decoder'` for causal masking.
- **Objective:** Train model to predict next token in sequence given all previous tokens only.
- **Use Case:** Language generation and LLM-style tasks following GPT methodology. Model learns sequential dependencies with causal attention where each token can only attend to preceding tokens.
- **Model Class:** Typically uses `model_class='frankesteindecoder'` for autoregressive decoder architectures.

This dual task support enables unified experimentation across both encoder-style pre-training (MLM for bidirectional understanding) and decoder-style generation (AR for sequential text production) within the same codebase.

3.5 Complete Training Feature Inventory

Field	Type/Range	Meaning
<code>batch_size</code>	<code>int</code> ≥ 1	Loader batch size.
<code>dataloader_workers</code>	<code>int</code> ≥ 0	PyTorch dataloader workers.
<code>max_length</code>	<code>int</code> ≥ 1	Sequence length cap.
<code>task</code>	<code>enum</code>	Training objective: <code>mlm</code> (masked language modeling) or <code>sbert</code> (sentence embedding).

Field	Type/Range	Meaning
mlm_probability	float [0, 1]	MLM masking probability (applies only when <code>task='mlm'</code>).
max_samples	int ≥ 1	Maximum streamed samples.
dataset_batch_size	int ≥ 1	Internal streaming dataset chunk size.
num_workers	int ≥ 0	Streaming dataset workers.
cache_dir	string	Dataset cache directory.
local_parquet_dir	string	Optional local parquet path.
prefer_local_cache	bool	Prefer local cache when available.
stream_local_parquet	bool	Stream from local parquet mode.
use_amp	bool	Mixed precision toggle.
gradient_accumulation_steps	int ≥ 1	Effective batch through accumulation.
optimizer	object	Contains <code>optimizer_class</code> and prefixed <code>parameters</code> .
scheduler_total_steps	int ≥ 1	Scheduler horizon.
scheduler_warmup_ratio	float [0, 1]	Warmup ratio.
scheduler_type	enum	cosine, constant, linear_warmup_then_constant.
grad_clip_max_norm	float ≥ 0	Global norm clipping threshold.
inf_post_clip_threshold	float ≥ 0	Exploding-gradient guard threshold after clipping.
max_nan_retries	int ≥ 0	Retry budget for NaN/Inf instability.
checkpoint_every_n_steps	int ≥ 1	Rolling checkpoint frequency.
max_rolling_checkpoints	int ≥ 1	Number of rolling checkpoints to keep.
num_best_checkpoints	int ≥ 1	Number of best checkpoints tracked.
nan_check_interval	int ≥ 1	NaN/Inf check cadence.
log_gradient_stats	bool	Enable gradient statistics logging.
gradient_log_interval	int ≥ 1	Gradient logging cadence.
csv_log_path	string	Step-level CSV output path.
csv_rotate_on_schema_change	bool	Rotate CSV if logging schema changes.
gpu_metrics_backend	enum	nvml or none.
nvml_device_index	int ≥ 0	Device index for NVML telemetry.
enable_block_grad_norms	bool	Include per-block gradient norm telemetry.
telemetry_log_interval	int ≥ 1	Heavy telemetry interval (optimizer steps).
use_galore	bool	Enable GaLore strategy.
galore_rank	int ≥ 1	GaLore low-rank projection dimension.
galore_update_interval	int ≥ 1	Projection refresh interval.
galore_scale	float ≥ 0	Gradient scaling in projected space.
galore_max_dim	int ≥ 1	Maximum tensor dimension for GaLore projection.

3.6 Optimizer Prefix Contract (Full)

Supported `optimizer_class` values are: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `apollo`, `apollo_mini`, `q_apollo`.

Shared per-group suffix families (all prefixed by optimizer name) are:

- LR groups: `lr_embeddings`, `lr_norms`, `lr_ode`, `lr_retnet`, `lr_mamba`, `lr_attention`, `lr_other`
- Weight decay groups: `wd_embeddings`, `wd_norms`, `wd_ode`, `wd_retnet`, `wd_mamba`, `wd_attention`, `wd_other`
- Beta groups: `betas_embeddings`, `betas_norms`, `betas_ode`, `betas_retnet`, `betas_mamba`, `betas_attention`, `betas_other`

- Epsilon groups: `eps_embeddings`, `eps_norms`, `eps_ode`, `eps_retnet`, `eps_mamba`, `eps_attention`, `eps_other`

Optimizer-specific global suffixes:

- `sgd_momentum`: `momentum`, `nesterov`
- `adafactor`: `beta2_decay`, `clip_threshold`, `eps1`, `eps2`
- `galore_adamw`: `rank`, `update_proj_gap`
- `prodigy`: `d_coef`
- `sophia`: `rho`, `update_k`
- `muon` / `turbo_muon`: `momentum`, `nesterov`, `ns_steps`, `ns_eps`
- `cautious_adamw`: `cautious_clip`
- `apollo`: `rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`
- `apollo_mini`: `update_proj_gap`, `scale`, `proj_type`, `scale_front`, `disable_nl`
- `q_apollo`: `rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`, `quant_bits`

All other classes in the list above accept only prefixed shared groups.

3.7 Training Safety and Runtime Semantics

Schema-level safety features include accumulation, clipping, post-clip explosion checks, and NaN retries:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^K g_i, \quad K = \text{gradient_accumulation_steps}$$

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min \left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon} \right), \quad \tau = \text{grad_clip_max_norm}$$

then overflow guards use `inf_post_clip_threshold` and retry logic bounded by `max_nan_retries`.

The complete pseudocode for the schema-driven training step—including gradient accumulation, global-norm clipping, post-clip explosion guards, bounded NaN/Inf retries, scheduler dispatch, and rolling/best checkpoint rotation—is deferred to Appendix G.

3.8 Normalization Variants

Normalization controls activation scale across depth and is also a schema compatibility question, since only `layer_norm`, `dynamic_tanh`, and `derf` are accepted `norm_type` values; `rms_norm` is not currently a valid schema value. The mathematical formulations of LayerNorm, RMSNorm, Dynamic Tanh (DyT) [67], and Dynamic Erf (Derf) [8], together with a comparison table, are detailed in Appendix G.

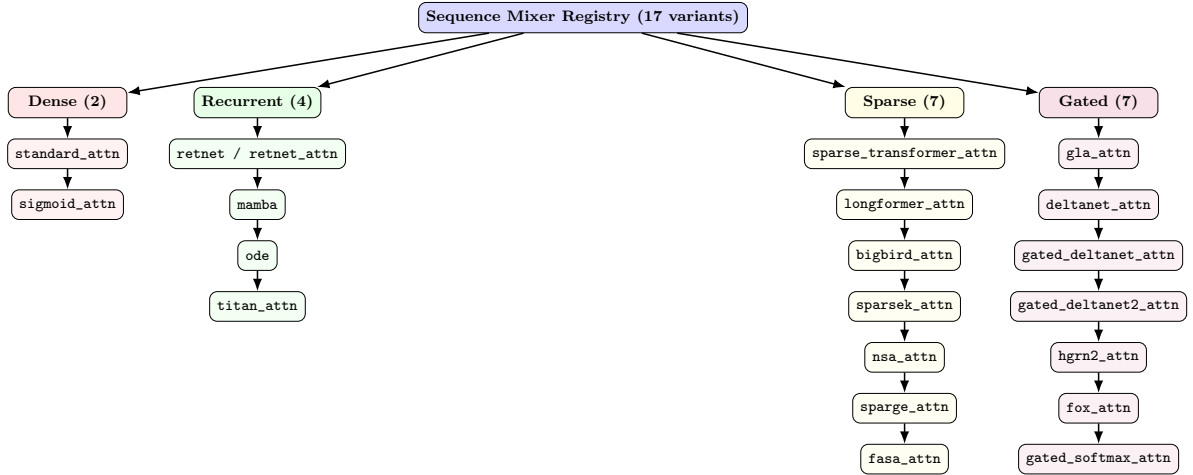


Figure 2: Comprehensive taxonomy of seventeen supported sequence mixer architectures. Dense baselines provide full global routing at quadratic cost. Recurrent architectures enable constant-time inference through state compression. Sparse variants reduce complexity through structured patterns or token selection. Gated mechanisms introduce data-dependent control over memory retention and forgetting.

4 Architecture Taxonomy and Implementation

4.1 Attention and Sequence-Mixer Families

This system implements seventeen distinct sequence mixer architectures organized into five functional categories reflecting research trends in sequence modeling design. The taxonomical organization reflects evolving understanding of how to balance expressivity, computational efficiency, and memory constraints.

1. **Dense Attention Baselines:** Standard softmax attention and sigmoid attention provide full global contextualization at quadratic computational cost, serving as reference baselines for comparison with more efficient alternatives.
2. **Recurrent and Retentive Architectures:** RetNet, Mamba, ODE-style blocks, and Titans maintain state representations enabling $\mathcal{O}(1)$ inference cost while preserving expressivity through recurrent dynamics, selective parameters, or test-time memory adaptation.
3. **Sparse Attention Patterns:** Seven sparse variants (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) reduce quadratic complexity through structured sparsity, token selection, or training-free pruning strategies.
4. **Gated Memory Mechanisms:** Seven gated architectures (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax) introduce data-dependent control over memory retention, forgetting, and update strength.

4.2 Dense and Recurrent Families

The dense baselines (standard softmax [50] and sigmoid [41]) and the recurrent/retentive family (RetNet [48], Mamba [17], ODE-style continuous blocks [63], and Titans test-time memory [4]) provide the foundational expressiveness-versus-efficiency tradeoff. Mathematical formulations, parallel and recurrent forms, algorithmic pseudocode, and an architectural comparison table are collected in Appendix B. The pattern-driven mixer dispatch algorithm (including the training-free enforcement policy for `fasa_attn` and `sparge_attn`) is likewise detailed there.

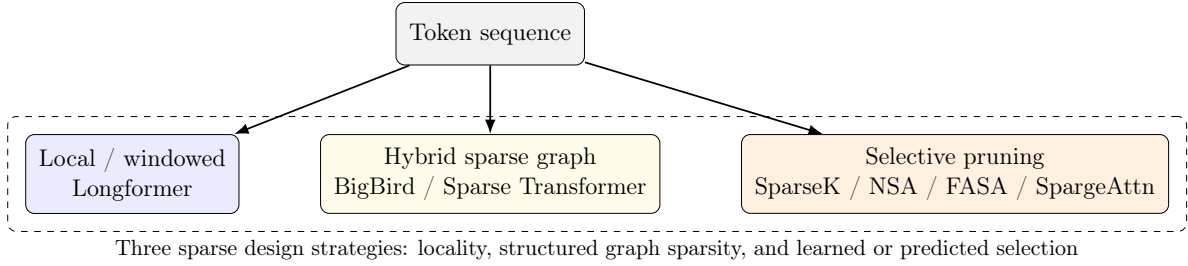


Figure 3: Conceptual map of sparse attention design choices used in the codebase. Different methods reduce cost by restricting neighborhoods, constructing sparse graphs, or selecting only high-value tokens/blocks.

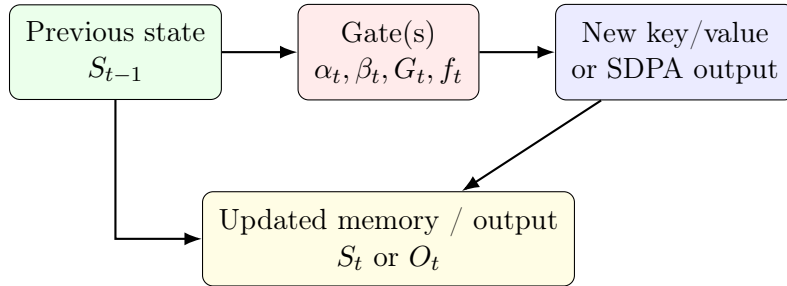


Figure 4: Generic gating template. A gate can decay existing memory, regulate write strength, or modulate dense attention outputs, depending on the block family.

4.3 Latent and Low-Rank Attention Family

A complementary family compresses the per-token key–value state into a low-rank latent: Multi-Head Latent Attention (MLA), Group-Query Latent Attention (GQLA), Multi-Head Low-Rank Attention (MLRA), Tucker Attention, Interleaved Head Attention (IHA), Grouped-head laTenT Attention (GTA), and Multi-head Temporal Latent Attention (MTLA). These methods unify grouped-query, latent, factorised, and temporal-latent designs under a common low-rank bottleneck. Full formulations, pseudocode, and a comparison table are collected in Appendix C.

4.4 Sparse Attention Extensions

The mixer registry includes seven sparse blocks: `sparse_transformer_attn`, `longformer_attn`, `bigbird_attn`, `sparsek_attn`, `nsa_attn`, `sparge_attn`, and `fasa_attn` [11, 5, 61, 32, 59, 64, 53]. The implementation enforces an explicit execution policy for training-free sparse methods: `fasa_attn` and `sparge_attn` are eval/inference-only and raise runtime errors if used while the model is in training mode. Mathematical formulations, algorithmic pseudocode, per-block characteristics, and a comparison table are collected in Appendix D.

4.5 Gated Attention Extensions

The mixer registry includes eight gated blocks: `gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `gated_deltanet2_attn`, `hgrn2_attn`, `fox_attn`, and `gated_softmax_attn` [55, 56, 57, 20, 39, 27, 40]. The unifying idea is that gating controls *what information survives*: some gates act on recurrent state updates (GLA, DeltaNet variants, HGRN2), while others modify the full attention path itself (FoX and Gated Softmax). This makes gating especially useful when the model must trade off recall, recency, and bounded memory. Mathematical formulations, algorithmic pseudocode, a generic gating template, and a comparison table are collected in Appendix E.

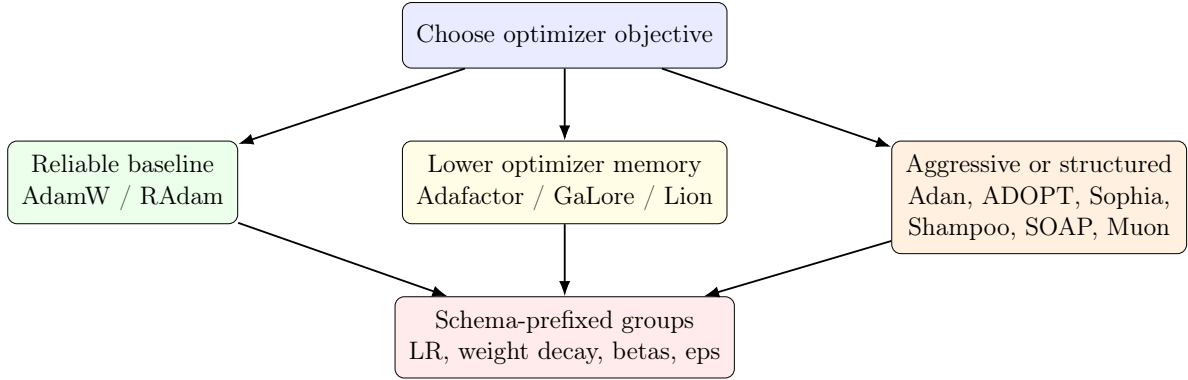


Figure 5: Optimizer selection in practice: the class determines the update rule, while the schema controls how hyperparameters are applied across embeddings, norms, recurrent blocks, attention blocks, and other parameters.

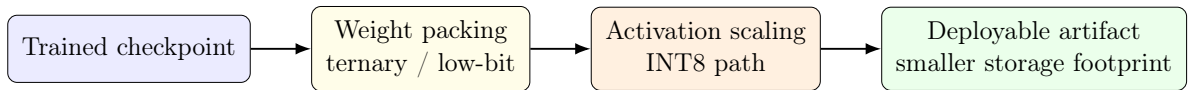


Figure 6: Deployment path from a trained checkpoint to a compact artifact. The codebase treats quantization as a deploy-stage transformation rather than a separate model family.

5 Optimizer Families and Training Dynamics

Optimization of highly parameterized transformer architectures presents significant challenges due to non-convex loss landscapes, saddle points, and block heterogeneity across parameter groups. This system addresses these challenges through a unified framework supporting twenty-three optimizer families spanning six algorithmic categories: (1) classical baselines (SGD, AdamW), (2) advanced momentum and variance reduction (Adan, ADOPT, AdEMAMix, MARS, Cautious), (3) memory-efficient variants (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), (4) schedule-free and parameter-free methods (Schedule-Free AdamW, Prodigy) plus large-batch scaling through LAMB, (5) curvature-aware and second-order (Sophia, Shampoo, SOAP), (6) geometry-oriented (Muon, Turbo-Muon), and (7) adaptivity-tunable optimizers (Anon). Detailed algorithmic descriptions, pseudocode, and a memory/complexity comparison table for all optimizers are provided in Appendix A.

6 Quantization and Deployment

The deploy stack uses ternary weight packing plus INT8 activation quantization for efficient artifacts.

6.1 Ternary Quantization and Activation Scaling

The deploy-stage transforms a trained checkpoint into a compact artifact via BitNet-style ternary weight packing [52], INT8 activation scaling, and 1.58-bit storage approximation aligned with lightweight deployment goals [43, 7]. The mathematical formulations of the ternary weight scaling, the INT8 activation quantization/dequantization, and the FP32/FP16/1.58-bit size estimates are deferred to Appendix G.

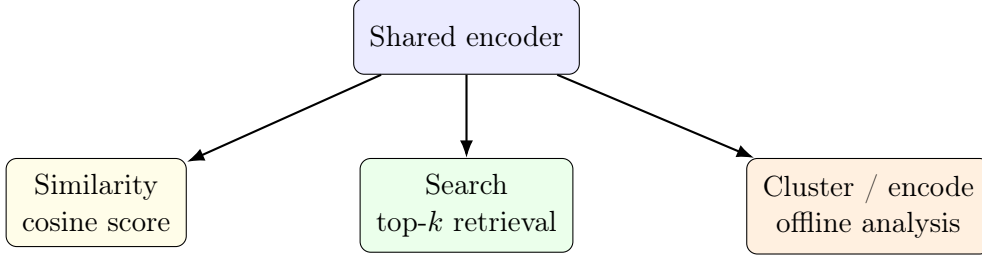


Figure 7: SBERT workflow reuse. A single encoder supports online pair scoring, corpus retrieval, clustering, and persistent embedding export.

7 SBERT Downstream Tasks

Sentence embedding is built on Siamese-style training [42]. For sentence pair (s_1, s_2) with embeddings (e_1, e_2) :

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\| \|e_2\|}$$

and regression-style cosine loss:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

with $y \in [-1, 1]$ in this pipeline.

Supported downstream modes:

- **Similarity**: pairwise score between two sentences.
- **Search**: top- k nearest neighbors over a corpus.
- **Cluster**: grouping embeddings (e.g., k-means).
- **Encode**: persistent embedding export for later retrieval.

The pseudocode for the SBERT inference mode router—dispatching among similarity, search, cluster, and encode modes over a shared encoder—is deferred to Appendix G.

8 Summary Tables

8.1 Attention and Sequence-Mixer Summary

Type		Core Equation	Train	Infer	Notes
Standard At-		$\text{softmax}(QK^\top / \sqrt{d_k})V$	$\mathcal{O}(n^2d)$	$\mathcal{O}(n)/\text{step}$	Baseline expressive global routing [50].
Sigmoid Atten-		$\sigma(QK^\top / \sqrt{d_k} + b)V$	$\mathcal{O}(n^2d)$	$\mathcal{O}(n)/\text{step}$	Element-wise gating; often needs stabilization norm [41].
RetNet		$(QK^\top \odot D)V$	$\mathcal{O}(n^2d)$ or chunkwise	$\mathcal{O}(1)/\text{step}$	Parallel/recurrent dual form with decay retention [48].
Mamba		$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$	$\mathcal{O}(nd)$	$\mathcal{O}(1)/\text{step}$	Selective state-space with hardware-aware scan [17].
ODE-style block		$\frac{dh}{dt} = f_\theta(h, t)$	solver-dependent	solver-dependent	Continuous-depth interpretation; RK integration [63].

Type	Core Equation	Train	Infer	Notes
Titans memory	$M_t = (1 - \alpha_t)M_{t-1} + S_t$	approx. $\mathcal{O}(nd)$	retrieval- centric	Test-time memory updates with surprise-driven dynamics [4].

8.2 Optimizer Summary

Optimizer	Family	State	Cost	Key Idea	Ref
SGD+Momentum	Classical first-order	Low		Momentum-accelerated baseline with minimal state	[38]
AdamW	Adaptive first/second moment	High		Decoupled weight decay baseline	[31]
RAdam	Adaptive variance-corrected	High		Rectifies early adaptive variance	[29]
Adan	Momentum + variance reduction	High		Nesterov-style adaptive update	[54]
ADOPT	Adam variant	High		Reordered updates with improved convergence guarantees	[49]
AdEMAMix	Multi-EMA adaptive	High		Mixes short and long horizon EMAs	[37]
MARS	Variance-reduced preconditioned	High		Recursive momentum correction	[60]
Cautious AdamW	Masked momentum	High		Apply updates only on sign-consistent directions	[26]
LAMB	Layer-wise adaptive moments	High		Trust-ratio scaling for very large-batch training	[58]
Schedule-free AdamW	Scheduler-free adaptive	High		Remove explicit LR schedule dependence	[12]
Adafactor	Memory-efficient adaptive	Medium		Factorized second moments for matrix tensors	[44]
GaLore AdamW	Low-rank gradient projection	Medium		Optimize in projected low-rank gradient space	[65]
APOLLO	Low-rank adaptive projection	Medium		Structured scaling from projected Adam-style moments	[66]
APOLLO-Mini	Rank-1 adaptive projection	Low		Tensor-wise scaled APOLLO variant for extreme memory savings	[66]
Q-APOLLO	Quantized low-rank projection	Low		Quantized APOLLO state for ultra-low-memory training	[66]
Anon	Adaptivity-tunable Adam-like	High		Continuously tunable adaptivity $\gamma \in \mathbb{R}$ with incremental delay update (IDU) for convergence	[2]
Prodigy	Parameter-free adaptation	Medium		Distance-adaptive step calibration	[35]
Lion	Sign momentum	Low		Momentum sign update, reduced state	[9]
Sophia	Approx. second-order	Medium		Diagonal Hessian preconditioning with clipping	[28]
Shampoo	Matrix preconditioner	High		Kronecker-structured second-order statistics	[19]
SOAP	Shampoo + Adam basis	High		Adam-like tracking in preconditioner eigenbasis	[51]
Muon	Orthogonality-based	Medium		Orthogonalized matrix updates	[45]

Optimizer	Family		State Cost	Key Idea		Ref
Turbo-Muon	Accelerated	or-	Medium	Preconditioned speedup	Newton-Schulz	[6]
	thogonalization					

9 Discussion

The design choices in Transformer Encoder Frankenstein reflect several engineering and research tensions in modern deep learning tooling.

9.1 Schema-Driven Design Trade-offs

The schema-first approach provides significant reproducibility benefits by enforcing explicit contracts and failing fast on invalid configurations. However, this approach also introduces rigidity: adding new architectures or optimizers requires schema extensions rather than loose command-line arguments. The prefixed hyperparameter system enables fine-grained control but increases configuration complexity for users accustomed to simpler interfaces.

The decision to enforce `additionalProperties: false` at all schema levels eliminates silent parameter swallowing that has plagued earlier configuration systems, but this strictness requires careful schema maintenance when extending system capabilities. Each new attention mechanism or optimizer variant must be properly integrated into the validation framework, including schema field definitions with appropriate types and constraints, prefixed hyperparameter mapping for optimizer-specific groups, default values aligned with research best practices, and documentation strings for web interface rendering.

9.2 Architectural Coverage and Gaps

The seventeen implemented mixer architectures span major research directions in sequence modeling, but certain gaps remain. The system lacks recent hybrid architectures such as Griffin [46] and Jamba [18], which combine gating with state-space models. MoE (Mixture of Experts) routing is implemented for FFN layers but not for attention computation, where recent work has shown benefits [25].

The sparse attention coverage is comprehensive, but implementation of training-free methods (FASA, SpargeAttn) raises runtime errors during training, reflecting architectural constraints: these methods require pretrained checkpoints from full-attention models or specific fine-tuning procedures that are not currently automated.

The gated mechanism coverage is strong across major categories (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).

9.3 Optimizer Landscape Fragmentation

The support for twenty-two optimizer families across six algorithmic categories demonstrates comprehensiveness but also highlights the fragmented state of optimization research. Users face significant decision complexity when choosing among variance-reduction methods (Adan, MARS), memory-efficient variants (GaLore, Adafactor, APOLLO), and curvature-aware approaches (Sophia, Shampoo). The prefixed hyperparameter system, while powerful, requires understanding of which parameters are relevant for each optimizer class.

The implementation quality varies across optimizers: classical methods (AdamW, SGD with momentum) are highly optimized in PyTorch, while newer methods (Muon, Turbo-Muon, SOAP) may require custom implementations that affect numerical stability and performance characteristics.

9.4 Deployment and Production Considerations

The quantization pipeline demonstrates practical deployment concerns but makes specific engineering trade-offs. Ternary weight packing reduces storage to approximately 1.58 bits per parameter, but this aggressive compression may degrade performance, especially for smaller models where quantization error is more significant. The current implementation applies quantization uniformly across parameter types.

The SBERT workflows provide practical utility for semantic similarity and retrieval tasks, but implementation assumes standard pooling strategies (CLS token, mean pooling). Recent advances such as Matryoshka embeddings [36] and contrastive learning refinements [21] are not yet incorporated.

9.5 Integration and Extensibility Challenges

The current codebase structure, while functional, presents maintenance challenges as architecture and optimizer families expand. The dispatcher pattern for mixer selection and optimizer routing handles extensibility but risks becoming a “kitchen sink” of conditional logic. Future versions would benefit from plugin-based architectures where new mixers and optimizers could be registered declaratively rather than modifying core dispatch logic.

The web configuration interface provides significant usability improvements but introduces deployment complexity: running Streamlit alongside training jobs requires additional resources and infrastructure considerations that may not be appropriate for all environments, particularly HPC clusters without web access.

10 Conclusion

Transformer Encoder Frankenstein presents a unified, configuration-driven experimentation platform addressing critical challenges in modern deep learning research: architectural fragmentation across dense attention, recurrent models, sparse patterns, and gated mechanisms; optimizer landscape complexity spanning classical baselines, variance-reduction methods, memory-efficient variants, schedule-free approaches, curvature-aware algorithms, and geometry-oriented methods; and end-to-end deployment workflows spanning quantization and sentence embedding applications.

The system’s primary contributions are:

1. **Schema-First Design:** A strict YAML-based configuration contract with validation and prefixed hyperparameter routing enabling reproducible experiments across seventeen mixer architectures and twenty-three optimizer families.
2. **Comprehensive Architecture Support:** Implementation spanning major research categories including dense baselines (standard, sigmoid attention), recurrent alternatives (RetNet, Mamba, ODE-style, Titans), sparse attention (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA), and gated mechanisms (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).
3. **Unified Optimizer Framework:** Prefixed hyperparameter groups enabling fine-grained control over embeddings, normalization layers, recurrent blocks, attention weights, and FFN parameters across classical baselines (SGD+Momentum, AdamW), variance-reduction (Adan, ADOPT, AdEMAMix, MARS, Cautious), memory-efficient (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), large-batch and schedule simplification (LAMB, Schedule-Free AdamW, Prodigy), curvature-aware (Sophia), second-order (Shampoo, SOAP), and geometry-oriented (Muon, Turbo-Muon) optimizers.

4. **End-to-End Workflows:** Integrated deployment pipeline supporting ternary weight packing and INT8 activation quantization; SBERT-inspired training and inference for semantic similarity, retrieval, and clustering tasks.
5. **Interactive Configuration:** Streamlit-based web interface providing schema-driven form generation, real-time validation, inline documentation, and CLI command synthesis.

This system enables rapid experimental iteration while maintaining reproducibility through strict configuration contracts. By consolidating diverse research contributions into a unified toolkit, it lowers barriers to exploring novel architectures and optimization strategies, particularly for researchers who may lack resources to implement and validate each variant independently.

10.1 Limitations and Future Directions

Several limitations and promising directions for future work emerge from this system’s design and implementation:

1. **Architecture Integration:** Recent hybrid architectures (Griffin, Jamba, Mamba-X) demonstrate benefits of combining multiple mechanisms into unified blocks. Future versions should integrate these architectures and explore systematic composition patterns.
2. **Advanced Quantization:** Current implementation uses uniform ternary packing across all parameters. Research on layer-wise, channel-wise, and importance-aware quantization suggests more sophisticated strategies could improve quality-efficiency trade-offs.
3. **Plugin-Based Extensibility:** The current dispatch pattern becomes increasingly complex with each new addition. A plugin architecture allowing declarative registration of new mixers, optimizers, and normalization methods would improve maintainability and reduce risk of bugs in core dispatch logic.
4. **Automated Hyperparameter Optimization:** The schema supports extensive hyperparameter spaces, but users must manually explore these spaces. Integration with Bayesian optimization, multi-armed bandit strategies, or gradient-based hyperparameter tuning could automate effective configuration discovery.
5. **Production Deployment:** The web interface improves usability but may not be appropriate for all deployment environments. Headless configuration modes, API-based configuration management, or improved CLI ergonomics could serve HPC and production workflows.
6. **Evaluation Benchmarking:** While the system enables training with diverse architectures, comprehensive benchmarking comparing performance across mixers and optimizers on standardized tasks would provide valuable guidance for configuration selection.
7. **Training Stability Guarantees:** Current implementation includes NaN/Inf guards and gradient clipping, but formal analysis of stability conditions for different mixer-optimizer combinations, particularly with looped blocks and aggressive quantization, remains open.
8. **Multimodal and Task-Specific Extensions:** The current design focuses on sequence modeling. Extensions for vision-language models, multimodal architectures, and task-specific fine-tuning workflows (e.g., instruction tuning, RLHF) would broaden applicability.

The research trajectory of sequence modeling continues toward hybrid approaches that combine strengths of multiple paradigms—compression from recurrence, selectivity from attention, gating for memory management, and sparsity for efficiency. A unified experimentation platform

like Transformer Encoder Frankenstein is increasingly valuable as this convergence accelerates, enabling researchers to systematically explore this expanding design space with reproducible, well-engineered infrastructure.

Bibliography

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyansky, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL <https://arxiv.org/abs/2305.13245>.
- [2] Anonymous authors. Anon: Extrapolating adaptivity beyond sgd and adam, 2026. URL <https://arxiv.org/abs/2605.02317>.
- [3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- [4] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. URL <https://arxiv.org/abs/2501.00663>. Version Number: 1.
- [5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [6] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbo-muon: Accelerating orthogonality-based optimization with pre-conditioning. URL <https://arxiv.org/abs/2512.04632>. Version Number: 1.
- [7] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory. URL <http://arxiv.org/abs/2502.10001>.
- [8] Mingzhi Chen, Taiming Lu, Jiachen Zhu, Mingjie Sun, and Zhuang Liu. Stronger normalization-free transformers, . URL <http://arxiv.org/abs/2512.10938>.
- [9] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, . URL <https://arxiv.org/abs/2302.06675>. Version Number: 4.
- [10] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL <https://arxiv.org/abs/2601.07372>.
- [11] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL <https://arxiv.org/abs/1904.10509>.
- [12] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. URL <https://arxiv.org/abs/2405.15682>. Version Number: 4.
- [13] Keqi Deng and Philip C. Woodland. Multi-head temporal latent attention, 2025. URL <https://arxiv.org/abs/2505.13544>.
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. URL <http://arxiv.org/abs/1810.04805>.

- [15] Sai Surya Duvvuri, Chanakya Ekbote, Rachit Bansal, Rishabh Tiwari, Devvrit Khatri, David Brandfonbrener, Paul Liang, Inderjit Dhillon, and Manzil Zaheer. Interleaved head attention, 2026. URL <https://arxiv.org/abs/2602.21371>.
- [16] Yaosheng Fu, Guangxuan Xiao, Xin Dong, Song Han, and Oreste Villa. Sparda: Sparse decoupled attention for efficient long-context llm inference, 2026. URL <https://arxiv.org/abs/2606.04511>.
- [17] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. URL <https://arxiv.org/abs/2312.00752>. Version Number: 2.
- [18] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>.
- [19] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. URL <https://arxiv.org/abs/1802.09568>. Version Number: 2.
- [20] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated deltanet-2: Decoupling erase and write in linear attention, 2026. URL <https://arxiv.org/abs/2605.22791>.
- [21] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschiot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL <https://arxiv.org/abs/2004.11362>.
- [22] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>.
- [23] Timon Klein, Jonas Kusch, Sebastian Sager, Stefan Schnake, and Steffen Schotthöfer. Tucker attention: A generalization of approximate attention mechanisms, 2026. URL <https://arxiv.org/abs/2603.30033>.
- [24] Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. Minimax sparse attention, 2026. URL <https://arxiv.org/abs/2606.13392>.
- [25] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL <https://arxiv.org/abs/2103.16716>.
- [26] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. URL <https://arxiv.org/abs/2411.16085>. Version Number: 4.
- [27] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL <https://arxiv.org/abs/2503.02130>.

- [28] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, . URL <https://arxiv.org/abs/2305.14342>. Version Number: 4.
- [29] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, . URL <https://arxiv.org/abs/1908.03265>. Version Number: 4.
- [30] Songtao Liu, Hongwu Peng, Zhiwei Zhang, Zhengyu Chen, and Yue Guo. Multi-head low-rank attention, 2026. URL <https://arxiv.org/abs/2603.02188>.
- [31] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. URL <https://arxiv.org/abs/1711.05101>. Version Number: 3.
- [32] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL <https://arxiv.org/abs/2406.16747>.
- [33] Sushant Mehta, Raj Dandekar, Rajat Dandekar, and Sreedath Panat. Latent multi-head attention for small language models, 2025. URL <https://arxiv.org/abs/2506.09342>.
- [34] Fanxu Meng. Gqla: Group-query latent attention for hardware-adaptive large language model decoding, 2026. URL <https://arxiv.org/abs/2605.15250>.
- [35] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner. URL <https://arxiv.org/abs/2306.06101>. Version Number: 4.
- [36] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL <https://arxiv.org/abs/2205.13147>.
- [37] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older. URL <https://arxiv.org/abs/2409.03137>. Version Number: 2.
- [38] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.
- [39] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL <https://arxiv.org/abs/2404.07904>.
- [40] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL <https://arxiv.org/abs/2505.06708>.
- [41] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention. URL <https://arxiv.org/abs/2409.04431>. Version Number: 2.
- [42] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. URL <http://arxiv.org/abs/1908.10084>.
- [43] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications. URL <http://arxiv.org/abs/2601.03290>.
- [44] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. URL <https://arxiv.org/abs/1804.04235>. Version Number: 1.

- [45] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon. URL <https://arxiv.org/abs/2505.23737>. Version Number: 1.
- [46] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL <https://arxiv.org/abs/2402.19427>.
- [47] Luoyang Sun, Cheng Deng, Jiwen Jiang, Xinjian Wu, Haifeng Zhang, Lei Chen, Lionel Ni, and Jun Wang. Gta: Grouped-head latent attention, 2025. URL <https://arxiv.org/abs/2506.17286>.
- [48] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. URL <https://arxiv.org/abs/2307.08621>. Version Number: 4.
- [49] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any β_2 with the optimal rate. URL <https://arxiv.org/abs/2411.02853>. Version Number: 3.
- [50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. URL <https://arxiv.org/abs/1706.03762>. Version Number: 7.
- [51] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam. URL <https://arxiv.org/abs/2409.11321>. Version Number: 2.
- [52] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. URL <http://arxiv.org/abs/2310.11453>.
- [53] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL <https://arxiv.org/abs/2602.03152>.
- [54] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. URL <https://arxiv.org/abs/2208.06677>. Version Number: 5.
- [55] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL <https://arxiv.org/abs/2312.06635>.
- [56] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL <https://arxiv.org/abs/2406.06484>.
- [57] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL <https://arxiv.org/abs/2412.06464>.
- [58] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes. URL <https://arxiv.org/abs/1904.00962>. Version Number: 3.
- [59] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL <https://arxiv.org/abs/2502.11089>.

- [60] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models. URL <https://arxiv.org/abs/2411.10438>. Version Number: 4.
- [61] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL <https://arxiv.org/abs/2007.14062>.
- [62] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. URL <http://arxiv.org/abs/1910.07467>.
- [63] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. 35(16):14393–14401. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL <https://ojs.aaai.org/index.php/AAAI/article/view/17692>.
- [64] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL <https://arxiv.org/abs/2502.18137>.
- [65] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. URL <https://arxiv.org/abs/2403.03507>. Version Number: 2.
- [66] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance, 2025. URL <https://arxiv.org/abs/2412.05270>.
- [67] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. URL <http://arxiv.org/abs/2503.10622>.
- [68] Lianghui Zhu, Yuxin Fang, Bencheng Liao, Shijie Wang, Tianheng Cheng, Zilong Huang, Chen Chen, Lai Wei, Yutao Zeng, Ya Wang, Yi Lin, Yu Li, and Xinggang Wang. Mixture-of-depths attention, 2026. URL <https://arxiv.org/abs/2603.15619>.

A Annex A: Optimizer Families

A.1 Memory and Complexity Comparison

Table 5 summarizes the memory overhead (number of state buffers per parameter), per-step computational complexity, and key hyperparameters for all supported optimizers. Memory overhead is expressed in terms of the number of state tensors maintained per model parameter, where each tensor has the same shape as the parameter.

Optimizer	State Buffers	Per-Step Cost	Key Hyperparams
SGD+Momentum	1 (m)	$\mathcal{O}(n)$	lr, momentum, wd
AdamW	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
RAdam	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
Adan	3 (m, v, s)	$\mathcal{O}(n)$	lr, $\beta_1, \beta_2, \beta_3$, eps, wd
ADOPT	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
AdEMAMix	3 (m_1, m_2, v)	$\mathcal{O}(n)$	lr, $\beta_1, \beta_2, \beta_3$, eps, wd

contin

Optimizer	State Buffers	Per-Step Cost	Key Hyperparams
MARS	3 (m, v, z)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, γ
Cautious AdamW	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
LAMB	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
Schedule-Free	3 (z, x, n)	$\mathcal{O}(n)$	lr, β_1, β_2 , wd
Adafactor	1–2 (row/col)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, factored
GaLore	2 (m, v) + SVD/proj	$\mathcal{O}(nr)$	lr, rank r , β_1, β_2 , eps, wd
Prodigy	3 (m, v, d)	$\mathcal{O}(n)$	β_1, β_2 , eps, wd, d_0
Lion	1 (m)	$\mathcal{O}(n)$	lr, β_1, β_2 , wd
Shampoo	$2d (L_i, R_i)$	$\mathcal{O}(n^{1+2/d})$	lr, eps, wd, matrix eps
SOAP	$2d + 2 (m, v)$	$\mathcal{O}(n^{1+1/d})$	lr, β_1, β_2 , eps, wd, shampoo eps
Sophia	3 (m, v, h)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, k
Muon	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k
Turbo-Muon	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k
APOLLO	2 low-rank (m_R, v_R) + proj	$\mathcal{O}(nr)$	lr, rank r , update gap, scale, betas, eps
APOLLO-Mini	2 rank-1 (m_R, v_R) + proj	$\mathcal{O}(n)$	lr, update gap, scale, betas, eps, wd
Q-APOLLO	2 quantized low-rank + proj	$\mathcal{O}(nr)$	lr, rank r , update gap, scale, quant bits
Anon	3 (m, v , fixed_lr)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, γ

Table 5: Memory and complexity comparison of all supported optimizers. n = number of parameters, d = tensor dimensionality, r = low-rank dimension, k = Newton–Schulz iteration count. State buffers lists the number of optimizer state tensors maintained per parameter group. Per-step cost focuses on the dominant additional cost beyond the gradient computation itself.

A.2 The Evolution of Optimization in Neural Networks

The optimizer survey frames transformer optimization as a response to three structural pressures: non-convex loss landscapes, severe curvature heterogeneity across parameter blocks, and the memory cost of storing optimizer state for very large models. The report argues that the field has diverged into several trajectories: adaptive first-order baselines, variance-reduction methods, memory-efficient methods, structured second-order preconditioners, schedule-free methods, and orthogonality-oriented updates.

A.3 Standard Baseline and Adaptive Optimizers

SGD with Momentum. The classical update accumulates a momentum buffer and then applies a fixed learning rate. At each iteration t , the algorithm maintains an exponential moving average m_t of past gradients:

$$m_t = \beta m_{t-1} + g_t \quad (1)$$

$$\theta_{t+1} = \theta_t - \eta m_t \quad (2)$$

where $g_t = \nabla f(\theta_t)$, $\beta \in [0.8, 0.99]$ controls the momentum decay, and η is the fixed learning rate. The momentum term acts as velocity: it accelerates movement in consistent directions while dampening oscillations in variable directions. Its strengths are low memory overhead (single momentum buffer per parameter) and strong generalization when tuned carefully. Its main weakness in transformer workloads is poor robustness to heterogeneous curvature (different Hessian spectra across parameter groups) and strong dependence on learning-rate schedules.

Algorithm 1 SGD with Momentum

Require: Initial parameters θ_0 , learning rate η , momentum coefficient β , weight decay λ

Ensure: Updated parameters θ_T

- 1: Initialize: momentum buffer $m \leftarrow 0$
 - 2: **for** $t = 0, 1, 2, \dots, T - 1$ **do**
 - 3: Compute gradient: $g_t \leftarrow \nabla f(\theta_t)$
 - 4: **if** `weight_decay` > 0 **then**
 - 5: $g_t \leftarrow g_t + \lambda \theta_t$ $\triangleright$ L2 regularization
 - 6: **end if**
 - 7: Update momentum: $m \leftarrow \beta \cdot m + g_t$
 - 8: Update parameters: $\theta_{t+1} \leftarrow \theta_t - \eta \cdot m$
 - 9: **end for** **return** θ_T
-

Adam and AdamW. Adam tracks exponential moving averages of both first moment (mean) and second moment (uncentered variance) to achieve element-wise adaptive learning rates. The update rule is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction}) \quad (5)$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) \quad (6)$$

AdamW introduces a crucial modification: weight decay is applied directly to parameters (decoupled from gradients): $\theta_t \leftarrow \theta_t(1 - \eta\lambda)$ rather than adding $\lambda\theta_t$ to the gradient. This decoupling prevents the adaptive scaling from interfering with regularization strength. The report treats AdamW as the practical baseline for transformer fine-tuning because it converges quickly and is relatively forgiving to hyperparameter variation. The tradeoff is memory cost: both moment tensors must be stored for every parameter, doubling optimizer-state memory relative to SGD.

Algorithm 2 AdamW (Adam with Decoupled Weight Decay)

Require: Initial parameters θ_0 , learning rate η , exponential decay rates $\beta_1, \beta_2 \in [0, 1]$

Require: Momentum constant ϵ , weight decay λ

Ensure: Updated parameters θ_T

- 1: Initialize: first moment $m \leftarrow 0$, second moment $v \leftarrow 0$, step counter $t \leftarrow 0$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: Compute gradient: $g_t \leftarrow \nabla f(\theta_{t-1})$
 - 4: Weight decay (decoupled): $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
 - 5: Update moments: $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$
 - 6: $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
 - 7: Bias correction: $\hat{m} \leftarrow m / (1 - \beta_1^t)$, $\hat{v} \leftarrow v / (1 - \beta_2^t)$
 - 8: Update parameters: $\theta_t \leftarrow \theta_{t-1} - \eta(\hat{m} / (\sqrt{\hat{v}} + \epsilon))$
 - 9: **end for** **return** θ_T
-

RAdam (Rectified Adam). RAdam addresses Adam’s instability in early training by dynamically rectifying the adaptive learning rate. The key observation is that the second moment v_t has very high variance in the first few steps, causing unreliable adaptive scaling. RAdam computes the effective simple moving average (SMA) window length:

$$\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1 - \beta_2^t}, \quad \text{where} \quad \rho_\infty = \frac{2}{1 - \beta_2} - 1$$

When $\rho_t > 4$ (sufficient samples for variance estimation), RAdam applies adaptive scaling with a rectification term:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}, \quad \theta_{t+1} = \theta_t - \eta r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

When $\rho_t \leq 4$, RAdam falls back to SGD with momentum: $\theta_{t+1} = \theta_t - \eta \hat{m}_t$. This graceful transition eliminates the need for manual learning-rate warmup schedules and improves robustness.

Algorithm 3 RAdam (Rectified Adam)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , weight decay λ

Ensure: Updated parameters θ_T

```

1: Initialize:  $m \leftarrow 0, v \leftarrow 0, t \leftarrow 0$ 
2: Compute:  $\rho_\infty \leftarrow 2/(1 - \beta_2) - 1$ 
3: for  $t = 1, 2, \dots, T$  do
4:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
5:   if weight_decay  $> 0$  then
6:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
7:   end if
8:    $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$ 
9:    $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
10:   $\hat{m} \leftarrow m/(1 - \beta_1^t), \hat{v} \leftarrow v/(1 - \beta_2^t)$ 
11:   $\rho_t \leftarrow \rho_\infty - 2t\beta_2^t/(1 - \beta_2^t)$ 
12:  if  $\rho_t > 4$  then
13:     $r_t \leftarrow \sqrt{((\rho_t - 4)(\rho_t - 2)\rho_\infty)/((\rho_\infty - 4)(\rho_\infty - 2)\rho_t)}$ 
14:     $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{m}/(\sqrt{\hat{v}} + \epsilon)$ 
15:  else
16:     $\theta_t \leftarrow \theta_{t-1} - \eta \hat{m}$  ▷ SGD with momentum
17:  end if
18: end for return  $\theta_T$ 

```

A.4 Advanced Momentum and Variance Reduction (2024–2025)

Adan (Adaptive Nesterov Momentum). Adan reformulates Nesterov acceleration without the extra gradient computation required by classical Nesterov SGD. The algorithm maintains three momentum buffers:

$$m_t = (1 - \beta_1)m_{t-1} + \beta_1 g_t \quad (\text{first moment}) \quad (7)$$

$$v_t = (1 - \beta_2)v_{t-1} + \beta_2(g_t - g_{t-1}) \quad (\text{velocity/gradient difference}) \quad (8)$$

$$n_t = (1 - \beta_3)n_{t-1} + \beta_3[g_t + (1 - \beta_1)(g_t - g_{t-1})]^2 \quad (\text{Nesterov second moment}) \quad (9)$$

The **Nesterov Momentum Estimation** (NME) term $\bar{g}_t = g_t + (1 - \beta_1)(g_t - g_{t-1})$ estimates the gradient at a future position without evaluating it. The update combines momentum with velocity:

$$\bar{m}_t = m_t + (1 - \beta_1)v_t, \quad \theta_{t+1} = \theta_t - \eta \frac{\bar{m}_t}{\sqrt{n_t} + \epsilon}$$

Adan achieves fast convergence across diverse architectures (CNNs, GANs, Transformers) through this acceleration. The cost is maintaining three momentum-like buffers, increasing memory overhead relative to Adam.

Algorithm 4 Adan (Adaptive Nesterov Momentum)

Require: Initial parameters θ_0 , learning rate η , $\beta_1, \beta_2, \beta_3 \in (0, 1)$

Ensure: Updated parameters θ_T

```
1: Initialize:  $m \leftarrow 0, v \leftarrow 0, n \leftarrow 0, g_{-1} \leftarrow 0$  (previous gradient)
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow (1 - \beta_1)m + \beta_1 g_t$ 
5:    $\Delta g_t \leftarrow g_t - g_{t-1}$ 
6:    $v \leftarrow (1 - \beta_2)v + \beta_2 \Delta g_t$ 
7:   Nesterov estimation:  $\bar{g}_t \leftarrow g_t + (1 - \beta_1)\Delta g_t$ 
8:    $n \leftarrow (1 - \beta_3)n + \beta_3 \bar{g}_t^2$ 
9:   Combined momentum:  $\bar{m} \leftarrow m + (1 - \beta_1)v$ 
10:   $\theta_t \leftarrow \theta_{t-1} - \eta(\bar{m}/(\sqrt{n} + \epsilon))$ 
11:   $g_{t-1} \leftarrow g_t$ 
12: end for return  $\theta_T$ 
```

ADOPT (Adam with Optimal Pruned Tuning). ADOPT fixes a fundamental theoretical issue in Adam: the gradient appears in both the first moment and second moment estimates, creating circularity. ADOPT **decouples** by using the previous step’s second moment for the denominator:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (10)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{g_t}{\sqrt{v_{t-1}} + \epsilon} \quad (\text{uses } v_{t-1}, \text{ not } v_t) \quad (11)$$

$$\theta_{t+1} = \theta_t - \eta m_t \quad (12)$$

This simple reordering achieves the optimal convergence rate $O(1/\sqrt{T})$ with **any** choice of $\beta_2 \in (0, 1)$, without bounded-noise assumptions. The practical consequence is that ADOPT is a drop-in replacement for Adam with stronger theoretical guarantees and comparable or superior empirical performance across vision, NLP, RL, and generative modeling domains.

Algorithm 5 ADOPT (Adam with Optimal Pruned Tuning)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , tolerance ϵ

Ensure: Updated parameters θ_T

```
1: Initialize:  $m \leftarrow 0, v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   if weight_decay  $> 0$  then
5:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
6:   end if
7:   Use previous second moment:  $\text{denom} \leftarrow \sqrt{\max(v, \epsilon)} + \epsilon$ 
8:   Update first moment using previous variance:  $m \leftarrow \beta_1 m + (1 - \beta_1)(g_t/\text{denom})$ 
9:   Update second moment with current gradient:  $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
10:   $\theta_t \leftarrow \theta_{t-1} - \eta m$ 
11: end for return  $\theta_T$ 
```

AdEMAMix (Exponential Moving Average Mixture). AdEMAMix replaces Adam’s single EMA of gradients with a **mixture of two EMAs**: one fast-decaying and one slow-decaying. This addresses the observation that gradients remain informative over tens of thousands of steps,

not just hundreds:

$$m_{1,t} = \beta_1 m_{1,t-1} + (1 - \beta_1) g_t \quad (\text{fast EMA, } \beta_1 \approx 0.9) \quad (13)$$

$$m_{2,t} = \beta_3 m_{2,t-1} + (1 - \beta_3) g_t \quad (\text{slow EMA, } \beta_3 \approx 0.9999) \quad (14)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (15)$$

$$m_t = m_{1,t} + \alpha_t m_{2,t} \quad (\text{mixture with scheduled weight } \alpha_t) \quad (16)$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (17)$$

The mixture weight α_t typically increases during training, allowing the fast EMA to provide immediate adaptation while the slow EMA accumulates long-range gradient correlations. Empirically, a 1.3B model on 101B tokens achieves similar loss to AdamW on 197B tokens (95% data efficiency gain), suggesting the slow EMA significantly reduces model forgetting.

Algorithm 6 AdEMAMix (Exponential Moving Average Mixture)

Require: Initial parameters θ_0 , learning rate η , β_1 (fast), β_3 (slow), β_2 (second moment)

Require: Mixture weight α_t (typically increasing), tolerance ϵ

Ensure: Updated parameters θ_T

```

1: Initialize:  $m_1 \leftarrow 0$  (fast EMA),  $m_2 \leftarrow 0$  (slow EMA),  $v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   if weight_decay > 0 then
5:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
6:   end if
7:    $m_1 \leftarrow \beta_1 m_1 + (1 - \beta_1) g_t$  ▷ Fast EMA
8:    $m_2 \leftarrow \beta_3 m_2 + (1 - \beta_3) g_t$  ▷ Slow EMA
9:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ 
10:   $m_t \leftarrow m_1 + \alpha_t m_2$  ▷ Mixture
11:   $\theta_t \leftarrow \theta_{t-1} - \eta(m_t / (\sqrt{v} + \epsilon))$ 
12: end for return  $\theta_T$ 

```

MARS (Make Adaptive learning Rates Shine). MARS combines preconditioned gradient methods (e.g., AdamW) with **variance reduction** via scaled stochastic recursive momentum. The core innovation is a variance-reduced gradient estimate:

$$c_t = g_t + \gamma(c_{t-1} - g_{t-1}) \quad (\text{SVRG-style recursive estimate}) \quad (18)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) c_t \quad (19)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \quad (20)$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (21)$$

where $\gamma \in [0.01, 0.1]$ controls how much historical gradient information is retained. The variance-reduced gradient c_t acts as an implicit noise filter, amplifying consistent signal directions and dampening contradictory noise. MARS-AdamW consistently outperforms bare AdamW by significant margins on GPT-2 pretraining, suggesting that variance reduction substantially improves convergence in mini-batch stochastic training.

Algorithm 7 MARS (Make Adaptive learning Rates Shine)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , weight decay λ

Require: Variance reduction coefficient $\gamma \in [0.01, 0.1]$

Ensure: Updated parameters θ_T

```
1: Initialize:  $m \leftarrow 0, v \leftarrow 0, c \leftarrow 0$  (variance-reduced gradient),  $g_{-1} \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   Variance reduction:  $c \leftarrow g_t + \gamma(c - g_{t-1})$ 
5:    $m \leftarrow \beta_1 m + (1 - \beta_1)c$ 
6:    $v \leftarrow \beta_2 v + (1 - \beta_2)c^2$ 
7:   if weight_decay  $> 0$  then
8:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
9:   end if
10:   $\theta_t \leftarrow \theta_{t-1} - \eta(m/(\sqrt{v} + \epsilon))$ 
11:   $g_{t-1} \leftarrow g_t$ 
12: end for return  $\theta_T$ 
```

Cautious Optimizers (Cautious AdamW, Cautious Lion). The Cautious framework applies a **one-line modification** to any momentum-based optimizer: mask the update so that only dimensions where momentum and gradient directions agree are applied:

$$\text{mask}_i = \begin{cases} 1 & \text{if } m_t[i] \cdot g_t[i] > 0 \quad (\text{agreement}) \\ 0 & \text{otherwise} \end{cases} \quad (22)$$

$$u_t = \left(\frac{m_t}{\sqrt{v_t} + \epsilon} \right) \odot \text{mask} \quad (\text{element-wise masking}) \quad (23)$$

$$\theta_{t+1} = \theta_t - \eta u_t \quad (24)$$

The intuition: momentum m_t estimates the gradient direction from history; the current gradient g_t is instantaneous signal. When both agree, the optimizer is confident and should update aggressively. When they disagree, the optimizer is conflicted—the historical trend points toward a region that may have been good before, but current evidence contradicts it. By masking conflicted dimensions, Cautious becomes more conservative and avoids corrupted steps. Empirically, this simple masking achieves up to **1.47 $\times$ speedup** on Llama and MAE pretraining while preserving convergence guarantees.

Algorithm 8 Cautious AdamW (Consensus-based Update Masking)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , weight decay λ

Ensure: Updated parameters θ_T

```
1: Initialize:  $m \leftarrow 0, v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$ 
5:    $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
6:   Consensus mask:  $\text{mask}_i \leftarrow 1$  if  $m[i] \cdot g_t[i] > 0$ , else 0  $\triangleright$  Agreement
7:   Base Adam update:  $u \leftarrow m/(\sqrt{v} + \epsilon)$ 
8:   Apply mask:  $u_{\text{masked}} \leftarrow u \odot \text{mask}$   $\triangleright$  Element-wise
9:   if weight_decay  $> 0$  then
10:     $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
11:   end if
12:    $\theta_t \leftarrow \theta_{t-1} - \eta u_{\text{masked}}$ 
13: end for return  $\theta_T$ 
```

A.5 Large-Batch, Memory-Efficient, and Parameter-Free Optimizers

LAMB (Layer-wise Adaptive Moments optimizer for Batch training). LAMB extends Adam with **layer-wise adaptive rate scaling** (inspired by LARS), enabling stable training with extreme batch sizes (e.g., 64K on BERT):

$$m_t^L = \beta_1 m_{t-1}^L + (1 - \beta_1) g_t^L \quad (\text{layer-wise first moment}) \quad (25)$$

$$v_t^L = \beta_2 v_{t-1}^L + (1 - \beta_2) (g_t^L)^2 \quad (26)$$

$$u_{\text{adam}}^L = \frac{m_t^L}{\sqrt{v_t^L} + \epsilon} + \lambda \theta_{t-1}^L \quad (\text{base adaptive step with decay}) \quad (27)$$

$$\phi^L = \frac{\|\theta_{t-1}^L\|_2}{\|u_{\text{adam}}^L\|_2} \quad (\text{trust ratio: layer normalization}) \quad (28)$$

$$\theta_t^L = \theta_{t-1}^L - \eta \cdot \phi^L \cdot u_{\text{adam}}^L \quad (29)$$

The **trust ratio** $\phi^L = \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$ normalizes the effective update step relative to weight magnitude. In very large batches, stochastic gradient noise can exceed signal, destabilizing layer-wise learning rates. By scaling updates proportionally to weight norms, LAMB ensures relative changes rather than absolute ones, preventing small confident updates in large vectors from dominating. LAMB preserves small-batch generalization benefits while enabling efficient large-batch training.

Algorithm 9 LAMB (Layer-wise Adaptive Moments optimizer for Batch training)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , weight decay λ

Ensure: Updated parameters θ_T

```

1: Initialize: For each layer  $L$ :  $m^L \leftarrow 0$ ,  $v^L \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:   for each layer  $L$  do
4:      $g_t^L \leftarrow \nabla f(\theta_t^L)$ 
5:      $m^L \leftarrow \beta_1 m^L + (1 - \beta_1) g_t^L$ 
6:      $v^L \leftarrow \beta_2 v^L + (1 - \beta_2) (g_t^L)^2$ 
7:     Base Adam:  $u_{\text{adam}}^L \leftarrow m^L / (\sqrt{v^L} + \epsilon)$ 
8:     if  $\text{weight\_decay} > 0$  then
9:        $u_{\text{adam}}^L \leftarrow u_{\text{adam}}^L + \lambda \theta_{t-1}^L$ 
10:    end if
11:    Trust ratio:  $\phi^L \leftarrow \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$  if both nonzero, else 1
12:     $\theta_t^L \leftarrow \theta_{t-1}^L - \eta \phi^L u_{\text{adam}}^L$ 
13:   end for
14: end for return  $\theta_T$ 

```

Schedule-Free AdamW. Schedule-free methods remove explicit scheduler design from the optimization recipe. The algorithm maintains two parameter streams: z_t (exploration) and x_t (smooth average):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{variance}) \quad (30)$$

$$z_t = z_{t-1} (1 - \eta \lambda) - \eta \frac{g_t}{\sqrt{v_t} + \epsilon} \quad (\text{adaptive step with decay}) \quad (31)$$

$$c_t = \frac{1}{t + 1} \quad (\text{averaging coefficient, decreases over time}) \quad (32)$$

$$x_t = (1 - c_t) x_{t-1} + c_t z_t \quad (\text{iterate averaging}) \quad (33)$$

$$y_t = (1 - \beta) z_t + \beta x_t \quad (\text{interpolation for evaluation point}) \quad (34)$$

The averaging coefficient $c_t = 1/(t + 1)$ implements an implicit learning-rate schedule without explicitly specifying total steps T . This framework unifies scheduling and iterate averaging: the algorithm explores via z_t while accumulating stable direction via x_t . The evaluation point y_t (used for gradient computation) interpolates between exploration and stability. Schedule-Free achieves state-of-the-art convergence across convex optimization, large-scale deep learning, and reinforcement learning, while removing a major hyperparameter.

Algorithm 10 Schedule-Free AdamW

Require: Initial parameters θ_0 , learning rate η (fixed), β_1, β_2 , weight decay λ

Ensure: Updated parameters θ_T or averaging x_T

```

1: Initialize:  $z \leftarrow \theta_0$  (exploration),  $x \leftarrow \theta_0$  (average),  $v \leftarrow 0$ ,  $t \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(y_{t-1})$  (gradient at interpolation point)
4:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$  ▷ Variance
5:   Weight decay on  $z$ :  $z \leftarrow z(1 - \eta\lambda)$ 
6:    $z \leftarrow z - \eta g_t / (\sqrt{v} + \epsilon)$  ▷ Adaptive step on raw
7:    $c_t \leftarrow 1/(t + 1)$  ▷ Averaging coefficient
8:    $x \leftarrow (1 - c_t)x + c_t z$  ▷ Iterate averaging
9:    $y_t \leftarrow (1 - \beta_1)z + \beta_1 x$  ▷ Interpolation for next eval
10: end for return  $x_T$  or  $y_T$ 

```

Adafactor. Adafactor reduces optimizer memory by **factorizing** second-moment statistics for matrix-shaped parameters, storing only row and column accumulators rather than dense variance states. For a gradient matrix $G_t \in \mathbb{R}^{m \times n}$:

$$R_t = \beta_2 R_{t-1} + (1 - \beta_2)(G_t^2) \mathbf{1}_n^T \quad (\text{row variance}) \quad (35)$$

$$C_t = \beta_2 C_{t-1} + (1 - \beta_2) \mathbf{1}_m^\top (G_t^2) \quad (\text{column variance}) \quad (36)$$

$$\hat{V}_t = \frac{R_t C_t}{\mathbf{1}_n^\top R_t} \quad (\text{reconstructed variance via outer product}) \quad (37)$$

$$U_t = \frac{G_t}{\sqrt{\hat{V}_t} + \epsilon} \quad (\text{normalized adaptive step}) \quad (38)$$

$$\hat{U}_t = \frac{U_t}{\max(1, \text{RMS}(U_t))} \quad (\text{stability clipping}) \quad (39)$$

Instead of storing $m \times n$ second-moment values, Adafactor stores only $m + n$ accumulators, reducing memory from $O(n_{\text{params}})$ to $O(\sqrt{n_{\text{params}}})$. This is most attractive when VRAM is dominated by optimizer state rather than activations. The tradeoff is reduced optimization expressiveness and potential instability on some tasks.

Algorithm 11 Adafactor (Factorized Second-Moment Statistics)

Require: Gradient matrix $G_t \in \mathbb{R}^{m \times n}$, learning rate η , $\beta_2 \approx 0.999$

Ensure: Updated parameters

```
1: Initialize: Row accumulators  $R \leftarrow \epsilon \mathbf{1}_m^T$ , Column  $C \leftarrow \epsilon \mathbf{1}_n$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $G_t \leftarrow \nabla f(\theta_t)$  (matrix gradient)
4:    $R \leftarrow \beta_2 R + (1 - \beta_2)(G_t^2) \mathbf{1}_n^T$  ▷ Row inner products
5:    $C \leftarrow \beta_2 C + (1 - \beta_2) \mathbf{1}_m^T (G_t^2)$  ▷ Column inner products
6:   Reconstructed variance:  $\hat{V} \leftarrow (R \cdot C) / (\mathbf{1}_n^T R)$  ▷ Via outer product
7:   Normalized adaptive step:  $U_t \leftarrow G_t / (\sqrt{\hat{V}} + \epsilon)$ 
8:   Per-row RMS clipping:  $\hat{U}_t \leftarrow U_t / \max(1, \text{RMS}(U_t))$ 
9:    $\theta_{t+1} \leftarrow \theta_t - \eta \hat{U}_t$ 
10: end for
```

GaLore (Gradient Low-Rank Projection). GaLore projects 2D gradients into a low-rank subspace before optimization, reducing optimizer-state memory while maintaining adaptive step scaling. For a 2D parameter matrix $W \in \mathbb{R}^{m \times n}$:

$$U, S, V = \text{SVD}(G_t) \quad (\text{compute singular decomposition}) \quad (40)$$

$$P \in \mathbb{R}^{n \times r} \quad \text{or} \quad P^\top \in \mathbb{R}^{r \times m} \quad (\text{select top-}r \text{ singular vectors}) \quad (41)$$

$$G_{\text{low}} = P^\top G_t \quad (\text{project to low-rank space}) \quad (42)$$

$$\Delta_{\text{low}} = \text{Adam}(G_{\text{low}}) \quad (\text{optimize in compressed space}) \quad (43)$$

$$\Delta = P \Delta_{\text{low}} \quad (\text{reconstruct in original space}) \quad (44)$$

By projecting into a rank- r subspace (typically $r \ll \min(m, n)$), optimizer state is reduced from $O(mn)$ to $O(r(m + n))$ for 2D parameters. This complementary memory-saving approach is especially relevant when the model is too large for full-rank optimizer state. GaLore works synergistically with other techniques and has shown strong empirical results on billion-parameter models.

Algorithm 12 GaLore (Gradient Low-Rank Projection)

Require: 2D parameter matrix $W \in \mathbb{R}^{m \times n}$, learning rate η , rank $r \ll \min(m, n)$

Ensure: Updated parameters

```
1: Initialize: Adam state in low-rank space (if rank-2 variant)
2: for  $t = 1, 2, \dots, T$  do
3:    $G_t \leftarrow \nabla f(W_t)$ 
4:   if  $t \bmod K = 1$  then ▷ Periodic SVD
5:      $U_t, S_t, V_t^\top \leftarrow \text{SVD}(G_t)$  (full or thin)
6:     Select projection:  $P \leftarrow U_t[:, :r]$  or  $P \leftarrow V_t[:, :r]$ 
7:   end if
8:   Project gradient:  $G_{\text{low}} \leftarrow P^\top G_t$  (or  $G_t P^\top$  for right projection)
9:   Run Adam on low-rank:  $\Delta_{\text{low}} \leftarrow \text{Adam}(G_{\text{low}})$ 
10:  Reconstruct in original space:  $\Delta \leftarrow P \Delta_{\text{low}}$  (or  $\Delta_{\text{low}} P^\top$ )
11:   $W_t \leftarrow W_t - \eta \Delta$ 
12: end for
```

APOLLO, APOLLO-Mini, and Q-APOLLO. The APOLLO family [66] begins from the observation that AdamW’s element-wise denominator can be **coarsened into a structured learning-rate update**. Rather than storing dense moments for every parameter entry,

APOLLO projects a matrix gradient $G_t \in \mathbb{R}^{m \times n}$ into a compact random subspace and tracks Adam-style moments there:

$$R_t = P_t G_t \quad \text{or} \quad R_t = G_t P_t^\top \quad (45)$$

$$M_t^R = \beta_1 M_{t-1}^R + (1 - \beta_1) R_t \quad (46)$$

$$V_t^R = \beta_2 V_{t-1}^R + (1 - \beta_2) R_t^2 \quad (47)$$

$$\tilde{R}_t = \frac{M_t^R}{\sqrt{V_t^R} + \epsilon} \quad (48)$$

The projected state is *not* expanded back into a dense low-rank update as in SVD-based methods. Instead, APOLLO estimates a structured scaling tensor S_t in the original space. In the standard APOLLO variant, that scaling is **channel-wise**: each row or channel receives its own norm ratio. In APOLLO-Mini, the scaling is reduced to a single **tensor-wise** scalar, corresponding to the rank-1 extreme described in the paper. The resulting parameter update is therefore Adam-like in adaptation but much closer to SGD in state cost:

$$W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\alpha(G_t \odot S_t)$$

where α is the extra scale factor used to stabilize highly compressed variants.

The paper’s practical contribution is twofold. First, APOLLO replaces expensive repeated SVD with periodically refreshed **Gaussian random projection**, so the systems burden is ordinary matrix multiplication rather than spectral decomposition. Second, the optimizer is unusually tolerant to extreme compression: even **APOLLO-Mini**, which keeps only rank-1 auxiliary state, remains competitive with or better than AdamW in the reported pre-training experiments while approaching SGD-level memory cost.

Algorithm 13 APOLLO / APOLLO-Mini Structured Gradient Scaling

Require: Weight matrix $W \in \mathbb{R}^{m \times n}$ with $m \leq n$, learning rate η , scale factor α , decay rates (β_1, β_2) , weight decay λ , rank r , projection refresh interval T

Ensure: Updated parameters W_T

- 1: Initialize projected moments: $M^R \leftarrow 0, V^R \leftarrow 0$, step $t \leftarrow 0$
 - 2: **repeat**
 - 3: Compute gradient: $G_t \leftarrow \nabla_W \phi(W_t)$
 - 4: **if** $t \bmod T = 0$ **then**
 - 5: Sample Gaussian projector $P_t \sim \mathcal{N}(0, 1/r)$ with a fresh seed
 - 6: **end if**
 - 7: Project gradient: $R_t \leftarrow P_t G_t$ $\triangleright$ or $G_t P_t^\top$ depending on layout
 - 8: Update projected AdamW moments: $M_t^R, V_t^R \leftarrow \text{AdamWState}(R_t; \beta_1, \beta_2)$
 - 9: Normalize projected state: $\tilde{R}_t \leftarrow M_t^R / (\sqrt{V_t^R} + \epsilon)$
 - 10: **if** APOLLO **then**
 - 11: $S_t \leftarrow \text{diag}(s_1^R, \dots, s_m^R)$, where $s_i^R = \|\tilde{R}_t[i, :]\|_2 / \|R_t[i, :]\|_2$
 - 12: **else**
 - 13: $S_t \leftarrow s_t^R$, where $s_t^R = \|\tilde{R}_t\|_2 / \|R_t\|_2$
 - 14: **end if**
 - 15: Update weights: $W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\alpha(G_t \odot S_t)$
 - 16: $t \leftarrow t + 1$
 - 17: **until** convergence
-

APOLLO-Mini. APOLLO-Mini is the family member optimized for **extreme memory efficiency**. The paper motivates it by arguing that, in a rank-1 compact space, channel-wise scaling becomes too noisy, so the update is coarsened further to a single tensor-wise scale. The

loss of granularity is partially offset by the explicit scaling factor α (the paper discusses values such as 128 in this regime), giving a useful point on the optimization Pareto frontier: very small state, no SVD overhead, and still strong pre-training behavior.

Q-APOLLO. The APOLLO paper also stresses that the family combines naturally with **quantization** for ultra-low-memory training. This repository turns that systems observation into a concrete optimizer variant, `q_apollo`. In the implementation, APOLLO’s low-rank first and second moments are stored in quantized form together with per-tensor scales and offsets, and are dequantized only when needed for the next step. Q-APOLLO therefore preserves APOLLO’s projected-gradient logic while reducing the precision of the remaining optimizer state, making it the most aggressive memory-saving member of the local optimizer stack.

Anon (Adaptivity Non-restricted Optimizer with Novel convergence technique). Anon [2] introduces continuously tunable adaptivity via a single parameter $\gamma \in \mathbb{R}$, bridging the gap between SGD-like ($\gamma = 0$) and Adam-like ($\gamma = 1$) behaviors, and extrapolating beyond both. The core innovation is the Incremental Delay Update (IDU) mechanism, which enables stable convergence across the entire real-valued adaptivity spectrum:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{first moment}) \quad (49)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{second moment}) \quad (50)$$

$$\bar{v}_t = \frac{v_t}{1 - \beta_2^{\max(t,1)}} + \epsilon \quad (\text{bias-corrected second moment}) \quad (51)$$

$$\text{fixed_lr}_k = \begin{cases} \bar{v}_k^{-\gamma/2} & \text{if } k = 0 \\ \sqrt{2 / \left(\frac{1}{\text{fixed_lr}_{k-1}^2} + \bar{v}_k^\gamma \right)} & \text{if } k > 0 \end{cases} \quad (52)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{1 - \beta_1^t} \cdot m_t \odot \text{fixed_lr}_{\lfloor \log_2 t \rfloor} \quad (53)$$

Unlike standard adaptive optimizers, IDU updates the preconditioner only at powers of two ($t = 2^k$), accumulating gradient statistics between updates and aggregating them recursively. This soft, multi-scale accumulator avoids the hard max-tracking of AMSGrad while provably stabilizing convergence for any $\gamma \in \mathbb{R}$. The parameter γ directly controls adaptivity: $\gamma > 1$ accelerates escape from saddle points (favorable for complex architectures), $\gamma = 1$ recovers Adam-like behavior, $\gamma = 0$ yields SGD-like scaling, and $\gamma < 0$ biases toward flatter minima (beneficial for classical architectures like CNNs).

Anon maintains three state buffers (m , v , fixed_lr) with $\mathcal{O}(n)$ per-step cost, placing it in the same complexity class as AdamW but with the added flexibility of tunable adaptivity. The IDU technique provably converges at $\mathcal{O}(\sqrt{T})$ regret for convex problems and $\mathcal{O}(\ln T / \sqrt{T})$ for non-convex settings, matching the guarantees of mainstream adaptive optimizers while supporting the full real-valued adaptivity range.

Algorithm 14 Anon (Adaptivity Non-restricted Optimizer with IDU)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , tolerance ϵ , adaptivity $\gamma \in \mathbb{R}$

Ensure: Updated parameters θ_T

```
1: Initialize:  $m \leftarrow 0$ ,  $v \leftarrow 0$ , fixed_lr  $\leftarrow 0$ , step  $t \leftarrow 0$ , IDU counter  $k \leftarrow -1$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$ 
5:    $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
6:   if  $t = 2^k$  then
7:      $k \leftarrow k + 1$ 
8:      $\bar{v} \leftarrow v / (1 - \beta_2^{\max(t/2, 1)}) + \epsilon$ 
9:     if  $k = 0$  then
10:      fixed_lr  $\leftarrow \bar{v}^{-\gamma/2}$ 
11:     else
12:      fixed_lr  $\leftarrow \sqrt{2 / (1/\text{fixed\_lr}^2 + \bar{v}^\gamma)}$ 
13:     end if
14:      $v \leftarrow 0$ 
15:   end if
16:    $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{1 - \beta_1^t} \cdot m \odot \text{fixed\_lr}$ 
17: end for return  $\theta_T$ 
```

Prodigy (Approximating the Distance Estimate). Prodigy adapts the effective step scale through a running distance-like statistic, eliminating the need for explicit learning-rate tuning. The algorithm maintains a cumulative distance estimate:

$$u_t = \frac{g_t}{\sqrt{v_t} + \epsilon} \quad (\text{unnormalized adaptive step}) \quad (54)$$

$$s_t = s_{t-1} + \langle u_t, \theta_t - \theta_0 \rangle \quad (\text{cumulative signed distance}) \quad (55)$$

$$d_t = \max(d_{t-1}, d_0 + d_{\text{coef}} \cdot s_t) \quad (\text{distance estimate with lower bound}) \quad (56)$$

$$\theta_{t+1} = \theta_t - \eta \cdot d_t \cdot u_t \quad (57)$$

where d_0 is an initial bound and d_{coef} controls how aggressively the estimate adapts. The distance estimate d_t captures the combined magnitude of past gradients weighted by actual parameter displacement, implementing an implicit effective learning rate. This distance-aware scaling achieves robust convergence across problem scales and batch sizes without requiring a learning-rate schedule. Prodigy reduces hyperparameter sensitivity by estimating step sizes from optimization geometry rather than problem-specific prior knowledge.

Algorithm 15 Prodigy (Approximating the Distance Estimate)

Require: Initial parameters θ_0 , learning rate η , coeff d_{coef} , initial dist $d_0 > 0$

Ensure: Updated parameters θ_T

```
1: Initialize:  $s \leftarrow 0$  (cumulative signed distance),  $d \leftarrow d_0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$  ▷ Second moment
5:    $u_t \leftarrow g_t / (\sqrt{v_t} + \epsilon)$  ▷ Unnormalized adaptive
6:   Update distance:  $s \leftarrow s + \langle u_t, \theta_t - \theta_0 \rangle$  ▷ Signed distance
7:    $d \leftarrow \max(d, d_0 + d_{\text{coef}} \cdot s)$  ▷ Lower-bounded distance
8:    $\theta_t \leftarrow \theta_{t-1} - \eta \cdot d \cdot u_t$  ▷ Distance-scaled update
9: end for return  $\theta_T$ 
```

A.6 Second-Order, Geometric, and Orthogonality Optimizers

Algorithm 16 Shampoo (Matrix Preconditioning via Kronecker Products)

Require: Initial parameters θ_0 , learning rate η , eigendecomposition interval K

Ensure: Updated parameters θ_T

```

1: Initialize:  $L \leftarrow \epsilon I_m, R \leftarrow \epsilon I_n$  (left and right Gram matrices)
2: for  $t = 1, 2, \dots, T$  do
3:    $G \leftarrow \nabla f(\theta_{t-1})$  ▷ Gradient
4:   Update Gram matrices:  $L \leftarrow L + GG^\top, R \leftarrow R + G^\top G$ 
5:   if  $t \bmod K == 0$  then
6:      $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$  ▷ Eigendecomposition of  $L$ 
7:      $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$  ▷ Eigendecomposition of  $R$ 
8:      $L^{-1/4} \leftarrow Q_L(\Lambda_L + \epsilon)^{-1/4}Q_L^\top$ 
9:      $R^{-1/4} \leftarrow Q_R(\Lambda_R + \epsilon)^{-1/4}Q_R^\top$ 
10:  end if
11:   $\Delta\theta \leftarrow L^{-1/4}GR^{-1/4}$  ▷ Preconditioned gradient
12:   $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$ 
13: end for return  $\theta_T$ 

```

Shampoo (Matrix Preconditioning via Kronecker Products). Shampoo is a **structured second-order method** that computes matrix preconditioners from Kronecker-structured outer-product statistics. For a 2D parameter matrix $W \in \mathbb{R}^{m \times n}$:

$$L_t = L_{t-1} + G_t G_t^\top \quad (\text{left/row Gram matrix, } m \times m) \quad (58)$$

$$R_t = R_{t-1} + G_t^\top G_t \quad (\text{right/column Gram matrix, } n \times n) \quad (59)$$

$$L_t^{-1/4} = Q_L(\Lambda_L)^{-1/4}Q_L^\top \quad (\text{via eigendecomposition}) \quad (60)$$

$$R_t^{-1/4} = Q_R(\Lambda_R)^{-1/4}Q_R^\top \quad (61)$$

$$\Delta W_t = L_t^{-1/4}G_tR_t^{-1/4} \quad (\text{preconditioned gradient}) \quad (62)$$

Shampoo approximates the full-matrix Adagrad preconditioner $H^{-1/2}$ (where H is the Hessian) using Kronecker-factored structure. Instead of storing a $(mn) \times (mn)$ preconditioner, Shampoo maintains two smaller matrices ($m \times m$ and $n \times n$), capturing cross-parameter correlations within and across groups. The method has proven effective at scale (Google production systems) and is well-suited to transformer architectures with high-rank structure. Eigendecompositions are performed periodically (e.g., every $K = 10$ steps) to amortize cost. Tradeoff: higher per-step compute and periodic $O(m^3 + n^3)$ eigendecomposition versus improved conditioning and accelerated convergence.

SOAP (Shampoo with Adam in eigenbasis). SOAP is a simplified variant of Shampoo that decouples preconditioning from momentum tracking. Instead of complex matrix algebra on preconditioned gradients, SOAP runs standard Adam in the eigenbasis of Shampoo’s preconditioned gradients.

tioners:

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t \quad (\text{Gram accumulation}) \quad (63)$$

$$Q_L, \Lambda_L = \text{eigh}(L_t), \quad Q_R, \Lambda_R = \text{eigh}(R_t) \quad (\text{periodic eigendecomposition}) \quad (64)$$

$$G_t^{\text{rot}} = Q_L^\top G_t Q_R \quad (\text{rotate gradient into eigenbasis}) \quad (65)$$

$$m_t^{\text{rot}} = \beta_1 m_{t-1}^{\text{rot}} + (1 - \beta_1) G_t^{\text{rot}} \quad (66)$$

$$v_t^{\text{rot}} = \beta_2 v_{t-1}^{\text{rot}} + (1 - \beta_2) (G_t^{\text{rot}})^2 \quad (\text{Adam in rotated space}) \quad (67)$$

$$U_t^{\text{rot}} = \frac{m_t^{\text{rot}}}{\sqrt{v_t^{\text{rot}} + \epsilon}}, \quad U_t = Q_L U_t^{\text{rot}} Q_R^\top \quad (\text{rotate back}) \quad (68)$$

The key insight: all momentum tracking occurs in the well-conditioned eigenbasis, simplifying numerical stability and theoretical analysis. SOAP combines benefits of Shampoo (explicit curvature structure) and Adam (proven momentum mechanics), while being cleaner and often more stable than full Shampoo. Eigendecompositions are recomputed every K steps, amortizing the cost.

Algorithm 17 SOAP (Shampoo with Adam in Eigenbasis)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , eigendecomposition interval K

Ensure: Updated parameters θ_T

```

1: Initialize:  $L \leftarrow \epsilon I_m, R \leftarrow \epsilon I_n, m \leftarrow 0, v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $G \leftarrow \nabla f(\theta_{t-1})$  ▷ Gradient
4:   Update Gram:  $L \leftarrow L + G G^\top, R \leftarrow R + G^\top G$ 
5:   if  $t \bmod K == 0$  then
6:      $Q_L, \Lambda_L \leftarrow \text{eigh}(L), Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ 
7:   end if
8:    $G^{\text{rot}} \leftarrow Q_L^\top G Q_R$  ▷ Rotate gradient into eigenbasis
9:    $m \leftarrow \beta_1 m + (1 - \beta_1) G^{\text{rot}}$  ▷ Exponential moving average (bias-correct outside)
10:   $v \leftarrow \beta_2 v + (1 - \beta_2) (G^{\text{rot}})^2$  ▷ Second moment (bias-correct outside)
11:   $u \leftarrow m / (\sqrt{v} + \epsilon)$  ▷ Adaptive step in eigenbasis
12:   $\Delta \theta \leftarrow Q_L u Q_R^\top$  ▷ Rotate back to parameter space
13:   $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta \theta$ 
14: end for return  $\theta_T$ 

```

Lion (EvoLved Sign Momentum). Lion achieves **minimal memory overhead** by using sign-based updates instead of full adaptive scaling:

$$c_t = \beta_1 m_t + (1 - \beta_1) g_t \quad (\text{momentum input}) \quad (69)$$

$$\theta_{t+1} = \theta_t - \eta (\text{sign}(c_t) + \lambda \theta_t) \quad (\text{sign operator update}) \quad (70)$$

$$m_{t+1} = \beta_2 m_t + (1 - \beta_2) g_t \quad (\text{momentum accumulation}) \quad (71)$$

All element-wise scaling operations are replaced with **sign()**, which outputs $\{-1, 0, +1\}$. This dramatically reduces memory compared to Adam-like methods: Lion stores only the momentum buffer, no second moment variance. The tradeoff: sign-based updates sacrifice the element-wise learning-rate adaptation that makes Adam effective. Lion is positioned not as a universally superior optimizer but as a specialized low-memory, high-throughput alternative for scenarios where activation memory dominates and some optimizer sophistication can be sacrificed. Works well in large-batch regimes and when hardware throughput is the primary constraint.

Algorithm 18 Lion (EvoLved Sign Momentum)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , weight decay λ

Ensure: Updated parameters θ_T

- 1: Initialize: $m \leftarrow 0$ (momentum buffer)
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $g \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradient
 - 4: $c \leftarrow \beta_1 m + (1 - \beta_1)g$ ▷ Momentum input
 - 5: $\theta_t \leftarrow \theta_{t-1} - \eta(\text{sign}(c) + \lambda\theta_{t-1})$ ▷ Sign-based update with weight decay
 - 6: $m \leftarrow \beta_2 m + (1 - \beta_2)g$ ▷ Momentum accumulation (decoupled)
 - 7: **end for** **return** θ_T
-

Sophia (Second-Order Hessian Information with Optimized Approximation). Sophia uses **diagonal Hessian estimates** for curvature-aware scaling without the memory and compute cost of dense second-order methods.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t \quad (\text{first moment}) \quad (72)$$

$$h_t = \beta_2 h_{t-(k-1)} + (1 - \beta_2)\hat{h}_t \quad (\text{diagonal Hessian, updated every } k \text{ steps}) \quad (73)$$

$$\text{clip}(x, C) = \min(\max(x, -C), C) \quad (\text{element-wise clipping}) \quad (74)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \text{clip}\left(\frac{m_t}{\max(\gamma h_t, \epsilon)}, 1\right) \quad (75)$$

where $\hat{h}_t$ is a diagonal estimate (e.g., Hutchinson-trace estimator) and γ is a scaling factor. The diagonal Hessian h_t captures local curvature, allowing the optimizer to take smaller steps in sharp directions and larger steps in flat directions. The clipping operation $\text{clip}(\cdot, 1)$ prevents adaptive steps from exploding. Sophia belongs to the family of curvature-aware methods seeking better conditioning without the $O(n^2)$ or $O(n^3)$ cost of full second-order methods. Works particularly well in second-pass fine-tuning scenarios.

Algorithm 19 Sophia (Diagonal Hessian with Clipped Updates)

Require: Initial parameters θ_0 , learning rate η , β_1, β_2 , Hessian update freq k , clip threshold C

Ensure: Updated parameters θ_T

- 1: Initialize: $m \leftarrow 0$, $h \leftarrow \epsilon$ (diagonal Hessian estimate)
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $g \leftarrow \nabla f(\theta_{t-1})$
 - 4: $m \leftarrow \beta_1 m + (1 - \beta_1)g$ ▷ First moment (bias-correct outside)
 - 5: **if** $t \bmod k == 0$ **then**
 - 6: $\hat{h} \leftarrow \text{HutchinsonEstimate}(g)$ ▷ Diagonal Hessian via Hutchinson
 - 7: $h \leftarrow \beta_2 h + (1 - \beta_2)\hat{h}$ ▷ Update Hessian estimate
 - 8: **end if**
 - 9: $u \leftarrow \frac{m}{\max(\gamma h, \epsilon)}$ ▷ Adaptive step (element-wise)
 - 10: $u \leftarrow \text{clip}(u, C)$ ▷ Element-wise clipping to $[-C, C]$
 - 11: $\theta_t \leftarrow \theta_{t-1} - \eta \cdot u$
 - 12: **end for** **return** θ_T
-

Muon and Turbo-Muon (Orthogonality-Based Optimizers). Muon and Turbo-Muon are **orthogonality-oriented optimizers** that reshape update geometry using Newton-Schulz

polynomial iterations for orthogonalization. For a 2D parameter matrix W with gradient G_t :

$$X_0 = \frac{G_t}{\|G_t\|_F + \epsilon} \quad (\text{normalized gradient}) \quad (76)$$

$$A_k = X_k X_k^\top \quad (\text{Gramian}) \quad (77)$$

$$B_k = bA_k + cA_k^2 \quad (\text{polynomial step, coefficients } b, c \text{ from Newton-Schulz}) \quad (78)$$

$$X_{k+1} = aX_k + B_k X_k \quad (\text{iteration } k = 0, 1, \dots, 4) \quad (79)$$

$$W_{t+1} = W_t - \eta X_K \quad (\text{update with orthogonalized direction}) \quad (80)$$

Muon performs 5 Newton-Schulz iterations to produce an approximately orthogonal direction, ensuring updates respect geometric constraints. Turbo-Muon adds an **almost-orthogonal preconditioning** (AOL) step before iterations, reducing the number of required orthogonalization steps to 4. The rationale: orthogonal updates preserve norms during training, avoiding the norm creep observed in momentum-based methods. These optimizers show promise on large-scale training but require custom CUDA kernels for efficiency. Tradeoff: high per-step compute (matrix multiplications and polynomial iterations) versus fundamentally better-conditioned update geometry.

Algorithm 20 Muon (Newton-Schulz Orthogonalization)

Require: Initial parameters θ (matrices), learning rate η , Newton-Schulz iters K

Ensure: Updated parameters θ

```

1: for each 2D parameter matrix  $W$  do
2:    $G \leftarrow \nabla f(W)$  ▷ Gradient
3:    $X_0 \leftarrow \frac{G}{\|G\|_F + \epsilon}$  ▷ Normalize gradient by Frobenius norm
4:   for  $k = 0, 1, \dots, K - 1$  do
5:      $A_k \leftarrow X_k X_k^\top$  ▷ Gramian (cost:  $O(mn^2)$  or  $O(m^2n)$  for tall/wide)
6:      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$  ▷ Newton-Schulz coefficients
7:      $X_{k+1} \leftarrow B_k X_k$  ▷ Polynomial step
8:   end for
9:    $W \leftarrow W - \eta X_K$  ▷ Update with orthogonalized direction
10: end for return updated  $\theta$ 

```

Algorithm 21 Turbo-Muon (AOL-Preconditioned Orthogonalization)

Require: Initial parameters θ (matrices), learning rate η , Newton-Schulz iters K' (typically 4)

Ensure: Updated parameters θ

```

1: for each 2D parameter matrix  $W$  do
2:    $G \leftarrow \nabla f(W)$  ▷ Gradient
3:    $X_0 \leftarrow \frac{G}{\|G\|_F + \epsilon}$  ▷ Normalize gradient
4:   AOL (Almost-Orthogonal preconditioner): ▷ Preconditioning step
5:    $P \leftarrow (1.5I - 0.5X_0 X_0^\top)$  ▷ Preconditioning matrix
6:    $X_0 \leftarrow P X_0$  ▷ Preconditioned start
7:   for  $k = 0, 1, \dots, K' - 1$  do
8:      $A_k \leftarrow X_k X_k^\top$ 
9:      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$ 
10:     $X_{k+1} \leftarrow B_k X_k$  ▷ Reduced iterations due to AOL preconditioning
11:   end for
12:    $W \leftarrow W - \eta X_{K'}$  ▷ Update with preconditioned orthogonalized direction
13: end for return updated  $\theta$ 

```

Group	Methods	Primary Goal	Interpretation from the Survey
Classical baseline	SGD, AdamW, RAdam	stability and reference baselines	These define the comparison floor for newer optimizer claims.
Momentum re-design	Adan, AdEMAMix, MARS, Cautious, AdamW	faster or safer first-order adaptation	Best when convergence speed or noisy-gradient stability is the main concern.
Large-batch and schedule simplification	LAMB, Schedule-Free, AdamW	operational robustness at scale	Reduce brittleness from batch-size growth or schedule engineering.
Memory-efficient	Adafactor, GaLore, APOLLO, APOLLO-Mini, Q-APOLLO, Lion	optimizer-state reduction	Most useful when VRAM is dominated by optimizer state rather than activations; APOLLO-family methods replace dense AdamW moments with projected or quantized structured scaling.
Curvature-aware	Shampoo, Sophia, SOAP	better conditioning	Prefer when richer geometry is worth implementation and compute overhead.
Geometry-oriented	Muon, Turbo-Muon	orthogonalized update structure	Specialized options for matrix geometry and representation shaping.

B Annex B: Dense, Recurrent, and Memory-Augmented Transformers

This annex synthesizes the dense, recurrent, and memory-augmented transformer families. The field has evolved toward several primary paradigms: dense global attention with variants, key-value head compression via grouped-query attention, recurrent state compression with decay, selective state-space models, continuous-depth numerical integration, and memory-augmented architectures. Understanding this taxonomy illuminates fundamental tradeoffs between expressiveness, computational cost, memory footprint, and deployment simplicity.

B.1 Dense Attention Baselines: Standard and Sigmoid

B.1.1 Standard Softmax Attention

Standard multi-head self-attention [50] computes scaled dot-product similarity between token embeddings:

- 1: **Input:** Query matrix $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key matrix $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value matrix $\mathbf{V} \in \mathbb{R}^{n \times d}$
- 2: $\text{scores} \leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \in \mathbb{R}^{n \times n}$ ▷ compute pairwise similarities
- 3: $\text{attention_weights} \leftarrow \text{softmax}(\text{scores}, \text{axis} = 1) \in \mathbb{R}^{n \times n}$ ▷ normalize across keys
- 4: **Output:** $\mathbf{Y} = \text{attention_weights} \cdot \mathbf{V} \in \mathbb{R}^{n \times d}$ ▷ weighted value combination

For autoregressive generation, KV caching stores past keys and values to avoid $\mathcal{O}(n^2)$ recomputation. However, this linearly growing cache occupies $\sim n \cdot d_{\text{hidden}}$ bytes, which can consume hundreds of gigabytes for billion-parameter models. Standard attention achieves **perfect expressiveness** within the context window—any token can attend to any other token with learned weights—but pays the price of dense computation.

- **Training complexity:** $\mathcal{O}(n^2 \cdot d)$ time, $\mathcal{O}(n^2)$ space (attention matrix materialization).

- **Inference complexity:** $\mathcal{O}(n)$ time per token, $\mathcal{O}(n)$ space (KV cache).
- **Strengths:** Unparalleled expressiveness; perfect history recall; highly parallelizable.
- **Weaknesses:** Quadratic bottleneck prohibits very long contexts; KV cache dominates memory during generation.

B.1.2 Sigmoid Attention

Sigmoid attention replaces row-wise softmax with element-wise sigmoid activation [41]:

- 1: **Input:** $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ as above, learnable bias $\mathbf{b} \in \mathbb{R}^{n \times n}$
- 2: logits $\leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{b}$
- 3: attention_weights $\leftarrow \sigma(\text{logits})$ ▷ element-wise sigmoid, not softmax
- 4: **Output:** $\mathbf{Y} = \text{attention_weights} \odot \mathbf{V}$

Unlike softmax, sigmoid does not enforce a probability distribution (weights need not sum to 1), enabling stronger token independence. Theoretical analysis via mixture-of-experts shows sigmoid achieves superior sample complexity: $\mathcal{O}(n^{-0.51})$ convergence for ReLU experts versus softmax’s $\mathcal{O}(n^{-0.24})$. However, empirical training revealed gradient instabilities at scale. The remedy is **hybrid-norm**—adding normalization after the attention output—which stabilizes gradients without sacrificing the theoretical benefits of element-wise gating.

- **Training complexity:** $\mathcal{O}(n^2 \cdot d)$ (identical to standard), but element-wise operations enable 17% inference speedup via FlashSigmoid.
- **Inference complexity:** $\mathcal{O}(n)$ per token with KV cache (asymptotically same, but lower constant factors).
- **Strengths:** Overcomes zero-sum competition; avoids row-wise synchronization; hardware-friendly implementation.
- **Weaknesses:** Requires careful stabilization (hybrid-norm); training instability at large scales without auxiliary loss.

B.1.3 Grouped-Query Attention (GQA)

Grouped-query attention [1] bridges multi-head attention (MHA) and multi-query attention (MQA) by partitioning query heads into groups, each sharing a single key-value head. If h is the total number of attention heads, h_k is the number of key-value heads, and $G = h/h_k$ is the group size, then every group of G query heads shares one KV head:

- 1: **Input:** $\mathbf{Q} \in \mathbb{R}^{n \times h \cdot d_h}$, $\mathbf{K} \in \mathbb{R}^{n \times h_k \cdot d_h}$, $\mathbf{V} \in \mathbb{R}^{n \times h_k \cdot d_h}$
- 2: $\mathbf{K}_{\text{full}} \leftarrow \text{repeat_interleave}(\mathbf{K}, G)$ ▷ expand from h_k to h heads
- 3: $\mathbf{V}_{\text{full}} \leftarrow \text{repeat_interleave}(\mathbf{V}, G)$
- 4: scores $\leftarrow \frac{\mathbf{Q}\mathbf{K}_{\text{full}}^\top}{\sqrt{d_h}} \in \mathbb{R}^{n \times n}$ ▷ standard dot product after expansion
- 5: attention_weights $\leftarrow \text{softmax}(\text{scores}, \text{axis} = 1)$
- 6: **Output:** $\mathbf{Y} = \text{attention_weights} \cdot \mathbf{V}_{\text{full}} \in \mathbb{R}^{n \times h \cdot d_h}$

GQA interpolates smoothly between MHA ($h_k = h$, $G = 1$) and MQA ($h_k = 1$, $G = h$). The key advantage is a tunable tradeoff between KV-cache footprint and representational capacity. Each additional KV head consumes $2 \cdot d_h \cdot n$ extra bytes in the cache, so reducing h_k yields proportional memory savings.

- **Training complexity:** $\mathcal{O}(n^2 \cdot h \cdot d_h)$ (identical to MHA after head expansion).
- **Inference complexity:** $\mathcal{O}(n)$ per token, $\mathcal{O}(h_k \cdot n \cdot d_h)$ KV-cache space.

- **Strengths:** Flexible head-count tradeoff; proven at scale (Llama 2 & 3 use $h_k = 8$ KV heads with $h = 32$ query heads); reduces KV-cache by $G\times$; supports fast inference with FlashAttention.
- **Weaknesses:** Fewer KV heads can reduce representational capacity in small models; requiring h_k to divide h constrains architectural choices.

B.2 Recurrent and Retentive Architectures

B.2.1 Retentive Networks (RetNet)

RetNet [48] unifies three computation paradigms: parallel training, recurrent inference, and chunkwise deployment. Its core innovation is the **retention mechanism**, which uses a fixed exponential decay matrix to model temporal importance:

- 1: **Parallel (training) form:**
- 2: $\text{decay_matrix}[i, j] \leftarrow \gamma^{i-j}$ for $i \geq j$, else 0 ▷ causal exponential decay
- 3: $\text{decay_matrix}[i, j] \leftarrow 0$ for $i < j$ ▷ causal masking
- 4: $\mathbf{Y}_{\text{parallel}} \leftarrow (\mathbf{Q}\mathbf{K}^\top \odot \text{decay_matrix})\mathbf{V}$ ▷ element-wise multiplication with decay
- 1: **Recurrent (inference) form:**
- 2: **for** $t = 1, \dots, n$ **do**
- 3: $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$ ▷ state update with decay
- 4: $\mathbf{y}_t \leftarrow \mathbf{q}_t \mathbf{s}_t$ ▷ output via query-state interaction
- 5: **end for**

The decay scalar $\gamma \in (0, 1)$ controls the temporal window. RetNet uses **multi-scale retention** with different γ values per head (e.g., $\gamma = 1 - 2^{-5}, 1 - 2^{-6}, \dots$), allowing short-term and long-term dependencies simultaneously. Chunkwise recurrent mode divides sequences into chunks, processes each chunk in parallel, and threads a recurrent state between chunks.

- **Training complexity:** $\mathcal{O}(n^2 \cdot d)$ (parallel), or $\mathcal{O}(n \cdot c \cdot d)$ (chunkwise recurrent with chunk size c).
- **Inference complexity:** $\mathcal{O}(1)$ per token, $\mathcal{O}(d^2)$ state space (fixed matrix).
- **Strengths:** Constant-time inference; triple computation paradigm; multi-scale consolidation.
- **Weaknesses:** Fixed decay imposes rigid inductive bias; may truncate learned long-range patterns.

B.2.2 Mamba: Selective State-Space Models

Mamba [17] frames recurrence as a continuous-time dynamical system with input-dependent parameters, achieving both linear training and constant-time inference:

- 1: **Continuous dynamics:** $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t)$, $y(t) = \mathbf{C}h(t)$
- 2: **Discretization with step size Δ_t :**
- 3: $\bar{\mathbf{A}}_t \leftarrow \exp(\Delta_t \mathbf{A})$
- 4: $\bar{\mathbf{B}}_t \leftarrow (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B}_t$
- 5: **Recurrent update:**
- 6: **for** $t = 1, \dots, n$ **do**
- 7: $\Delta_t \leftarrow \text{softplus}(\text{Linear}(x_t))$ ▷ input-dependent step size
- 8: $\mathbf{B}_t \leftarrow \text{Linear}(x_t)$, $\mathbf{C}_t \leftarrow \text{Linear}(x_t)$ ▷ input-dependent projection matrices
- 9: $h_t \leftarrow \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t x_t$ ▷ state transition
- 10: $y_t \leftarrow \mathbf{C}_t h_t$ ▷ output projection
- 11: **end for**

The critical innovation is that $\mathbf{A}$ (the system matrix) is *not* input-dependent, but Δ_t , $\mathbf{B}_t$, and $\mathbf{C}_t$ are, making the system **time-varying**. This selectivity allows the model to *ignore* irrelevant information by setting Δ_t near zero, effectively creating a gate. A hardware-aware parallel scan algorithm implements the recurrence efficiently on GPUs by fusing computation within SRAM, avoiding expensive HBM bandwidth.

- **Training complexity:** $\mathcal{O}(n \cdot d)$ via hardware-aware scan (linear in sequence length).
- **Inference complexity:** $\mathcal{O}(1)$ per token, $\mathcal{O}(d)$ state (hidden vector).
- **Strengths:** Achieves linear training and constant-inference simultaneously; practical on long sequences (millions of tokens).
- **Weaknesses:** State vector compression can weaken exact copying and dense associative recall versus full attention.

B.3 Continuous-Depth Transformers: ODE Integration

B.3.1 ODE Transformer

The ODE Transformer [63] interprets network depth as numerical integration of a continuous dynamical system, using higher-order Runge-Kutta solvers to reduce truncation error:

- 1: **Continuous formulation:** $\frac{dh(t)}{dt} = f_\theta(h(t), t)$ where f_θ is the transformer sub-network
- 2: **Runge-Kutta-4 discrete approximation:**
- 3: $k_1 \leftarrow f_\theta(h_t, t)$
- 4: $k_2 \leftarrow f_\theta(h_t + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)$
- 5: $k_3 \leftarrow f_\theta(h_t + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)$
- 6: $k_4 \leftarrow f_\theta(h_t + k_3, t + \Delta t)$
- 7: $h_{t+1} \leftarrow h_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ ▷ weighted combination

Instead of simple Euler residual connections $h_{t+1} = h_t + f_\theta(h_t)$, the RK4 block computes four intermediate evaluations and combines them with classical RK4 weights. To avoid vanishing gradients, the architecture introduces **learned gating** that interpolates between intermediate approximations:

- 1: $g \leftarrow \sigma(\text{Linear}([k_1, k_2, k_3, k_4]))$ ▷ learnable gate
- 2: $h_{t+1} \leftarrow h_t + g \cdot k_1 + (1 - g) \cdot k_2$ ▷ interpolated refinement

This formulation reduces the effective number of parameters through weight sharing—the same f_θ is evaluated multiple times—while providing richer trajectory refinement.

- **Training complexity:** $\mathcal{O}(k \cdot n^2 \cdot d)$ where k is the RK order (4 for RK4).
- **Inference complexity:** $\mathcal{O}(k \cdot n)$ per token (higher constant overhead).
- **Strengths:** Significantly higher accuracy on generation tasks (state-of-the-art BLEU); parameter-efficient via weight sharing.
- **Weaknesses:** High per-step compute; inference latency increases by constant factor k ; complex gating required for stability.

B.4 Test-Time Memory: Titans

The Titans architecture [4] introduces an orthogonal dimension: instead of static weights, the model maintains a learnable memory that is *updated during inference* based on a surprise-driven signal:

- 1: **Local short-term attention:** Apply standard or sparse attention within a fixed context window c

- 2: $y_t^{\text{local}} \leftarrow \text{Attention}(q_t, k_{[t-c:t]}, v_{[t-c:t]})$
- 3: **Memory update signal (surprise):**
- 4: $S_t \leftarrow \eta_t S_{t-1} - \theta_t \nabla_{\ell}(\mathcal{M}_{t-1}; x_t)$ ▷ surprise as gradient of loss w.r.t. memory
- 5: $\mathcal{M}_t \leftarrow (1 - \alpha_t) \mathcal{M}_{t-1} + S_t$ ▷ memory updated via EMA
- 6: **Long-term memory retrieval:**
- 7: $y_t^{\text{memory}} \leftarrow \mathcal{M}_t^*(q_t)$ ▷ query memory module for retrieved information
- 8: **Output combination:**
- 9: $y_t \leftarrow y_t^{\text{local}} + \text{gate}(y_t^{\text{memory}})$ ▷ combine local and retrieved memory

The memory module $\mathcal{M}$ is literally updated during the forward pass by computing gradients of an associative loss and applying SGD steps with momentum. The decay rates $\eta_t, \theta_t, \alpha_t$ are themselves input-dependent, allowing the model to switch memory paradigms when context shifts.

- **Training complexity:** Roughly $\mathcal{O}(c^2 + n \cdot f)$ where c is local window and f is memory update overhead.
- **Inference complexity:** $\mathcal{O}(c^2)$ local attention plus $\mathcal{O}(1)$ memory retrieval per token.
- **Strengths:** Handles extreme context lengths; enables true associative recall; memory adapts to input.
- **Weaknesses:** Significantly more complex; inference includes gradient computation; higher coordination overhead.

B.5 Pattern-Driven Mixer Dispatch

The codebase selects a mixer per block from the configured `layer_pattern`. The dispatch logic enforces the training-free policy for eval-only blocks and routes to the corresponding family implementation.

B.6 Architectural Comparison and Synthesis

Architecture	Train	Infer	State	Key Characteristic
Standard Attention	$\mathcal{O}(n^2d)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(nd)$	Perfect expressiveness, full token routing, KV bottleneck.
GQA	$\mathcal{O}(n^2d)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(h_k \cdot n \cdot d_h)$	Tunable KV compression, proven at scale, $G \times$ cache reduction.
Sigmoid Attention	$\mathcal{O}(n^2d)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(nd)$	Element-wise gating, hardware-efficient, stable at scale with hybrid-norm.
RetNet	$\mathcal{O}(n^2d)$	$\mathcal{O}(1)$	$\mathcal{O}(d^2)$	Multi-scale decay, triple computation mode, fixed forgetting pattern.
Mamba	$\mathcal{O}(nd)$	$\mathcal{O}(1)$	$\mathcal{O}(d)$	Selective input-dependent dynamics, linear training and inference, hardware scan.
ODE Transformer	$\mathcal{O}(kn^2d)$	$\mathcal{O}(kn)$	$\mathcal{O}(nd)$	Numerical integration refinement, weight sharing via stages, higher accuracy.

Architecture	Train	Infer	State	Key Characteristic
Titans	$O(c^2 + nf)$	$O(c^2 + 1)$	$O(1)$	Test-time memory adaptation, extreme context, on-inference parameter updates.

The field exhibits a clear progression along two axes: (i) **computational efficiency**, moving from $O(n^2)$ to $O(n)$ or $O(1)$, and (ii) **memory adaptivity**, shifting from static weights to dynamic, test-time updated models. Standard attention remains the expressiveness baseline; Mamba and RetNet represent the practical efficiency frontier; Titans introduces an orthogonal innovation axis (test-time learning). The choice of architecture reflects the fundamental engineering constraint: expressiveness versus deployment cost. The latent-attention family, which compresses the KV cache into a low-rank latent, is covered in Appendix C.

C Annex B: Latent and Low-Rank Attention Families

This annex covers the latent / low-rank KV compression family of attention mechanisms. These architectures compress the per-token key–value state into a low-rank latent vector that is the only quantity materialised in the KV cache during decoding, then reconstruct full multi-head keys and values on the fly. The family unifies grouped-query, latent, factorised, and temporal-latent designs under a common low-rank bottleneck.

C.1 Multi-Head Latent Attention (MLA)

Multi-Head Latent Attention (MLA) [33] compresses the per-token key–value state into a single low-rank latent vector c_{KV} of rank r_{kv} , which is the only quantity materialised in the KV cache during decoding. A pair of up-projection matrices reconstructs the full multi-head keys and values on the fly, while RoPE is applied to the decompressed query and key tensors so that positional structure is preserved. The authors show that the Pareto-optimal latent width sits near $r_{kv} = d_{\text{hidden}}/2$, at which point MLA halves the KV cache yet matches multi-head attention (MHA) quality on small models.

C.1.1 Mathematical Formulation

$$c_{KV} = W_{DKV} x \in \mathbb{R}^{r_{kv}}, \quad k = W_{UK} c_{KV}, \quad v = W_{UV} c_{KV}, \quad q = W_Q x.$$

$$\tilde{q} = \text{RoPE}(q), \quad \tilde{k} = \text{RoPE}(k), \quad \text{attn} = \text{softmax}\left(\frac{\tilde{q} \tilde{k}^\top}{\sqrt{d_h}}\right) v.$$

C.1.2 Algorithmic Pseudocode

- 1: **Input:** hidden state $x \in \mathbb{R}^{n \times d}$, rank r_{kv} , projections $W_{DKV}, W_{UK}, W_{UV}, W_Q$
- 2: $c_{KV} \leftarrow W_{DKV} x$ ▷ down-project to latent of rank r_{kv} (cached at inference)
- 3: $k \leftarrow W_{UK} c_{KV}, \quad v \leftarrow W_{UV} c_{KV}$ ▷ up-project to full heads
- 4: $q \leftarrow W_Q x$
- 5: $\tilde{q} \leftarrow \text{RoPE}(q), \quad \tilde{k} \leftarrow \text{RoPE}(k)$ ▷ apply rotary embeddings
- 6: $\text{weights} \leftarrow \text{softmax}(\tilde{q} \tilde{k}^\top / \sqrt{d_h})$
- 7: **Output:** $Y = \text{weights} \cdot v$

C.1.3 Key Characteristics

- **Training complexity:** $\mathcal{O}(n^2 \cdot d)$ (full attention on decompressed heads).
- **Inference complexity:** $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ KV-cache space (latent only).

Algorithm 22 Pattern-Driven Mixer Forward (Conceptual)

Require: Hidden states H , pattern P , layer index ℓ

```
1:  $m \leftarrow P[\ell \bmod |P|]$ 
2: if  $m \in \{\text{fasa\_attn}, \text{sparse\_attn}\}$  and model is in training mode then
3:   raise configuration/runtime error (training-free block in train mode)
4: else if  $m = \text{standard\_attn}$  then
5:    $H \leftarrow \text{softmax-attention}(H)$ 
6: else if  $m = \text{sigmoid\_attn}$  then
7:    $H \leftarrow \text{sigmoid-attention}(H)$ 
8: else if  $m \in \{\text{retnet}, \text{retnet\_attn}\}$  then
9:    $H \leftarrow \text{retention}(H)$ 
10: else if  $m = \text{mamba}$  then
11:    $H \leftarrow \text{selective-ssm}(H)$ 
12: else if  $m = \text{ode}$  then
13:    $H \leftarrow \text{rk-step}(H)$ 
14: else if  $m$  is a sparse attention key then
15:    $H \leftarrow \text{sparse-attention-family}(H)$ 
16: else if  $m$  is a gated attention key then
17:    $H \leftarrow \text{gated-attention-family}(H)$ 
18: else
19:    $H \leftarrow \text{memory-augmented-attn}(H)$ 
20: end if
21: return  $H$ 
```

- **Trainable:** fully trainable; r_{kv} is a hyperparameter (Pareto-optimal $\approx d/2$).
- **Strengths:** Halves KV cache versus MHA at matched quality; RoPE-compatible; clean latent bottleneck.
- **Limitations:** Up-projection adds decode FLOPs; latent rank is a fixed global budget shared across heads.

C.2 Group-Query Latent Attention (GQLA)

Group-Query Latent Attention (GQLA) [34] is a minimal modification of MLA that exposes two algebraically equivalent decoding paths over the same trained weights: an MQA-absorb path (fusing the up-projection into the query, optimal on H100-class hardware) and a GQA-expanded path (materialising full keys/values, optimal on H20 / multi-token-prediction hardware). The two paths are mathematically identical because the up-projection is linear, so $\text{softmax}(q_{\text{absorbed}} \cdot c_{KV}^\top) = \text{softmax}((q \cdot W_{UK}^\top) \cdot c_{KV}^\top)$, allowing hardware-adaptive dispatch with no retraining.

C.2.1 Mathematical Formulation

Same latent as MLA:

$$c_{KV} = W_{DKV} x, \quad k = W_{UK} c_{KV}, \quad v = W_{UV} c_{KV}, \quad q = W_Q x.$$

$$\text{Path A (MQA-absorb): } \text{attn} = \text{softmax}\left(\frac{(q W_{UK}^\top) c_{KV}^\top}{\sqrt{d_h}}\right) (W_{UV} c_{KV}).$$

$$\text{Path B (GQA-expanded): } \text{attn} = \text{softmax}\left(\frac{q k^\top}{\sqrt{d_h}}\right) v, \quad k = W_{UK} c_{KV}, \quad v = W_{UV} c_{KV}.$$

C.2.2 Algorithmic Pseudocode

```

1: Input:  $x$ , weights  $W_{DKV}, W_{UK}, W_{UV}, W_Q$ , hardware flag  $hw \in \{H100, H20\}$ 
2:  $c_{KV} \leftarrow W_{DKV} x$  ▷ shared latent, cached
3:  $q \leftarrow W_Q x$ 
4: if  $hw == H100$  then ▷ MQA-absorb path
5:    $q_{\text{abs}} \leftarrow q W_{UK}^\top$  ▷ absorb up-projection into query
6:    $\text{weights} \leftarrow \text{softmax}(q_{\text{abs}} c_{KV}^\top / \sqrt{d_h})$ 
7:    $Y \leftarrow \text{weights} \cdot (W_{UV} c_{KV})$ 
8: else ▷ GQA-expanded path
9:    $k \leftarrow W_{UK} c_{KV}, \quad v \leftarrow W_{UV} c_{KV}$ 
10:   $\text{weights} \leftarrow \text{softmax}(q k^\top / \sqrt{d_h})$ 
11:   $Y \leftarrow \text{weights} \cdot v$ 
12: end if
13: Output:  $Y$ 

```

C.2.3 Key Characteristics

- **Training complexity:** $\mathcal{O}(n^2 \cdot d)$, identical to MLA.
- **Inference complexity:** $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ latent state; path chosen by hardware.
- **Trainable:** trained once; the two decoding paths reuse the same weights.
- **Strengths:** Hardware-adaptive decode with no quality loss; algebraic equivalence guarantees consistency.
- **Limitations:** Same latent-rank tradeoffs as MLA; path selection is a deployment-time heuristic.

C.3 Multi-Head Low-Rank Attention (MLRA)

Multi-Head Low-Rank Attention (MLRA) [30] refactors the single MLA latent into L disjoint sub-heads each of rank r/L , so that the KV cache can be partitioned across L devices for 4-way tensor-parallel decoding (each device loads only $1/L$ of the cache). Per-sub-head up-projections reconstruct the corresponding slice of the full heads and are concatenated, yielding a $2.8\times$ decode speedup over MLA while preserving the global latent budget.

C.3.1 Mathematical Formulation

$$\begin{aligned}
c_{KV} &= [c_1, c_2, \dots, c_L], \quad c_i \in \mathbb{R}^{r/L}, \quad c_{KV} = W_{DKV} x. \\
k_i &= W_{UK}^{(i)} c_i, \quad v_i = W_{UV}^{(i)} c_i, \quad k = [k_1; \dots; k_L], \quad v = [v_1; \dots; v_L]. \\
\text{attn} &= \text{softmax}\left(\frac{q k^\top}{\sqrt{d_h}}\right) v.
\end{aligned}$$

C.3.2 Algorithmic Pseudocode

```

1: Input:  $x$ , sub-head count  $L$ , per-sub-head projections  $W_{UK}^{(i)}, W_{UV}^{(i)}$ 
2:  $c_{KV} \leftarrow W_{DKV} x = [c_1, \dots, c_L]$  ▷ partition latent into  $L$  blocks
3: for  $i = 1, \dots, L$  (parallel across  $L$  devices) do
4:    $k_i \leftarrow W_{UK}^{(i)} c_i, \quad v_i \leftarrow W_{UV}^{(i)} c_i$ 
5: end for
6:  $k \leftarrow \text{concat}(k_1, \dots, k_L), \quad v \leftarrow \text{concat}(v_1, \dots, v_L)$ 

```

- 7: $\text{weights} \leftarrow \text{softmax}(q k^\top / \sqrt{d_h})$
- 8: **Output:** $\mathbf{Y} = \text{weights} \cdot v$

C.3.3 Key Characteristics

- **Training complexity:** $\mathcal{O}(n^2 \cdot d)$ (full attention on concatenated heads).
- **Inference complexity:** $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ state partitioned across L devices.
- **Trainable:** fully trainable; L controls partition granularity.
- **Strengths:** 2.8× decode speedup; clean 4-way tensor-parallel cache sharding; preserves total latent budget.
- **Limitations:** Requires L to divide r_{kv} ; more projection matrices than MLA; inter-device all-reduce on output.

C.4 Tucker Attention

Tucker Attention [23] unifies MHA, GQA and MLA under a single Tucker-style factorisation of the query/key/value weight tensors with independent ranks $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$. MHA is recovered when all ranks equal the hidden size, GQA when $k_{\text{rank}} = v_{\text{rank}} < d$, and MLA when $k_{\text{rank}} = v_{\text{rank}}$ equals the latent rank. Because the factorisation is a pure linear reparameterisation, the method is fully compatible with FlashAttention and RoPE, and yields an order of magnitude fewer parameters for comparable metrics.

C.4.1 Mathematical Formulation

$$\mathbf{Q} = W_{Q,\text{core}} W_{Q,\text{factor}} x, \quad \mathbf{K} = W_{K,\text{core}} W_{K,\text{factor}} x, \quad \mathbf{V} = W_{V,\text{core}} W_{V,\text{factor}} x.$$

$$\text{attn} = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d_h}}\right) \mathbf{V}.$$

Special cases: MHA $\Leftrightarrow q_{\text{rank}} = k_{\text{rank}} = v_{\text{rank}} = d$; GQA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} < d$; MLA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} = r_{kv}$.

C.4.2 Algorithmic Pseudocode

- 1: **Input:** x , ranks $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$, core/factor matrices
- 2: $\mathbf{Q} \leftarrow W_{Q,\text{core}} W_{Q,\text{factor}} x$ ▷ rank- q_{rank} factorised query
- 3: $\mathbf{K} \leftarrow W_{K,\text{core}} W_{K,\text{factor}} x$ ▷ rank- k_{rank} factorised key
- 4: $\mathbf{V} \leftarrow W_{V,\text{core}} W_{V,\text{factor}} x$ ▷ rank- v_{rank} factorised value
- 5: **Apply RoPE to $\mathbf{Q}, \mathbf{K}$ if positional encoding is used**
- 6: $\text{weights} \leftarrow \text{softmax}(\mathbf{Q} \mathbf{K}^\top / \sqrt{d_h})$ ▷ FlashAttention-compatible
- 7: **Output:** $\mathbf{Y} = \text{weights} \cdot \mathbf{V}$

C.4.3 Key Characteristics

- **Training complexity:** $\mathcal{O}(n^2 \cdot d)$ on the reconstructed tensors.
- **Inference complexity:** $\mathcal{O}(n)$ per token; cache size governed by k_{rank} .
- **Trainable:** ranks are hyperparameters; the factorisation is a drop-in weight reparameterisation.
- **Strengths:** Unifies MHA/GQA/MLA; order-of-magnitude parameter reduction at matched metrics; FlashAttention/RoPE compatible.

- **Limitations:** Two matmuls per projection add latency; rank selection is a per-task tuning problem.

C.5 Interleaved Head Attention (IHA)

Interleaved Head Attention (IHA) [15] constructs P pseudo-heads per original head (typically $P = H$) where each pseudo query/key/value is a learned linear combination of all H original heads. This induces up to P^2 distinct attention patterns per head at an $\mathcal{O}(H^2P)$ overhead. Theoretically, IHA needs only $\Theta(\sqrt{k} n^2)$ parameters versus $\Theta(k n^2)$ for MHA on the Polynomial task; empirically it delivers +10–20% on RULER multi-key retrieval, +5.8% on GSM8K and +2.8% on MATH-500.

C.5.1 Mathematical Formulation

$$q_{\text{pseudo}}[p] = \sum_{h=1}^H \text{mix}_q[p, h] q[h], \quad k_{\text{pseudo}}[p] = \sum_h \text{mix}_k[p, h] k[h], \quad v_{\text{pseudo}}[p] = \sum_h \text{mix}_v[p, h] v[h].$$

$$\text{attn}_p = \text{softmax}\left(\frac{q_{\text{pseudo}}[p] k_{\text{pseudo}}[p]^\top}{\sqrt{d_h}}\right) v_{\text{pseudo}}[p], \quad \mathbf{Y}[h] = \frac{1}{P} \sum_p \text{attn}_p[h].$$

C.5.2 Algorithmic Pseudocode

- 1: **Input:** per-head $q[h], k[h], v[h]$, mixing matrices $\text{mix}_q, \text{mix}_k, \text{mix}_v \in \mathbb{R}^{P \times H}$
- 2: **for** $p = 1, \dots, P$ **do**
- 3: $q_p \leftarrow \sum_h \text{mix}_q[p, h] q[h]$ ▷ learned mix across heads
- 4: $k_p \leftarrow \sum_h \text{mix}_k[p, h] k[h], \quad v_p \leftarrow \sum_h \text{mix}_v[p, h] v[h]$
- 5: $\text{attn}_p \leftarrow \text{softmax}(q_p k_p^\top / \sqrt{d_h}) v_p$
- 6: **end for**
- 7: **Output:** $\mathbf{Y}[h] \leftarrow \frac{1}{P} \sum_p \text{attn}_p[h]$ ▷ average pseudo-head outputs per head

C.5.3 Key Characteristics

- **Training complexity:** $\mathcal{O}(n^2 \cdot H \cdot P)$ with $P \approx H$, i.e. $\mathcal{O}(n^2 \cdot H^2)$.
- **Inference complexity:** $\mathcal{O}(n)$ per token with KV cache; extra $\mathcal{O}(H^2P)$ mixing overhead.
- **Trainable:** mixing matrices are learned; P is a hyperparameter.
- **Strengths:** $\Theta(\sqrt{k})$ parameter efficiency on Polynomial; large gains on retrieval and reasoning benchmarks.
- **Limitations:** Quadratic-in-heads mixing cost; more attention patterns to compute; added latency at large H .

C.6 Grouped-head laTenT Attention (GTA)

Grouped-head laTenT Attention (GTA) [47] combines two compressions: (1) a *shared attention map* — one attention-score tensor per group of heads, reused across all heads in the group, shrinking the key cache; and (2) a *nonlinear value decoder* — the value cache is compressed to a latent by a down-projection and reconstructed by a SiLU-gated up-projection before the output projection. GTA cuts attention FLOPs by up to 62.5% versus GQA, shrinks the KV cache by up to 70%, and yields a 2× end-to-end inference speedup.

C.6.1 Mathematical Formulation

$$q_g = \text{mean}(q[\text{group } g]), \quad k_g = \text{mean}(k[\text{group } g]), \quad \text{attn}_g = \text{softmax}\left(\frac{q_g k_g^\top}{\sqrt{d_h}}\right) \quad (\text{reused across the group}).$$

$$v_{\text{latent}} = W_{DV} x, \quad v = \text{SiLU}(W_{UV} v_{\text{latent}}), \quad \mathbf{Y} = \text{attn}_g \cdot v.$$

C.6.2 Algorithmic Pseudocode

```

1: Input:  $q, k$  per head, hidden  $x$ , group partition  $\{g\}$ 
2:  $v_{\text{latent}} \leftarrow W_{DV} x$  ▷ compress values to latent (cached)
3:  $v \leftarrow \text{SiLU}(W_{UV} v_{\text{latent}})$  ▷ nonlinear value decode
4: for each group  $g$  do
5:    $q_g \leftarrow \text{mean}(q[g]), \quad k_g \leftarrow \text{mean}(k[g])$  ▷ shared per-group queries/keys
6:    $\text{attn}_g \leftarrow \text{softmax}(q_g k_g^\top / \sqrt{d_h})$ 
7:    $\mathbf{Y}[g] \leftarrow \text{attn}_g \cdot v[g]$  ▷ one map reused for all heads in  $g$ 
8: end for
9: Output:  $\mathbf{Y}$ 

```

C.6.3 Key Characteristics

- **Training complexity:** $\mathcal{O}(n^2 \cdot G \cdot d_h)$ where G is the number of groups (vs H heads for GQA).
- **Inference complexity:** $\mathcal{O}(n)$ per token; value cache $\mathcal{O}(r_v \cdot n)$, key cache reduced by group sharing.
- **Trainable:** group assignment and latent rank are trainable/hyperparameters.
- **Strengths:** up to 62.5% attention-FLOP reduction, 70% KV-cache shrink, $2\times$ end-to-end speedup.
- **Limitations:** Shared map reduces per-head diversity; nonlinear decoder adds latency; grouping must be chosen carefully.

C.7 Multi-head Temporal Latent Attention (MTLA)

Multi-head Temporal Latent Attention (MTLA) [13] extends MLA along the temporal axis: a hyper-network dynamically merges m temporally adjacent KV-cache vectors into a single slot, shrinking the temporal length by a factor of m . A stride-aware causal mask keeps parallel training consistent with autoregressive inference, yielding a $5.3\times$ decode speedup and $8.3\times$ GPU-memory reduction versus MHA on En-De speech translation.

C.7.1 Mathematical Formulation

$$c_{KV}^{(t)} = W_{DKV} x^{(t)} \in \mathbb{R}^{r_{kv}} \quad (\text{per-token latent}).$$

$$\bar{c}_{KV}^{(j)} = \sum_{i=0}^{m-1} \alpha_i c_{KV}^{(sj+i)}, \quad \alpha_i = \text{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)})),$$

where s is the stride and m the merge window. Queries attend over merged slots $\bar{c}_{KV}$ with a stride-aware causal mask M_{stride} :

$$\text{attn} = \text{softmax}\left(\frac{q \bar{k}^\top}{\sqrt{d_h}} + M_{\text{stride}}\right) \bar{v}, \quad \bar{k} = W_{UK} \bar{c}_{KV}, \quad \bar{v} = W_{UV} \bar{c}_{KV}.$$

C.7.2 Algorithmic Pseudocode

```

1: Input: per-token latents  $c_{KV}^{(t)}$ , merge window  $m$ , stride  $s$ 
2: Merge phase (hyper-network):
3: for each merged slot  $j = 0, 1, \dots$  do
4:    $\alpha_i \leftarrow \text{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)}))$  for  $i = 0, \dots, m-1$ 
5:    $\bar{c}_{KV}^{(j)} \leftarrow \sum_{i=0}^{m-1} \alpha_i c_{KV}^{(sj+i)}$  ▷ gated average over window
6: end for
7:  $\bar{k} \leftarrow W_{UK} \bar{c}_{KV}$ ,  $\bar{v} \leftarrow W_{UV} \bar{c}_{KV}$ 
8:  $\text{weights} \leftarrow \text{softmax}(q \bar{k}^\top / \sqrt{d_h} + M_{\text{stride}})$  ▷ stride-aware causal mask
9: Output:  $\mathbf{Y} = \text{weights} \cdot \bar{v}$ 

```

C.7.3 Key Characteristics

- **Training complexity:** $\mathcal{O}(n^2 \cdot d/m)$ on the merged sequence (length $\lceil n/m \rceil$) plus hyper-network cost.
- **Inference complexity:** $\mathcal{O}(n/m)$ per token over merged slots; $\mathcal{O}(r_{kv} \cdot n/m)$ state.
- **Trainable:** hyper-network and merge window m are learned/chosen; stride-aware mask enforces consistency.
- **Strengths:** $5.3\times$ decode speedup, $8.3\times$ GPU-memory reduction vs MHA; temporal compression orthogonal to rank compression.
- **Limitations:** Merge window can blur fine temporal detail; hyper-network adds compute; mask design is non-trivial.

C.8 Architectural Comparison and Synthesis

Architecture	Train	Infer	State	Key Characteristic
MLA	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(r_{kv} n)$	Latent KV of rank $r_{kv} \approx d/2$, halves cache, matches MHA quality.
GQLA	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(r_{kv} n)$	Same MLA weights, two algebraically equivalent hardware-adaptive decode paths.
MLRA	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(r_{kv} n)$ parti- tioned	Latent split into L sub-heads for 4-way tensor-parallel decode, $2.8\times$ speedup.
Tucker	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(k_{\text{rank}} n)$	Tucker factorisation unifies MHA/GQA/MLA, order-of-magnitude fewer parameters.
IHA	$\mathcal{O}(n^2 H^2)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(H n d_h)$	Pseudo-heads as learned head mixes, $\Theta(\sqrt{k})$ params, +10–20% RULER retrieval.
GTA	$\mathcal{O}(n^2 G d_h)$	$\mathcal{O}(n)$ cache	$\mathcal{O}(r_v n)$	Shared per-group map + SiLU value decoder, 62.5% FLOP cut, 70% cache shrink.
MTLA	$\mathcal{O}(n^2 d/m)$	$\mathcal{O}(n/m)$	$\mathcal{O}(r_{kv} n/m)$	Temporal merge of m KV slots, $5.3\times$ decode, $8.3\times$ memory reduction.

The latent family shows a clear progression along two axes: (i) **rank compression**, moving from full heads (MHA) to a shared latent (MLA) to partitioned sub-head latents (MLRA), and (ii) **deployment adaptivity**, moving from a single decode path to hardware-adaptive dispatch (GQLA) and temporal cache merging (MTLA). Tucker Attention provides the unifying algebraic frame: MHA, GQA, and MLA are all special cases of a Tucker factorisation with appropriate rank choices.

D Annex C: Comprehensive Sparse Attention Mechanisms

D.1 Executive Summary

Sparse attention mechanisms address the prohibitive $\mathcal{O}(n^2)$ computational and memory complexity of standard scaled dot-product attention by restricting the set of attended key-value pairs. Modern sparse attention spans a rich design space distinguished by four orthogonal dimensions: (i) **sparsity pattern** (fixed geometric, data-dependent, or frequency-based), (ii) **trainability** (end-to-end trained or inference-only), (iii) **sparsity unit** (individual tokens, local windows, or blocks), and (iv) **mechanism** (masking, selection, filtering, or compression). This annex synthesizes seven major sparse attention families—Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, and FASA—providing mathematical formulations, algorithmic pseudocode, and comprehensive architectural comparisons.

D.2 Sparse Transformer: Factorized Strided and Fixed Patterns

D.2.1 Mathematical Formulation

The Sparse Transformer [11] reduces quadratic complexity to $\mathcal{O}(n\sqrt{n})$ by factorizing sparse attention into two complementary sparse heads. Let n be the sequence length and $l = \lfloor \sqrt{n} \rfloor$ be the stride.

Strided attention head : Each position i attends to every l -th previous position:

$$\mathcal{A}_i^{(\text{strided})} = \{j : j \leq i, (i - j) \bmod l = 0\}$$

Fixed attention head : Each position i attends to positions within its local block plus fixed summary columns:

$$\mathcal{A}_i^{(\text{fixed})} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \cup \{j : j \bmod l \in \{l - c, \dots, l - 1\}\}$$

where c is a hyperparameter controlling the number of summary columns (typically $c = 1$ or $c = 2$).

The attention computation for each head h follows standard scaled dot-product rules restricted to the sparse set:

$$\text{Attn}_h(Q, K, V)_i = \text{softmax} \left(\frac{q_i^h K_{\mathcal{A}_i}^h \top}{\sqrt{d_k}} \right) V_{\mathcal{A}_i}^h$$

The key insight is that with the two factorized heads operating in parallel across a depth of L transformer layers, every position can reach every other position through a path of length at most $L + 1$, preserving long-range reachability with a fraction of dense attention cost.

D.2.2 Algorithmic Pseudocode

Algorithm 23 Sparse Transformer Attention: Strided + Fixed Dual Heads

Require: Query $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value $\mathbf{V} \in \mathbb{R}^{n \times d}$

Require: Stride $l = \lfloor \sqrt{n} \rfloor$, summary columns $c \in \{1, 2\}$

Ensure: Output $\mathbf{Y} \in \mathbb{R}^{n \times d}$

```

1: Head 1: Strided Attention
2: for  $i = 1, \dots, n$  do
3:    $\mathcal{A}_i \leftarrow \{j : j \leq i \text{ and } (i - j) \bmod l = 0\}$  ▷ Every  $l$ -th position
4:    $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
5:    $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$ 
6:    $\mathbf{y}_i^{(1)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
7: end for
8: Head 2: Fixed Block + Summary Attention
9: for  $i = 1, \dots, n$  do
10:   $\text{block\_start} \leftarrow \lfloor i/l \rfloor \cdot l$ ,  $\text{block\_end} \leftarrow \min(i + 1, \text{block\_start} + l)$ 
11:   $\mathcal{A}_i \leftarrow [\text{block\_start}, \text{block\_end}] \cup \{\text{cols from last } c \text{ columns}\}$ 
12:   $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
13:   $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$ 
14:   $\mathbf{y}_i^{(2)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
15: end for
16: Concatenate:  $\mathbf{Y} = [\mathbf{y}^{(1)} \parallel \mathbf{y}^{(2)}]$  and project via output linear layer return  $\mathbf{Y}$ 

```

D.2.3 Key Characteristics

- **Complexity:** $\mathcal{O}(n\sqrt{n} \cdot d)$ during training; $\mathcal{O}(n)$ per token during generation.
- **Trainable:** Yes; full backpropagation through selected positions.
- **Sparsity pattern:** Fixed and data-agnostic; patterns do not adapt to content.
- **Strength:** Structured reachability guarantees long-range dependencies; proven effective on long sequences (Enwik8, text images).
- **Limitation:** Fixed patterns may miss important but non-contiguous relationships; requires custom CUDA kernels for practical efficiency.

D.3 Longformer: Sliding Window, Dilation, and Global Tokens

D.3.1 Mathematical Formulation

Longformer [5] achieves linear $\mathcal{O}(n \cdot w)$ complexity by combining three complementary attention patterns.

Sliding window attention : Local neighborhood of size w :

$$\mathcal{A}_i^{(\text{window})} = \{j : |i - j| \leq w/2\}$$

Dilated sliding window : Windowed attention with dilation d , skipping stride $d + 1$:

$$\mathcal{A}_i^{(\text{dilated})} = \{j : |i - j| \leq (w/2)(d + 1) \text{ and } (i - j) \bmod (d + 1) = 0\}$$

Global attention : Designated global tokens (e.g., [CLS]) attend to all positions and are attended by all:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \dots, n\} & \text{if } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{window})} \cup \mathcal{G} & \text{otherwise} \end{cases}$$

The three patterns are applied via separate projections. The local and global attention can use either the same projections or task-specific separate ones.

D.3.2 Algorithmic Pseudocode

Algorithm 24 Longformer: Sliding Window + Dilated + Global

Require: Query $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key $\mathbf{K}$, Value $\mathbf{V}$, window size w , dilation d , global token indices $\mathcal{G}$

Ensure: Output $\mathbf{Y}$

```

1: for  $i = 1, \dots, n$  do
2:   if  $i \in \mathcal{G}$  then
3:      $\mathcal{A}_i \leftarrow \{1, \dots, n\}$  ▷ Global token attends to all
4:   else
5:     Local window:  $\mathcal{A}_i^{(\text{local})} \leftarrow [i - w/2, i + w/2] \cap [1, n]$ 
6:     Dilated offsets:  $\mathcal{A}_i^{(\text{dilated})} \leftarrow \{j : (i - j) \bmod (d + 1) = 0\} \cap \mathcal{A}_i^{(\text{dilated range})}$ 
7:     Combine:  $\mathcal{A}_i \leftarrow \mathcal{A}_i^{(\text{local})} \cup \mathcal{A}_i^{(\text{dilated})} \cup \mathcal{G}$ 
8:   end if
9:    $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
10:   $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$ 
11:   $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
12: end for return  $\mathbf{Y}$ 

```

D.3.3 Key Characteristics

- **Complexity:** $\mathcal{O}(n \cdot w)$ where w is the window size (linear when $w \ll n$).
- **Trainable:** Yes; drop-in replacement for standard attention.
- **Coverage:** With L layers and window size w , receptive field grows to $L \times w$, covering entire sequence with shallow networks.
- **Strength:** Practical for document-level tasks; dilation expands local receptive fields with minimal overhead; global tokens provide query-document correspondence.
- **Limitation:** Window size remains a design hyperparameter; suboptimal for tasks requiring dense attention patterns.

D.4 BigBird: Random, Local, and Global Sparse Graph

D.4.1 Mathematical Formulation

BigBird [61] preserves universal approximation and Turing completeness using a principled mix of three sparsity patterns.

Random connections : Each position connects to r randomly sampled positions:

$$\mathcal{A}_i^{(\text{random})} = \text{RandomSample}(\{1, \dots, n\}, r)$$

Local window : Sliding window neighborhood:

$$\mathcal{A}_i^{(\text{local})} = \{j : |i - j| \leq w/2\}$$

Global tokens : Task-specific global anchors:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \dots, n\} & \text{if } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{random})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{G} & \text{otherwise} \end{cases}$$

The composite sparse mask M is:

$$M_{ij} = \mathbf{1}[j \in \mathcal{A}_i^{(\text{random})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{A}_i^{(\text{global})}]$$

In practice, BigBird groups tokens into blocks of size b and applies the sparsity decision at the block level, enabling efficient block-sparse implementations.

D.4.2 Theoretical Guarantee

The authors prove that with $O(1)$ random connections and global tokens, the sparse graph maintains the same universal approximation and Turing completeness properties as dense attention. Formally, any computable function can be represented and any sequence of operations can be simulated within the BigBird connectivity graph.

D.4.3 Algorithmic Pseudocode

Algorithm 25 BigBird: Random + Local + Global Block-Sparse Attention

Require: Query $\mathbf{Q}$, Key $\mathbf{K}$, Value $\mathbf{V}$, block size b , random links per block r , global indices $\mathcal{G}$

Ensure: Output $\mathbf{Y}$

```

1: Reshape into blocks:  $n_b = \lceil n/b \rceil$  blocks
2: for block  $i = 0, \dots, n_b - 1$  do
3:   for position  $j$  in block  $i$  do
4:     Local window:  $\mathcal{A}_j^{(\text{local})} = \text{nearby positions in block } \cup \text{boundary positions}$ 
5:     Random block sampling: Sample  $r$  blocks uniformly at random, add all positions
      from those blocks
6:     Global: Add all global token positions
7:      $\mathcal{A}_j \leftarrow \mathcal{A}_j^{(\text{local})} \cup \mathcal{A}_j^{(\text{random})} \cup \mathcal{G}$ 
8:   end for
9: end for
10: for  $i = 1, \dots, n$  do
11:    $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
12:    $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$ 
13:    $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
14: end for return  $\mathbf{Y}$ 

```

D.4.4 Key Characteristics

- **Complexity:** $\mathcal{O}(n)$ or near-linear in optimal block-sparse implementation.
- **Trainable:** Yes.
- **Theoretical grounding:** Provably maintains universal approximation and Turing-completeness with sparse connectivity.
- **Strength:** Strong long-document performance on question answering, summarization, and genomics tasks.
- **Limitation:** Random sampling introduces variance and non-determinism; tuning of r, w, g hyperparameters remains task-dependent.

D.5 SparseK Attention: Differentiable Top- k Selection

D.5.1 Mathematical Formulation

SparseK [32] enables learnable sparsity via a **differentiable top- k operator**. A scoring network ϕ_θ evaluates key-value pair importance:

$$u_j = \phi_\theta(\mathbf{k}_j, \mathbf{q}_i) \in \mathbb{R}$$

The **SparseK operator** selects the top- k scores while remaining differentiable. It computes a threshold $\tau(u)$ such that the sum of active scores equals k :

$$\text{SparseK}(u, k)_j = \max(u_j - \tau(u), 0), \quad \text{where} \quad \sum_j \max(u_j - \tau, 0) = k$$

The threshold τ is found via bisection. The attention becomes:

$$m_j = \text{SparseK}(u, k)_j, \quad \text{Attn}_i = \text{softmax} \left(\frac{\mathbf{q}_i K_{\text{sel}}^\top}{\sqrt{d_k}} \right) V_{\text{sel}}$$

where $K_{\text{sel}}, V_{\text{sel}}$ contain only the top- k entries (those with $m_j > 0$).

D.5.2 Algorithmic Pseudocode

Algorithm 26 SparseK: Differentiable Top- k Attention

Require: Query $\mathbf{q} \in \mathbb{R}^d$, Key matrix $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value matrix $\mathbf{V} \in \mathbb{R}^{n \times d}$

Require: Scoring network ϕ_θ , target sparsity k

Ensure: Attention output $\mathbf{y}$

```

1: Compute importance scores:
2: for  $j = 1, \dots, n$  do
3:    $u_j \leftarrow \phi_\theta(\mathbf{k}_j, \mathbf{q})$ 
4: end for
5: Compute differentiable top- $k$  via threshold:
6:  $u_{\text{sorted}} \leftarrow \text{sort}(u, \text{descending} = \text{True})$ 
7:  $\text{cumsum} \leftarrow \text{cumsum}(u_{\text{sorted}})$ 
8: Find  $\rho = \max\{i : u_{\text{sorted}}[i] > 0 \text{ and } \text{cumsum}[i] \leq k\}$ 
9:  $\tau \leftarrow (u_{\text{sorted}}[\rho] - k) / (\rho + 1)$  ▷ Threshold for  $k$  active elements
10:  $m \leftarrow \max(u - \tau, 0)$  ▷ Differentiable selection mask
11: Apply mask and compute attention:
12:  $K_{\text{sel}} \leftarrow K[m > 0], V_{\text{sel}} \leftarrow V[m > 0]$ 
13:  $\text{scores} \leftarrow \mathbf{q} K_{\text{sel}}^\top / \sqrt{d_k}$ 
14:  $\text{attn} \leftarrow \text{softmax}(\text{scores})$ 
15:  $\mathbf{y} \leftarrow \text{attn} \cdot V_{\text{sel}}$  return  $\mathbf{y}$ 

```

D.5.3 Key Characteristics

- **Complexity:** $\mathcal{O}(n \cdot d)$ during training (linear in sequence length); $\mathcal{O}(k)$ per token during generation.
- **Trainable:** Yes; end-to-end gradient flow through the SparseK operator.
- **Incremental generation:** Supports efficient constant-memory autoregressive generation.
- **Strength:** Seamlessly integrates into existing LLM architectures; minimal fine-tuning needed.
- **Limitation:** Scattered memory access from top- k selection may limit cache efficiency on some hardware; scoring network adds overhead.

D.6 NSA (Native Sparse Attention): Hardware-Aligned Hierarchical Branches

D.6.1 Mathematical Formulation

NSA [59] decomposes sparse attention into three parallel branches that are combined through learned gates.

Compression branch : A learnable MLP φ compresses key-value blocks:

$$\tilde{K}_t^{\text{cmp}} = [\varphi(k_{id+1:id+l})]_{1 \leq i \leq \lfloor (t-l)/d \rfloor}, \quad \tilde{V}_t^{\text{cmp}} = [\varphi(v_{id+1:id+l})]$$

Selection branch : High-importance blocks are selected based on compressed scores:

$$p_t^{\text{cmp}} = \text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}} / \sqrt{d}), \quad I_t = \text{TopK}(p_t^{\text{cmp}}, n), \quad \tilde{K}_t^{\text{sel}} = \text{Gather}(K, I_t)$$

Sliding window branch : Fixed local context:

$$\tilde{K}_t^{\text{win}} = K_{t-w:t}, \quad \tilde{V}_t^{\text{win}} = V_{t-w:t}$$

Gated combination : The three attention outputs are combined via learned gates:

$$\alpha_t^{(\text{cmp})} = \sigma(\text{Linear}_{\text{cmp}}(q_t)), \quad \alpha_t^{(\text{sel})} = \sigma(\text{Linear}_{\text{sel}}(q_t)), \quad \alpha_t^{(\text{win})} = \sigma(\text{Linear}_{\text{win}}(q_t))$$

$$y_t = \alpha_t^{(\text{cmp})} \cdot \text{Attn}(q_t, \tilde{K}_t^{\text{cmp}}, \tilde{V}_t^{\text{cmp}}) + \alpha_t^{(\text{sel})} \cdot \text{Attn}(q_t, \tilde{K}_t^{\text{sel}}, \tilde{V}_t^{\text{sel}}) + \alpha_t^{(\text{win})} \cdot \text{Attn}(q_t, \tilde{K}_t^{\text{win}}, \tilde{V}_t^{\text{win}})$$

D.6.2 Algorithmic Pseudocode

Algorithm 27 NSA: Compressed + Selected + Window Branches with Learned Gating

Require: Query $\mathbf{q}_t$, cached Key/Value sequences, block size l , stride d , selection count n , window w

Require: Compression MLP φ , gate networks $\text{Linear}_{\text{cmp}}, \text{Linear}_{\text{sel}}, \text{Linear}_{\text{win}}$

Ensure: Output $\mathbf{y}_t$

```

1: Compression branch:
2: for  $i = 1$  to  $\lfloor t/d \rfloor$  do
3:    $k_i^{\text{cmp}} \leftarrow \varphi(\mathbf{K}_{id+1:id+l})$  ▷ Compress block via MLP
4:    $v_i^{\text{cmp}} \leftarrow \varphi(\mathbf{V}_{id+1:id+l})$ 
5: end for
6:  $\tilde{\mathbf{K}}^{\text{cmp}} \leftarrow [k_1^{\text{cmp}}, \dots, k_{\lfloor t/d \rfloor}^{\text{cmp}}], \tilde{\mathbf{V}}^{\text{cmp}} \leftarrow [v_1^{\text{cmp}}, \dots, v_{\lfloor t/d \rfloor}^{\text{cmp}}]$ 
7:  $\mathbf{y}_t^{(\text{cmp})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{cmp}}, \tilde{\mathbf{V}}^{\text{cmp}})$ 
8: Selection branch:
9: Compute scores on compressed:  $\mathbf{s} \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}^{\text{cmp}\top} / \sqrt{d})$ 
10: Select top- $n$  important blocks:  $I \leftarrow \text{TopK}(\mathbf{s}, n)$ 
11: Gather full blocks:  $\tilde{\mathbf{K}}^{\text{sel}} \leftarrow [\mathbf{K}_{I_d+1:I_d+l}]_i, \tilde{\mathbf{V}}^{\text{sel}} \leftarrow [\mathbf{V}_{I_d+1:I_d+l}]_i$ 
12:  $\mathbf{y}_t^{(\text{sel})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{sel}}, \tilde{\mathbf{V}}^{\text{sel}})$ 
13: Window branch:
14:  $\mathbf{y}_t^{(\text{win})} \leftarrow \text{SDPA}(\mathbf{q}_t, \mathbf{K}_{t-w:t}, \mathbf{V}_{t-w:t})$ 
15: Gated fusion:
16:  $\alpha^{(\text{cmp})} \leftarrow \sigma(\text{Linear}_{\text{cmp}}(\mathbf{q}_t)), \alpha^{(\text{sel})} \leftarrow \sigma(\text{Linear}_{\text{sel}}(\mathbf{q}_t)), \alpha^{(\text{win})} \leftarrow \sigma(\text{Linear}_{\text{win}}(\mathbf{q}_t))$ 
17:  $\mathbf{y}_t \leftarrow \alpha^{(\text{cmp})} \mathbf{y}_t^{(\text{cmp})} + \alpha^{(\text{sel})} \mathbf{y}_t^{(\text{sel})} + \alpha^{(\text{win})} \mathbf{y}_t^{(\text{win})}$  return  $\mathbf{y}_t$ 

```

D.6.3 Key Characteristics

- **Complexity:** $\mathcal{O}(t/d + n \cdot l + w)$ tokens attended per step (significantly reduced versus $\mathcal{O}(t)$ for dense).
- **Trainable:** Yes; direct training with all branches differentiable.
- **Hardware alignment:** Designed for efficient tensor core utilization; compression and selection reduce memory bandwidth pressure.
- **Strength:** $11.6\times$ decoding speedup and $9\times$ forward speedup on 64k sequences; outperforms full attention on many tasks.
- **Limitation:** Requires custom Triton/CUDA kernels; complex multi-branch architecture increases engineering overhead.

D.7 FASA: Frequency-Aware Sparse Attention

D.7.1 Mathematical Formulation

FASA [53] is a **training-free inference-time method** that exploits rotary position embedding (RoPE) structure. Under RoPE, the token embedding is rotated by $\theta_i = B^{-2(i-1)/d}$, decomposing the d -dimensional space into $d/2$ frequency chunks (FCs).

Key insight : Only a small subset ($< 1\%$) of FCs matter for contextual awareness; most encode positional patterns.

Dominant frequency chunk identification : For each layer l and head h , identify dominant FCs via contextual agreement (CA):

$$CA^{l,h,i} = \frac{|\text{TopK}(\alpha^{l,h}) \cap \text{TopK}(\alpha^{l,h,i})|}{K}$$

where $\alpha^{l,h}$ is the full attention mask and $\alpha^{l,h,i}$ is the mask using only FC i . Dominant FCs have high CA—they contribute meaningfully to the final attention pattern.

Token importance prediction (TIP) stage : Using only dominant FCs, compute lightweight importance per token:

$$S_t^{l,h} = \sum_{i \in \mathcal{I}_{\text{dom}}^{l,h}} \alpha^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}), \quad \mathcal{T}_t = \text{TopK-Indices}(S_t, N_{\text{fac}})$$

Focused attention computation (FAC) stage : Compute full-precision attention on selected tokens:

$$\hat{\alpha}_{\text{FAC}} = \text{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top}{\sqrt{d}}\right), \quad \mathbf{o}_t = \hat{\alpha}_{\text{FAC}} \mathbf{V}_{\mathcal{T}_t}$$

D.7.2 Algorithmic Pseudocode

Algorithm 28 FASA: Frequency-Aware Sparse Attention (Inference-Time)

Require: Current query $\mathbf{q}_t$, cached Key/Value $\mathbf{K}_{1:t}, \mathbf{V}_{1:t}$, RoPE base B , dominant FCs $\mathcal{I}_{\text{dom}}$

Require: TIP token budget N_{tip} , FAC token budget N_{fac}

Ensure: Output $\mathbf{o}_t$

```

1: Stage 1: Token Importance Prediction (TIP)
2: for layer  $l$  and head  $h$  do
3:   for position  $i = 1$  to  $t$  do
4:     Compute importance from dominant FCs:  $s_i \leftarrow 0$ 
5:     for  $fc \in \mathcal{I}_{\text{dom}}^{l,h}$  do
6:       Rotate query and key into FC  $fc$ :  $\mathbf{q}^{(fc)}, \mathbf{k}_i^{(fc)}$ 
7:        $s_i^{(fc)} \leftarrow \text{softmax}(\mathbf{q}^{(fc)} \mathbf{k}_i^{(fc)\top} / \sqrt{d})$ 
8:        $s_i \leftarrow s_i + s_i^{(fc)}$ 
9:     end for
10:  end for
11:   $\mathcal{T}_t \leftarrow \text{TopK} - \text{Indices}([s_1, \dots, s_t], N_{\text{fac}})$  ▷ Select top tokens
12: end for
13: Stage 2: Focused Attention Computation (FAC)
14:  $\text{scores}_{\text{fac}} \leftarrow \mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top / \sqrt{d}$  ▷ Full-precision dot product
15:  $\alpha_{\text{fac}} \leftarrow \text{softmax}(\text{scores}_{\text{fac}})$  ▷ Full softmax
16:  $\mathbf{o}_t \leftarrow \alpha_{\text{fac}} \mathbf{V}_{\mathcal{T}_t}$  ▷ Weighted value sum return  $\mathbf{o}_t$ 

```

D.7.3 Key Characteristics

- **Complexity:** TIP is $\mathcal{O}(t \cdot N_{\text{tip}})$; FAC is $\mathcal{O}(N_{\text{fac}} \cdot d)$.
- **Trainable:** No; training-free inference optimization.
- **Applicability:** Requires RoPE-based models; extended to ALiBi and MLA with modifications.
- **Strength:** Near-oracle accuracy with ≤ 256 tokens out of millions; up to $2.56\times$ decoding speedup; orthogonal to quantization.
- **Limitation:** Offline offline calibration required; dominant FC identification adds preprocessing overhead.

D.8 SpargeAttn: Two-Stage Block-Level Filtering

D.8.1 Mathematical Formulation

SparseAttn [64] is a **universal training-free method** that predicts which blocks of the attention matrix will contain negligible values and skips computation for those blocks.

Stage 1 — Sparse prediction : Compute low-cost proxy importance $\hat{s}_{ij}$ for block (i, j) :

$$\hat{s}_{ij} = f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j; \text{hyperparams}), \quad \text{skip if } \hat{s}_{ij} < \epsilon_1$$

Common predictors include block-mean similarity or self-similarity statistics.

Stage 2 — Softmax-aware filtering : After computing $\tilde{P}_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d})$, check if the block’s maximum probability is negligible relative to the running softmax maximum:

$$\text{skip } P_{ij} V_j \quad \text{if} \quad \max(\tilde{P}_{ij}) < e^{m_{\text{old}} - m_{\text{new}}} \cdot \epsilon_2$$

where $m_{\text{old}}, m_{\text{new}}$ are online softmax running maxima (computed via FlashAttention-style numerics).

D.8.2 Algorithmic Pseudocode

Algorithm 29 SparseAttn: Two-Stage Block-Sparse Filtering

Require: Query blocks $\mathbf{Q}_i$ for $i = 1, \dots, n_b$, Key blocks $\mathbf{K}_j$ for $j = 1, \dots, n_b$, Value blocks $\mathbf{V}_j$

Require: Prediction threshold ϵ_1 , softmax threshold ϵ_2 , block size b_s

Ensure: Output $\mathbf{Y}$

```

1: Initialize: online softmax max  $m_{\text{global}} \leftarrow -\infty$ 
2: for block row  $i = 1$  to  $n_b$  do
3:   Initialize: block row output  $\mathbf{Y}_i \leftarrow 0$ , local max  $m_i \leftarrow -\infty$ 
4:   for block column  $j = 1$  to  $n_b$  do
5:     Stage 1: Sparse Prediction
6:     Compute proxy importance:  $\hat{s}_{ij} \leftarrow f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j)$ 
7:     if  $\hat{s}_{ij} < \epsilon_1$  then
8:       skip this block  $[i, j]$  ▷ Early termination
9:       continue ▷ Move to next block
10:    end if
11:    Stage 2: Compute and Filter
12:     $\tilde{P}_{ij} \leftarrow \text{softmax}(\mathbf{Q}_i \mathbf{K}_j^\top / \sqrt{d})$ 
13:     $m_{\text{block}} \leftarrow \max(\tilde{P}_{ij})$ 
14:    if  $m_{\text{block}} < e^{m_i - m_{\text{global}}} \cdot \epsilon_2$  then
15:      Block contributes negligibly; skip ▷ Softmax-aware pruning
16:      continue
17:    end if
18:    Update global max:  $m_i \leftarrow \max(m_i, m_{\text{block}})$ 
19:    Accumulate:  $\mathbf{Y}_i \leftarrow \mathbf{Y}_i + \tilde{P}_{ij} \mathbf{V}_j$ 
20:  end for
21:   $m_{\text{global}} \leftarrow \max(m_{\text{global}}, m_i)$ 
22: end for return  $\mathbf{Y}$ 

```

D.8.3 Key Characteristics

- **Complexity:** Empirical $\mathcal{O}(n^2 \cdot s)$ where s is the fraction of non-skipped blocks (typically 0.2–0.5).
- **Trainable:** No; plug-and-play acceleration for existing models.
- **Universality:** Works on language models, image diffusion models, and video generation.
- **Strength:** 2.5–5× speedup compared to dense or previous sparse methods; compatible with quantization (SageAttention integration).
- **Limitation:** Speedup depends on inherent sparsity; block-level granularity may miss finer patterns; threshold tuning required per model family.

D.9 MSA (MiniMax Sparse Attention): Blockwise Sparse Attention with Lightweight Index Branch

D.9.1 Mathematical Formulation

MSA [24] is a blockwise sparse attention mechanism built on grouped-query attention (GQA) that combines a lightweight **Index Branch** for block selection with a **Main Branch** for exact block-sparse softmax attention over the selected blocks. Let N be the sequence length, $B_k = 128$ the block size, and $k = 16$ the number of blocks selected per query (yielding a fixed per-query budget of $k \cdot B_k = 2048$ tokens regardless of N).

Index Branch : A low-dimensional projection produces index queries $\mathbf{q}_t^{\text{idx}} \in \mathbb{R}^{d_{\text{idx}}}$ and block representatives $\mathbf{r}_b \in \mathbb{R}^{d_{\text{idx}}}$ for each KV block b . The index scores per GQA group are:

$$s_{t,b}^{(g)} = \mathbf{q}_t^{\text{idx},(g)} \cdot \mathbf{r}_b, \quad \mathcal{I}_t^{(g)} = \text{TopK} \left(\{s_{t,b}^{(g)}\}_{b=1}^{N/B_k}, k \right)$$

The index input is **detached** (stop-gradient), so gradients do not flow back through the indexer’s input; the Index Branch is trained instead by a KL alignment loss between its attention distribution and the Main Branch’s distribution. Because softmax is order-preserving, the raw index scores feed directly into the top- k operator *without* an explicit softmax (**exp-free top- k**), reducing numerical cost.

Forced local block : The block containing the query position t is **always** included in $\mathcal{I}_t^{(g)}$, guaranteeing local context coverage.

Main Branch : Standard scaled dot-product attention is computed exactly over the union of selected blocks:

$$\mathbf{o}_t^{(h)} = \text{softmax} \left(\frac{\mathbf{q}_t^{(h)} \mathbf{K}_{\mathcal{I}_t}^{(h)\top}}{\sqrt{d_h}} \right) \mathbf{V}_{\mathcal{I}_t}^{(h)}$$

where $\mathbf{K}_{\mathcal{I}_t}, \mathbf{V}_{\mathcal{I}_t}$ gather the full-precision key/value tensors of the selected blocks. Selection is performed independently per GQA group, so different query groups within the same layer may attend to different block subsets.

D.9.2 Algorithmic Pseudocode

Algorithm 30 MSA: MiniMax Sparse Attention with Index + Main Branches

Require: Query $\mathbf{Q} \in \mathbb{R}^{N \times d}$, Key $\mathbf{K}$, Value $\mathbf{V}$, block size $B_k = 128$, top- k blocks $k = 16$

Require: Index projection W_{idx} (detached input), block representatives $\{\mathbf{r}_b\}$

Ensure: Output $\mathbf{Y} \in \mathbb{R}^{N \times d}$

- 1: Compute block reps: $\mathbf{r}_b \leftarrow \text{mean}(\mathbf{K}_{bB_k:(b+1)B_k})$ for $b = 1, \dots, N/B_k$
 - 2: **for** query position $t = 1, \dots, N$ **do**
 - 3: **for** each GQA group g **do**
 - 4: $\mathbf{q}_t^{\text{idx}} \leftarrow \text{sg}(W_{\text{idx}} \mathbf{x}_t)$ ▷ detached index query
 - 5: Compute block scores: $s_b \leftarrow \mathbf{q}_t^{\text{idx}} \cdot \mathbf{r}_b$
 - 6: Identify local block: $b_{\text{local}} \leftarrow \lfloor t/B_k \rfloor$
 - 7: $\mathcal{I}_t^{(g)} \leftarrow \text{TopK}(\{s_b\}, k) \cup \{b_{\text{local}}\}$ ▷ exp-free top- k + forced local
 - 8: **end for**
 - 9: Gather selected KV: $\tilde{\mathbf{K}}_t \leftarrow \text{Gather}(\mathbf{K}, \mathcal{I}_t)$, $\tilde{\mathbf{V}}_t \leftarrow \text{Gather}(\mathbf{V}, \mathcal{I}_t)$
 - 10: Main branch SDPA: $\mathbf{y}_t \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}_t^\top / \sqrt{d_h}) \tilde{\mathbf{V}}_t$
 - 11: **end for**
 - 12: Add KL alignment loss $\mathcal{L}_{\text{KL}} = \text{KL}(\text{index dist} \parallel \text{main dist})$ during training **return** $\mathbf{Y}$
-

D.9.3 Key Characteristics

- **Complexity:** Index branch $\mathcal{O}(H_{kv} \cdot d_{idx} \cdot N^2)$; Main branch $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$. Per-query budget fixed at $k \cdot B_k = 2048$ tokens regardless of N .
- **Trainable:** Yes; end-to-end (the Index Branch is trained via a KL alignment loss, not eval-only).
- **State:** $\mathcal{O}(N)$ KV cache (GQA-sized); no expansion of the recurrent/state dimension.
- **Strength:** Deployed on the 109B MiniMax-M3, MSA achieves a $28.4\times$ per-token attention compute reduction at 1M context, with $14.2\times$ prefill and $7.6\times$ decode speedup on H800 GPUs.
- **Limitation:** Selection granularity is fixed at the block level ($B_k = 128$); the forced local block and per-group independent selection increase engineering complexity and require custom block-sparse kernels.

D.10 SparDA (Sparse Decoupled Attention): Forecast-Guided Lookahead Block Selection

D.10.1 Mathematical Formulation

SparDA [16] introduces a **decoupled** sparse attention scheme that augments the standard per-layer projections Q, K, V with a fourth projection, the **Forecast**, which predicts the KV blocks that will be needed by the *next* layer. This lookahead enables overlapping CPU-to-GPU prefetch of the next layer’s selected blocks with the current layer’s execution, hiding the offload latency that plagues sparse-attention deployments on memory-constrained hardware.

Forecast projection : For input $\mathbf{x} \in \mathbb{R}^{N \times d}$,

$$\mathbf{F} = \mathbf{x}\mathbf{W}_F \in \mathbb{R}^{N \times H_{kv} \times d_f}$$

with **one Forecast head per GQA group**, which reduces selection overhead relative to multi-head selectors used by prior work.

Block scoring and selection : For each query i and KV block b with representative $\mathbf{r}_b$:

$$S_{i,b} = \mathbf{F}_i \cdot \mathbf{r}_b, \quad \mathcal{I}_i = \text{TopK}(\{S_{i,b}\}_b, k)$$

Block-sparse attention : Standard scaled dot-product attention is then computed over the selected blocks:

$$\mathbf{o}_i = \text{softmax} \left(\frac{\mathbf{q}_i \mathbf{K}_{\mathcal{I}_i}^\top}{\sqrt{d_h}} \right) \mathbf{V}_{\mathcal{I}_i}$$

Training : SparDA adds fewer than 0.5% additional parameters. Only the Forecast projections are trained, by matching the original (frozen) sparse selector’s attention distribution—a lightweight distillation that preserves the base model’s behaviour while enabling lookahead prefetch.

D.10.2 Algorithmic Pseudocode

Algorithm 31 SparDA: Sparse Decoupled Attention with Forecast Prefetch

Require: Input $\mathbf{x} \in \mathbb{R}^{N \times d}$, cached Key/Value blocks, top- k blocks k

Require: Projections W_q, W_k, W_v, W_F ; frozen base selector π_{orig}

Ensure: Output $\mathbf{Y}$; prefetched blocks for next layer

- 1: Compute Forecast: $\mathbf{F} \leftarrow \mathbf{x} \mathbf{W}_F$ $\triangleright$ one head per GQA group
 - 2: **for** query $i = 1, \dots, N$ **do**
 - 3: Compute block scores: $S_{i,b} \leftarrow \mathbf{F}_i \cdot \mathbf{r}_b$ for each block b
 - 4: Select: $\mathcal{I}_i \leftarrow \text{TopK}(\{S_{i,b}\}, k)$
 - 5: **Prefetch** $\mathbf{K}_{\mathcal{I}_i}, \mathbf{V}_{\mathcal{I}_i}$ from CPU to GPU for the *next* layer $\triangleright$ overlaps with current layer
 - 6: **end for**
 - 7: Compute block-sparse SDPA over current-layer selected blocks: $\mathbf{Y} \leftarrow \text{BlockSparseSDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \{\mathcal{I}_i\})$
 - 8: Training loss: $\mathcal{L} \leftarrow \text{KL}(\text{Forecast dist} \parallel \pi_{\text{orig}})$ $\triangleright$ only W_F is updated **return** $\mathbf{Y}$
-

D.10.3 Key Characteristics

- **Complexity:** Forecast $\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_{\text{blocks}})$; Main attention $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$.
- **Trainable:** Yes; only the Forecast projections are trained ($< 0.5\%$ extra parameters).
- **Strength:** On two 8B sparse-pretrained models, SparDA achieves up to $1.25\times$ prefill and $1.7\times$ decode speedup over the sparse-attention offload baseline, and up to $5.3\times$ decode throughput versus a non-offload sparse baseline.
- **Limitation:** Requires a CPU–GPU offload pipeline and a frozen base selector; the lookahead benefit is contingent on next-layer block demand being predictable from the current Forecast.

D.11 Comparative Summary

Method	Complexity	Trainable	Unit	Year	Primary Contribution
Sparse Transformer	$O(n\sqrt{nd})$	Y	Token pattern	2019	Factorized strided + fixed masks
Longformer	$O(nwd)$	Y	Local + global	2020	Linear scaling with dilation
BigBird	$O(n)$	Y	Graph sparse	2020	Theoretical completeness proof
SparseK	$O(nd)$	Y	Top- k tokens	2024	Differentiable selection
NSA	$O((t/d + nl' + w)d)$	Y	Multi-branch	2025	Hardware-aligned 3-branch design
FASA	$O(tN_{\text{tip}} + N_{\text{fac}}d)$	N	Freq chunks	2026	RoPE frequency insight
SpargeAttn	$O(n^2 s \cdot d)$	N	Block filter	2025	Two-stage universal filtering
MSA	$O(H_{\text{kv}} d_{\text{idx}} N^2 + Y H_q d_h N k B_k)$		Block top- k	2026	Index-branch block sparse (deployed 109B)
SparDA	$O(H_{\text{kv}} d_f N N_b + Y H_q d_h N k B_k)$		Forecast block	2026	Lookahead prefetch decoupled selection

D.12 Design Space and Selection Criteria

The seven sparse attention methods occupy complementary positions in a multidimensional design space:

- **Geometric/fixed patterns** (Sparse Transformer, Longformer): Simple to implement and analyze, but patterns do not adapt to content.
- **Learned selection** (SparseK, NSA): Enable adaptation through trainable selection networks, at the cost of higher implementation complexity.
- **Training-free acceleration** (FASA, SpargeAttn): Enable drop-in speedup of existing models without retraining, suited for deployed systems.
- **Theoretical guarantees** (BigBird): Provide formal proofs of expressive completeness, important for understanding safety margins.

Selection guidance:

1. **New model training** where resources permit: NSA or SparseK for maximal inference speedup; BigBird for theoretical assurance.
2. **Long-context document understanding**: Longformer or FASA for practical simplicity; NSA for extreme scale.
3. **Accelerating existing models**: FASA for RoPE-based LLMs; SpargeAttn for any architecture.
4. **Constrained environments**: Sparse Transformer for simplicity and proven efficiency at moderate scale.

E Annex D: Gated Attention Families—Complete Literature Analysis

E.1 Executive Summary: Gating for Memory Control

The emerging field of *gated attention mechanisms* addresses a fundamental challenge in sequence modeling: how should information be retained, forgotten, and updated as the model processes new tokens? Traditional additive recurrent states accumulate all information without erasure, leading to memory saturation and inability to adapt to changing context. Softmax attention provides full expressiveness but uses $\mathcal{O}(Ld)$ KV cache during inference, limiting context window size. Gated mechanisms provide a middle ground: selective control over information survival.

The unifying principle across gated architectures is that gating controls *what information survives*. Gates operate at three distinct levels:

1. **Recurrent state decay** (GLA, HGRN2): Multiplicative gates on the memory matrix S_t control how much previous state persists into the next step.
2. **Write strength in recurrent updates** (DeltaNet, Gated DeltaNet, Gated DeltaNet-2): Gates control the magnitude and channel-wise direction of new information written into memory.
3. **Softmax logit biasing** (FoX): Gates inject token-level recency bias into attention logit computation.
4. **Post-attention sparsity** (Gated Softmax): Gates selectively scale SDPA output channels.

This appendix synthesizes eight major gated-attention architectures published 2023–2026, analyzing their mathematical foundations, hardware efficiency characteristics, and empirical trade-offs.

E.2 1. Gated Linear Attention (GLA)

E.2.1 Mathematical Foundation

Gated Linear Attention [55] augments vanilla linear attention (which uses a matrix-valued recurrent state) with data-dependent multiplicative decay. Standard linear attention reformulates the traditional softmax mechanism as a recurrence over hidden states:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

where state $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ accumulates outer products. This purely additive formulation suffers from “memory overload”: information accumulates monotonically and cannot be erased, degrading performance on tasks requiring context selection.

GLA introduces a data-dependent **diagonal gating matrix** $\mathbf{G}_t \in [0, 1]^{d_k \times d_k}$:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

The gate $\mathbf{G}_t$ is parameterized with an outer-product structure:

$$\mathbf{G}_t = \mathbf{1} \alpha_t^\top, \quad \alpha_t = \text{logsigmoid}(\mathbf{W}_{gk} \mathbf{x}_t) / c$$

where $\mathbf{W}_{gk}$ is a low-rank projection ($\mathbb{R}^{d \rightarrow d_k}$) and $c \approx 16$ is a normalizer. The logsigmoid activation maps $\alpha_t \in \mathbb{R}$ to approximately $[-1/2, 0]$, and division by c further contracts this to near-identity for initialization. The resulting gate $G_t = 1 + \alpha_t \approx 1$ near initialization, then learns to decay ($\alpha_t \rightarrow -\infty$) historical information.

Algorithm 32 Gated Linear Attention (Recurrent Form)

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1}$

Require: Projections: W_q, W_k, W_v (standard); W_{gk} (gate projection)

Ensure: Output $\mathbf{o}_t$ and updated state $\mathbf{S}_t$

- 1: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$
 - 2: $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$
 - 3: $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
 - 4: Compute gate: $\alpha_t \leftarrow \text{logsigmoid}(W_{gk} \mathbf{x}_t) / 16$ ▷ Per-key-dim decay
 - 5: Apply outer-product gating: $\mathbf{G}_t \leftarrow \mathbf{1} \alpha_t^\top$ ▷ diagonal outer product
 - 6: Decay previous state: $\mathbf{S}_t^{\text{decay}} \leftarrow \mathbf{G}_t \odot \mathbf{S}_{t-1}$ ▷ element-wise product
 - 7: Add new association: $\mathbf{S}_t \leftarrow \mathbf{S}_t^{\text{decay}} + \mathbf{v}_t \mathbf{k}_t^\top$
 - 8: Retrieve via query: $\mathbf{o}_t^{\text{raw}} \leftarrow \mathbf{S}_t \mathbf{q}_t$
 - 9: Apply output gate: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t^{\text{raw}}) \otimes \text{SiLU}(W_g \mathbf{x}_t)$ **return** $\mathbf{o}_t, \mathbf{S}_t$
-

E.2.2 Hardware-Efficient Training

For parallel training, GLA uses a chunkwise block-wise algorithm that groups tokens into chunks C and computes recurrent transitions in parallel:

$$\mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{\Gamma}_{[t]} \right) \mathbf{V}_{[t]}$$

where $\overleftarrow{\mathbf{Q}}_{[t]}$ are attending to the state at the chunk boundary (lookback), and $\mathbf{\Gamma}_{[t]}$ is the causal mask with decay-aware scaling:

$$\mathbf{\Gamma}_{[t],ij} = \begin{cases} \prod_{l=j}^{i-1} \alpha_l & \text{if } i > j \text{ (lookback)} \\ \prod_{l=1}^{i-j} \alpha_l & \text{if } i \leq j \text{ (in-chunk)} \end{cases}$$

This design maximizes matmul operations susceptible to tensor-core acceleration, achieving sub-quadratic total complexity $\mathcal{O}(nLd^2/C)$ where L is the number of attention heads and C is chunk size.

E.2.3 Strengths and Limitations

Aspect	Strengths	Limitations
Computational efficiency	Sub-quadratic training via chunkwise parallelism. Constant-memory inference $\mathcal{O}(d^2)$.	Requires custom CUDA/Triton kernels; not available in all frameworks.
Context generalization	Extends 2K-token training to 20K+ tokens with negligible perplexity degradation.	Still underperforms softmax on retrieval-heavy benchmarks (needle-in-haystack).
Selectivity	Per-key-dimension data-dependent decay.	Gate structure limits per-key-value selectivity; no direct control over which specific associations to erase.

E.3 2. DeltaNet: Error-Correcting Linear Attention

E.3.1 Mathematical Foundation

DeltaNet [56] applies a classical **delta learning rule** to linear attention state updates. Instead of additive accumulation, DeltaNet computes the difference between the predicted value (retrieved from memory) and the target value, then performs an error-correcting update:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

where $\beta_t \in (0, 1)$ is a data-dependent **writing strength** scalar. This update can be decomposed as an erase-write operation:

$$\mathbf{v}_t^{\text{old}} = \mathbf{S}_{t-1} \mathbf{k}_t \quad (\text{current prediction}), \quad \mathbf{v}_t^{\text{updated}} = \beta_t \mathbf{v}_t + (1 - \beta_t) \mathbf{v}_t^{\text{old}} \quad (\text{blend old/new})$$

so that:

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{old}} \mathbf{k}_t^\top + \mathbf{v}_t^{\text{updated}} \mathbf{k}_t^\top$$

The key insight is that this update rule minimizes an online MSE loss at each timestep:

$$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2} \|\mathbf{S} \mathbf{k}_t - \mathbf{v}_t\|^2 \Rightarrow \nabla_{\mathbf{S}} \mathcal{L}_t = (\mathbf{S} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top$$

One step of SGD with learning rate β_t yields the delta rule update. This connection to test-time training (TTT) provides theoretical grounding: DeltaNet is directly optimizing for value prediction at inference time.

E.3.2 Efficient Parallel Training

DeltaNet’s transition matrix $\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top$ is a generalized Householder matrix (rank-1 update to identity). For chunkwise training, DeltaNet uses the **WY representation**, which represents the product of m such rank-1 updates compactly:

$$\prod_{i=1}^m (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) = \mathbf{I} - \mathbf{W} \mathbf{Y}^\top$$

where $\mathbf{W}, \mathbf{Y} \in \mathbb{R}^{d \times m}$ are constructed recursively. This factorization enables matmul-rich parallelism without materializing the full product, keeping chunkwise training efficient.

Algorithm 33 DeltaNet State Update with WY Representation

Require: Input chunk $\mathbf{X}_{[t]}$; prior state $\mathbf{S}_{t-1}$

Require: L2-normalized queries $\hat{\mathbf{Q}}_t$, keys $\hat{\mathbf{K}}_t$, values $\mathbf{V}_t$

Ensure: Output $\mathbf{O}_{[t]}$ and updated state $\mathbf{S}_t$

1: **Chunkwise phase:**

2: **for** position $i = 1$ to `chunk_size` **do**

3: Normalize: $\hat{\mathbf{k}}_i \leftarrow \hat{\mathbf{K}}_t[i] / \|\hat{\mathbf{K}}_t[i]\|$, $\hat{\mathbf{q}}_i \leftarrow \hat{\mathbf{Q}}_t[i] / \|\hat{\mathbf{Q}}_t[i]\|$

4: Compute write strength: $\beta_i \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_i)$

5: Prediction: $\hat{\mathbf{v}}^{\text{pred}} \leftarrow \mathbf{S}_{i-1}^{\text{in-chunk}} \hat{\mathbf{k}}_i$

6: Error-aware blend: $\mathbf{v}_i^{\text{update}} \leftarrow \beta_i(\mathbf{v}_i - \hat{\mathbf{v}}^{\text{pred}})$

7: Erase-write: $\mathbf{S}_i^{\text{in-chunk}} \leftarrow \mathbf{S}_{i-1}^{\text{in-chunk}} - \hat{\mathbf{v}}^{\text{pred}} \hat{\mathbf{k}}_i^\top + (\beta_i \mathbf{v}_i + (1 - \beta_i) \hat{\mathbf{v}}^{\text{pred}}) \hat{\mathbf{k}}_i^\top$

8: Output: $\mathbf{o}_i \leftarrow \mathbf{S}_i^{\text{in-chunk}} \hat{\mathbf{q}}_i$

9: **end for**

10: **Inter-chunk phase:** Use WY representation of transition products **return** $\mathbf{O}_{[t]}$, $\mathbf{S}_t$

E.3.3 Strengths and Limitations

Aspect	Strengths	Limitations
Associative recall	Perfect recall on Multi-Query Associative Recall (MQAR) benchmark; error-correcting semantics naturally fit retrieval patterns.	Without global forgetting, memory still crowds over extreme sequence lengths; requires periodic reset mechanisms for unbounded context.
Theory	Grounded in online MSE optimization; clear connection to test-time training paradigm.	Requires L2-normalized keys for numerical stability; double-width normalization adds overhead.
Throughput	WY representation enables efficient chunkwise training.	Slightly slower than Mamba2 per-token due to richer transition matrices (rank-1 instead of diagonal).

E.4 3. Gated DeltaNet: Synthesis of Gating and Error Correction

E.4.1 Mathematical Foundation

Gated DeltaNet [57] synthesizes the strengths of GLA and DeltaNet by combining a **decay gate** α_t (from GLA) with the **writing strength gate** β_t (from DeltaNet):

$$\mathbf{S}_t = \mathbf{S}_{t-1} \left(\alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \right) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

Rewriting for clarity:

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

The two gates are complementary:

- $\alpha_t \rightarrow 0$ rapidly erases historical state: $\mathbf{S}_t \approx \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (context reset).

- $\alpha_t \rightarrow 1$ recovers pure delta-rule behavior: $\mathbf{S}_t \approx \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (targeted updates).
- $\beta_t \rightarrow 0$ skips writing but decays: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1}$ (pure forgetting).
- $\beta_t \rightarrow 1$ replaces memory aggressively: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ (replacement).

From an online learning perspective, Gated DeltaNet corresponds to:

$$\min_{\mathbf{S}_t} \|\mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\|_F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \beta_t (\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1} \mathbf{k}_t) \rangle$$

which introduces an adaptive weight decay α_t into an SGD-like update—analogous to decoupled weight decay in deep learning optimization.

Algorithm 34 Gated DeltaNet Parallel Chunkwise Forward

Require: Input chunk $\mathbf{X}_{[i]}$; prior state $\mathbf{S}_{i-1}$; chunk size C

Require: Projections: $W_q, W_k, W_v, W_\alpha, W_\beta$

Ensure: Output $\mathbf{O}_{[i]}$ and state $\mathbf{S}_i$

1: **In-chunk computation:**

2: **for** $j = 1$ to C **do**

3: $\mathbf{q}_j \leftarrow W_q \mathbf{x}_j, \mathbf{k}_j \leftarrow W_k \mathbf{x}_j, \mathbf{v}_j \leftarrow W_v \mathbf{x}_j$

4: $\alpha_j \leftarrow \text{sigmoid}(W_\alpha \mathbf{x}_j), \beta_j \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_j)$

5: Apply combined gate: $\mathbf{S}_j \leftarrow \alpha_j \mathbf{S}_{j-1}(\mathbf{I} - \beta_j \mathbf{k}_j \mathbf{k}_j^\top) + \beta_j \mathbf{v}_j \mathbf{k}_j^\top$

6: $\mathbf{o}_j \leftarrow \mathbf{S}_j \mathbf{q}_j$

7: **end for**

8: **Inter-chunk recurrence:**

9: $\mathbf{S}_i \leftarrow \mathbf{S}_C$ from in-chunk; propagate across chunks via cumulative $\prod \alpha$ masks

10: **Output gating:** $\mathbf{O}_{[i]} \leftarrow \text{GroupNorm}(\mathbf{O}^{\text{raw}}) \otimes \text{SiLU}(W_g \mathbf{X}_{[i]})$ **return** $\mathbf{O}_{[i]}, \mathbf{S}_i$

E.4.2 Empirical Performance and Trade-offs

Gated DeltaNet has been integrated into Alibaba’s Qwen3-Next and Qwen3.5 production models. Empirically:

Task	Gated DeltaNet	Mamba2	DeltaNet	GLA
Language Modeling	1.0 $\times$	1.08 $\times$	1.05 $\times$	1.12 $\times$
Associative Recall	1.0 $\times$	— — (<i>noperfect</i>)	1.0 $\times$	0.75
Length Extrapolation	1.0 $\times$	0.98 $\times$	0.96 $\times$	0.94 $\times$
Throughput (tokens/sec)	0.85 $\times$	1.0 $\times$	0.82 $\times$	0.88 $\times$

Gated DeltaNet achieves the best balance across diverse tasks at the cost of slightly reduced throughput due to richer transition matrices.

E.5 4. HGRN2: Hierarchical Gating with Outer-Product Expansion

E.5.1 Mathematical Foundation

HGRN2 [39] uses an outer-product-based state expansion with **hierarchically lower-bounded forget gates**. The state update is:

$$\mathbf{S}_t = \text{diag}(\mathbf{g}_t) \cdot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

where $\mathbf{g}_t \in [b_\ell, 1]^d$ is a lower-bounded forget gate for layer ℓ . The bounds satisfy:

$$0 \leq b_{\ell_{\text{shallow}}} < b_{\ell_{\text{middle}}} < b_{\ell_{\text{deep}}} \leq 1$$

i.e., bounds increase (become less restrictive) in the deeper layers. The gate itself is computed as:

$$\mathbf{g}_t = b_\ell + (1 - b_\ell) \cdot \sigma(W_g \mathbf{x}_t)$$

where σ is sigmoid. When W_g outputs weakly negative logits, $\mathbf{g}_t \approx b_\ell$ (forced retention at shallow layers). When outputs are strongly positive, $\mathbf{g}_t \approx 1$ (memory refresh at any layer).

The hierarchical structure encourages different layers to specialize to different timescales: shallow layers model local dependencies (high retention due to binding b_ℓ), while deeper layers capture long-range structure (flexible gating with high upper bound).

Algorithm 35 HGRN2 Forward with Hierarchical Bounds

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1}$; layer index $\ell \in [0, L - 1]$

Require: Non-decreasing bounds: $0 \leq b_0 < b_1 < \dots < b_{L-1} \leq 1$

Ensure: Output $\mathbf{o}_t$ and state $\mathbf{S}_t$

- 1: Layer-specific retain bound: $b \leftarrow b_\ell$
 - 2: Project: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
 - 3: Compute forget gate with binding: $\mathbf{g}_t \leftarrow b + (1 - b) \cdot \sigma(W_g \mathbf{x}_t)$ $\triangleright$ Constrained to $[b, 1]$
 - 4: Diagonal multiplication (vectorized): $\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \circ \text{diag}(\mathbf{g}_t) + \mathbf{v}_t \mathbf{k}_t^\top$
 - 5: Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t \mathbf{q}_t$
 - 6: Optional normalization: $\mathbf{o}_t \leftarrow \text{RMSNorm}(\mathbf{o}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$
-

E.5.2 Scaling and Empirical Results

At 3B scale on 100B tokens, HGRN2 slightly outperforms Mamba2 and LLaMA-architecture Transformers on language modeling. The hierarchical gating provides multi-scale temporal modeling without explicit layer-wise architectural variants. However, vector-valued diagonal gating (as opposed to element-wise selection) provides less per-item flexibility than delta-rule variants.

E.6 5. Forgetting Transformer (FoX): Gating in Softmax Logit Space

E.6.1 Mathematical Foundation

Forgetting Transformer (FoX), proposed by Lin et al. [27], embeds a forget gate directly into softmax attention logits. Rather than replacing softmax with linear recurrence, FoX preserves full softmax expressiveness while adding recency control through a data-dependent logit bias.

For each token position t and all keys $j \leq t$, compute a scalar forget gate:

$$f_t = \sigma(w_f^\top \mathbf{x}_t + b_f)$$

The logit bias at position (i, j) is the cumulative log-forget product:

$$d_{ij} = \sum_{l=j+1}^i \log f_l = \log \prod_{l=j+1}^i f_l$$

The full attention output becomes:

$$\mathbf{O} = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} + \mathbf{D} \right) \mathbf{V}$$

where $\mathbf{D}_{ij} = d_{ij}$. Equivalently:

$$\text{Attention}(i, j) = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k} + d_{ij})}{\sum_{j'=1}^i \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k} + d_{ij'})}$$

This weighting down-weights older tokens multiplicatively: a token from 10 steps ago is scaled by $\prod_{l=j+1}^i f_l$, which is typically much less than 1 if $f_t < 1$ on average.

The mechanism is mathematically equivalent to a **data-dependent learnable variant of ALiBi (Attention with Linear Biases)**, where earlier work used fixed $d_{ij} = -\infty \cdot |i - j|$ or $d_{ij} = -(i - j)$.

Algorithm 36 FoX: Forgetting Attention with Recency Bias

Require: Queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V}$; input $\mathbf{X}$

Require: Forget gate parameters: $w_f \in \mathbb{R}^d$, $b_f \in \mathbb{R}$

Ensure: Output attention $\mathbf{O}$

1: **Compute forget gates:**

2: **for** $t = 1$ to T **do**

3: $f_t \leftarrow \sigma(w_f^\top \mathbf{x}_t + b_f)$

▷ Per-token forget probability

4: **end for**

5: **Compute cumulative log-forget biases:**

6: **for** $i = 1$ to T **do**

7: **for** $j = 1$ to i **do**

8: $d_{ij} \leftarrow \sum_{l=j+1}^i \log f_l$

▷ Cumulative product in log space

9: **end for**

10: **end for**

11: **Standard SDPA with bias:**

12: Compute scores: $\mathbf{S} \leftarrow \mathbf{QK}^\top / \sqrt{d_k}$

13: Add bias: $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{D}$

14: Apply softmax: $\mathbf{A} \leftarrow \text{softmax}(\mathbf{S}, \text{dim} = -1)$

15: Weight values: $\mathbf{O} \leftarrow \mathbf{AV}$ **return** $\mathbf{O}$

E.6.2 Integration with FlashAttention

The key advantage of FoX is compatibility with FlashAttention and existing optimized implementations. The forget biases are added in the softmax logit computation, which is already a core operation in FlashAttention’s block-wise algorithm. The overhead is:

$$\text{Overhead} \approx 0.5 - 2\% \text{ wall-clock time}$$

since the bias addition is amortized across matmul operations.

E.6.3 Strengths and Limitations

FoX achieves state-of-the-art results on length extrapolation, near-perfect needle-in-haystack retrieval, and superior long-context understanding compared to Transformers. However, it remains $\mathcal{O}(L^2d)$ at training and $\mathcal{O}(Ld)$ per step at inference with KV cache, limiting extreme sequence lengths.

E.7 6. Gated Attention (Post-SDPA Sigmoid Gating)

E.7.1 Mathematical Foundation

The NeurIPS 2025 Best Paper by the Qwen team proposes applying a sigmoid gate **after** scaled dot-product attention (SDPA):

$$\mathbf{Y} = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

$$\mathbf{Y}' = \mathbf{Y} \odot \sigma(\mathbf{XW}_\theta)$$

where the gate score $\sigma(\mathbf{XW}_\theta)$ can have shape $\mathbb{R}^{n \times q \times d_k}$ (element-wise gating per query head) or $\mathbb{R}^{n \times q}$ (head-wise fixed gating). Across 30+ variants and 15B MoE + 1.7B dense models trained on 3.5T tokens, the team found:

- Headwise gating adds only $\sim 1.6M$ parameters to a 15B model (negligible overhead).
- Effective gates are **sparse**: mean activation ≈ 0.116 (i.e., 88% are zero or near-zero).
- Gates act as query-dependent filters, suppressing low-value channels while preserving high-value signal.
- Gates demonstrably eliminate the **attention sink** phenomenon, where the attention pattern becomes dominated by a single early token.

Algorithm 37 Gated Softmax Attention (Post-SDPA)

Require: Queries $\mathbf{Q}$, Keys $\mathbf{K}$, Values $\mathbf{V}$; input $\mathbf{X}$

Require: Gate projection matrix $\mathbf{W}_g \in \mathbb{R}^{d \rightarrow d_{\text{gate}}}$ where $d_{\text{gate}} \in \{1, d_k\}$

Ensure: Gated output $\mathbf{Y}'$

- 1: **Standard SDPA:**
 - 2: Compute attention: $\mathbf{A} \leftarrow \text{softmax}(\mathbf{QK}^\top / \sqrt{d_k})$
 - 3: Weight values: $\mathbf{Y} \leftarrow \mathbf{AV}$
 - 4: **Compute gates:**
 - 5: Project input: $\mathbf{G} \leftarrow \mathbf{XW}_g$ $\triangleright$ shape: $[n, q, d_{\text{gate}}]$
 - 6: Apply sigmoid: $\mathbf{G} \leftarrow \sigma(\mathbf{G})$ $\triangleright$ Element-wise gating
 - 7: **Apply gating:**
 - 8: **if** $d_{\text{gate}} = 1$ **then**
 - 9: Broadcast and scale: $\mathbf{Y}' \leftarrow \mathbf{Y} \cdot \mathbf{G}$ $\triangleright$ Head-wise scaling
 - 10: **else** $\triangleright d_{\text{gate}} = d_k$
 - 11: Element-wise modulation: $\mathbf{Y}' \leftarrow \mathbf{Y} \odot \mathbf{G}$ $\triangleright$ Per-dimension gating
 - 12: **end if** **return** $\mathbf{Y}'$
-

E.7.2 Key Findings

- **Attention-sink suppression:** Gating breaks the feedback loop where softmax concentrates on the first token, enabling more balanced attention.
- **Training stability:** Models with gating tolerate larger learning rates (+30% higher stable η_{max}).
- **Minimal overhead:** Latency impact is only 1.6% on H100 GPUs; throughput drops at most 2 – 3%.
- **Scaling law improvement:** Improves loss across model scales from 800M to 110B parameters consistently.

E.8 7. Gated DeltaNet-2: Decoupled Channel-wise Erase and Write

E.8.1 Mathematical Foundation

Gated DeltaNet-2 [20] addresses a shared limitation of Gated DeltaNet and Kimi Delta Attention (KDA): both use a single *scalar* delta gate β_t to simultaneously control (i) how much old content is erased along the key-side read direction and (ii) how much new value is written along the value axis. Gated DeltaNet-2 **decouples** these two roles into a channel-wise erase gate $\mathbf{b}_t$ (key axis) and a channel-wise write gate $\mathbf{w}_t$ (value axis), while inheriting the channel-wise decay α_t from

KDA. With state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$, decay $\mathbf{D}_t = \text{Diag}(\boldsymbol{\alpha}_t)$, erase direction $\mathbf{e}_t = \mathbf{b}_t \odot \mathbf{k}_t$, and write target $\mathbf{z}_t = \mathbf{w}_t \odot \mathbf{v}_t$, the state update is:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t \mathbf{e}_t^\top \right) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{z}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

or, expanding the gate products:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t (\mathbf{b}_t \odot \mathbf{k}_t)^\top \right) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t (\mathbf{w}_t \odot \mathbf{v}_t)^\top$$

The two gates are produced by independent linear projections followed by sigmoid:

$$\mathbf{b}_t = \sigma(\mathbf{W}_b \mathbf{x}_t), \quad \mathbf{w}_t = \sigma(\mathbf{W}_w \mathbf{x}_t)$$

and the channel-wise decay follows a log-decay parameterization (computed in fp32 to avoid precision loss in long cumulative products):

$$\mathbf{g}_t = \mathbf{a} - \text{softplus}(\boldsymbol{\delta}), \quad \boldsymbol{\alpha}_t = \exp(\mathbf{g}_t)$$

The structural roles of the two gates are complementary:

- $\mathbf{b}_t$ (key axis) makes the **read direction** channel-selective: it determines which key channels are removed before writing.
- $\mathbf{w}_t$ (value axis) makes the **write target** channel-selective: it determines which value channels are deposited into memory.
- $\boldsymbol{\alpha}_t$ (per key channel) provides global decay, as in KDA.

Reductions. Setting $\mathbf{b}_t = \beta_t \mathbf{1}_{d_k}$ and $\mathbf{w}_t = \beta_t \mathbf{1}_{d_v}$ recovers KDA; further collapsing $\boldsymbol{\alpha}_t$ to a scalar recovers Gated DeltaNet. From the fast-weight / online-learning perspective, the update is the minimizer of:

$$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2} \|\mathbf{S} - \mathbf{D}_t \mathbf{S}_{t-1}\|_F^2 - 2 \left\langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{z}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top \mathbf{e}_t \right\rangle$$

Algorithm 38 Gated DeltaNet-2 Recurrent Forward (Reference Form)

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$

Require: Projections: W_q, W_k, W_v, W_b, W_w ; log-decay parameters $\mathbf{a}, \boldsymbol{\delta}$

Ensure: Output $\mathbf{o}_t$ and state $\mathbf{S}_t$

- 1: $\mathbf{q}_t \leftarrow \text{normalize}(W_q \mathbf{x}_t)$, $\mathbf{k}_t \leftarrow \text{normalize}(W_k \mathbf{x}_t)$, $\mathbf{v}_t \leftarrow \text{silu}(W_v \mathbf{x}_t)$
 - 2: $\mathbf{b}_t \leftarrow \sigma(W_b \mathbf{x}_t)$ $\triangleright$ key-axis erase gate, $\mathbb{R}^{d_k}$
 - 3: $\mathbf{w}_t \leftarrow \sigma(W_w \mathbf{x}_t)$ $\triangleright$ value-axis write gate, $\mathbb{R}^{d_v}$
 - 4: $\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\boldsymbol{\delta}))$ $\triangleright$ channel-wise decay, fp32
 - 5: Erase direction: $\mathbf{e}_t \leftarrow \mathbf{b}_t \odot \mathbf{k}_t$; write target: $\mathbf{z}_t \leftarrow \mathbf{w}_t \odot \mathbf{v}_t$
 - 6: Decay state: $\mathbf{S}_t \leftarrow \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1}$
 - 7: Read: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{e}_t$ $\triangleright$ sum over key axis
 - 8: Delta-rule update: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \mathbf{k}_t \mathbf{r}_t^\top + \mathbf{k}_t \mathbf{z}_t^\top$
 - 9: Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
 - 10: Output gating: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g \mathbf{x}_t)$
 - 11: **return** $\mathbf{o}_t, \mathbf{S}_t$
-

The chunkwise training algorithm extends KDA’s WY representation by absorbing the channel-wise decay into an asymmetric delta recurrence on a decay-normalized state, enabling matmul-rich parallelism on tensor cores. Because $\mathbf{b}_t$ and $\mathbf{w}_t$ are per-channel diagonal operators varying per row, the scalar post-scaling shortcut of KDA breaks: the backward pass requires a **gate-aware WY** inversion that bakes the gates into each dot product accumulating the gradient.

E.8.2 Empirical Performance and Trade-offs

At 1.3B parameters trained on 100B FineWeb-Edu tokens, Gated DeltaNet-2 outperforms Mamba-2, Mamba-3, Gated DeltaNet, and KDA on language modeling, commonsense reasoning, and—most markedly—real-world and RULER long-context retrieval (especially multi-key needle-in-a-haystack). Its throughput on H100 stays near-flat ($38.0 \rightarrow 36.1$ Kt/s) as sequence length grows from 2K to 16K, where a Transformer’s drops sharply. Ablations show the erase gate $\mathbf{b}_t$ accounts for most of the gain; the write gate $\mathbf{w}_t$ contributes a smaller additional improvement. The hybrid variant (recurrent Gated DeltaNet-2 + sliding-window attention) further closes the gap with pure-attention models on tasks requiring local evidence aggregation.

Aspect	Strength	Limitation
Recall	Best-in-class among recurrent delta-rule models on multi-key NIAH.	Recurrent variant still gaps Transformer on NQ/DROP.
Throughput	Near-flat scaling 2K→16K; small overhead vs KDA.	Requires a new gate-aware WY Triton backward kernel.
Memory	Constant $\mathcal{O}(d^2)$ inference state.	Neg-eigenvalue erase range $[0, 2]$ gives no consistent gain at 1.3B scale.

Table 10: Gated DeltaNet-2 trade-offs at 1.3B / 100B-token scale [20].

E.9 8. Kimi Delta Attention (KDA): Channel-wise Decay Gating for the Delta Rule

E.9.1 Mathematical Foundation

Kimi Delta Attention [22] is the core module of Kimi Linear and extends Gated DeltaNet with a finer-grained **channel-wise decay gate**. Where Gated DeltaNet uses a single scalar per-head decay α_t alongside the scalar delta write gate β_t , KDA replaces the scalar decay with a **per-key-channel** decay $\boldsymbol{\alpha}_t \in \mathbb{R}^{d_k}$, parameterised through a log-decay formulation and applied as a diagonal matrix $\mathbf{D}_t = \text{diag}(\boldsymbol{\alpha}_t)$ before the delta-rule update. The delta rule itself retains a scalar per-head write gate β_t :

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

The channel-wise decay follows a log-decay parameterisation (computed in fp32 for numerical stability over long cumulative products):

$$\mathbf{g} = \mathbf{a} - \text{softplus}(\boldsymbol{\delta}), \quad \boldsymbol{\alpha}_t = \exp(\mathbf{g})$$

where $\mathbf{a}$ and $\boldsymbol{\delta}$ are learned per-channel parameters. This gives each key channel an independent forgetting timescale, allowing the model to retain some channels longer than others within the same head.

Reduction to Gated DeltaNet : When $\boldsymbol{\alpha}_t$ collapses to a scalar (i.e., all channels share the same decay), KDA reduces exactly to Gated DeltaNet. The channel-wise decay is thus a strict generalisation that adds per-channel selectivity on top of the scalar decay/write gating of Gated DeltaNet.

Chunkwise training : A bespoke chunkwise algorithm uses a specialised **Diagonal-Plus-Low-Rank (DPLR)** variant of the transition matrix to remain hardware-efficient. The product of per-step transitions $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t$ admits a DPLR decomposition that allows matmul-rich parallelism on tensor cores while preserving the channel-wise decay structure.

Algorithm 39 Kimi Delta Attention Recurrent Forward (Reference Form)

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$ **Require:** Projections: W_q, W_k, W_v, W_β ; log-decay parameters $\mathbf{a}, \delta$ **Ensure:** Output $\mathbf{o}_t$ and state $\mathbf{S}_t$ 1: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t, \mathbf{k}_t \leftarrow W_k \mathbf{x}_t, \mathbf{v}_t \leftarrow W_v \mathbf{x}_t$ 2: $\beta_t \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_t)$ $\triangleright$ scalar per-head write gate3: $\alpha_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\delta))$ $\triangleright$ channel-wise decay, fp324: $\mathbf{D}_t \leftarrow \text{diag}(\alpha_t)$ $\triangleright$ diagonal decay matrix5: Decay state: $\mathbf{S}_t \leftarrow \mathbf{D}_t \mathbf{S}_{t-1}$ 6: Read: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{k}_t$ $\triangleright$ current prediction along key7: Delta-rule update: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \beta_t \mathbf{k}_t \mathbf{r}_t^\top + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ 8: Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$ 9: Output gating: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g \mathbf{x}_t)$ 10: **return** $\mathbf{o}_t, \mathbf{S}_t$

E.9.2 Empirical Performance and Trade-offs

KDA is deployed in Kimi Linear, a hybrid model (3B activated / 48B total parameters) combining KDA layers with Multi-head Latent Attention (MLA). Kimi Linear outperforms a full-MLA baseline across all evaluated tasks, reduces the KV cache by up to 75%, and achieves up to 6 $\times$ decoding throughput for 1M-token context. The channel-wise decay is the principal driver of the gain: it allows fine-grained control over which key channels persist, improving long-context recall relative to scalar-decay Gated DeltaNet.

Aspect	Strength	Limitation
Recall	Channel-wise decay improves long-context recall over scalar-decay baselines; deployed in Kimi Linear (3B/48B hybrid).	Recurrent matrix state $\mathcal{O}(d^2)$ is heavier than diagonal-gated linear attention.
Throughput	Up to 6 $\times$ decoding throughput at 1M context; 75% KV cache reduction vs full MLA.	Requires DPLR-aware chunkwise kernels for hardware efficiency.
Memory	Constant $\mathcal{O}(d^2)$ inference state (matrix recurrent).	Log-decay fp32 computation adds overhead; reduction to Gated DeltaNet loses channel selectivity.

Table 11: KDA trade-offs in Kimi Linear (3B activated / 48B total) [22].

E.10 Comparative Analysis: Taxonomy of Gated Mechanisms

Architecture	Publication Year	State Representation	Gate Type	Training Complexity	Inference Complexity	Ref
GLA	2023	Matrix (re-current)	Diagonal decay	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memory	[55]
DeltaNet	2024	Matrix (re-current)	Scalar write (β)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memory	[56]
Gated DeltaNet	2024	Matrix (re-current)	Decay + write	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memory	[57]
RetNet	2023	Matrix (re-current)	Fixed exponential	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d)$ per-step	[48]
HGRN2	2024	Matrix (re-current)	Diagonal (bounded)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memory	[39]
FoX	2025	Full attention	Logit-space bias	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ KV cache	[27]

Architecture	Publication Year	State Representation	Gate Type	Training Complexity	Inference Complexity	Ref
Gated Soft-max	2025	Full attention	Post-SDPA	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ KV cache	[40]
Gated DeltaNet-2	2026	Matrix (re-current)	sigmoid Decoupled erase + write (channel-wise)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memory	[20]
KDA	2025	Matrix (re-current)	Channel-wise decay + scalar write	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memory	[22]

Architecture	Trainable	In-Context Recall	Associative Recall	Length Extrapolation	Key Advantage
GLA	Yes	Moderate	Weak	Good (20K)	Hardware efficiency; chunkwise parallelism
DeltaNet	Yes	Strong	Perfect (MQAR)	Good	Error-correcting semantics; strong retrieval
Gated DeltaNet	Yes	Very strong	Perfect	Excellent	Balanced: forgetting + targeted updates
RetNet	Yes	Weak	Weak	Good	Simplicity; no KV cache needed
HGRN2	Yes	Moderate	Weak	Moderate	Multi-scale temporal modeling
FoX	Yes	Very strong	Very strong (near-perfect)	Excellent	Preserve softmax; minimal overhead
Gated Soft-max	Yes	Strong	Good	Good	Simplicity; no replacement of SDPA
Gated DeltaNet-2	Yes	Very strong	Very strong	Excellent	Decoupled channel-wise erase/write; best RULER multi-key
KDA	Yes	Very strong	Very strong	Excellent	Channel-wise decay; deployed in Kimi Linear (75% KV reduction, 6× decode)

Architecture	Typical Throughput	Memory Inference	per-	Primary Use Case
GLA	High (chunkwise CUDA)	$O(d^2)$ constant		Long-context with memory constraints
DeltaNet	Medium-High	$O(d^2)$ constant		Retrieval and associative reasoning
Gated DeltaNet	Medium	$O(d^2)$ constant		Production: Qwen3-Next, balanced performance
RetNet	High	$O(d)$ per-step		Extremely fast inference; simple models
HGRN2	Medium-High	$O(d^2)$ constant		Multi-scale temporal patterns
FoX	Moderate (quadratic)	$O(Ld)$ KV cache		Length extrapolation; long-context generation
Gated Softmax	High (minimal overhead)	$O(Ld)$ KV cache		Drop-in improvement for existing models
Gated DeltaNet-2	High (near-flat scaling)	$O(d^2)$ constant		Long-context retrieval; multi-key NIAH
KDA	High (up to $6\times$ decode at 1M)	$O(d^2)$ constant		Production hybrid (Kimi Linear); long-context decode

E.11 Unified Gating Principle

All eight architectures share a common principle: **gating is a mechanism to control information flow and selective memory retention**. The specific design choices diverge along several dimensions:

1. **Location of gating:** Recurrent state updates (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2) versus softmax logit/output space (FoX, Gated Softmax, RetNet).
2. **Granularity of control:** Dimension-wise (GLA, HGRN2 diagonal), key-specific (DeltaNet), token-wise (FoX), head-wise (Gated Softmax), or channel-wise decoupled erase/write (Gated DeltaNet-2).
3. **Data dependence:** Fully data-dependent (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, FoX, Gated Softmax) versus fixed schedules (RetNet with exponential decay).
4. **Expressiveness trade-off:** Linear recurrent efficiency (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, RetNet) versus full softmax expressiveness (FoX, Gated Softmax).

E.12 Implementation and Practical Guidance

E.12.1 When to Use Each Architecture

1. **GLA:** Fast inference with moderate-to-long contexts (2K–20K tokens), when custom CUDA kernels are acceptable, and retrieval performance is not critical.
2. **DeltaNet:** Strong associative recall and in-context learning tasks; good for synthetic tasks (MQAR) and key-value association testing.
3. **Gated DeltaNet:** Production models requiring the best balance of recall, forgetting, and throughput (proven in Qwen3-Next; ICLR 2025).
4. **Gated DeltaNet-2:** Long-context retrieval workloads (multi-key NIAH) where decoupled channel-wise erase/write yields the strongest recall; when a gate-aware WY kernel is available.

5. **RetNet**: Extremely efficient inference without KV caching; suitable for edge/mobile deployments or toy models.
6. **HGRN2**: Multi-scale temporal hierarchies; when layer-specific forget bounds are desirable.
7. **FoX**: Length extrapolation and long-context understanding; when quadratic training is acceptable and replacing softmax is not desired.
8. **Gated Softmax**: Quick improvement to existing softmax models with minimal code changes; best for practitioners wanting immediate gains without major refactoring.

E.12.2 Hardware Considerations

Hardware Target	Recommended Architectures	Notes
GPU (H100/A100)	Gated Softmax, FoX, Gated DeltaNet, Gated DeltaNet-2	Mature softmax/linear implementations. GLA/DeltaNet require custom kernels.
TPU (high memory)	GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2	Hardware supports efficient matmuls; linear recurrent methods shine.
Mobile/Edge	RetNet, lightweight Gated Softmax	Constant-memory inference critical.

F Annex E: Conceptual Introduction—Transformers and Attention for Beginners

This appendix provides an accessible introduction to the fundamental concepts of transformers and the attention mechanism, aimed at readers without deep prior knowledge in deep learning. Transformers are the engine behind modern artificial intelligence systems like ChatGPT, but their operation can be understood through real-world analogies.

F.1 What is a Transformer?

A transformer is a neural architecture that processes text or data sequences by interpreting the meaning of each word while considering all other words in context. Imagine you read a sentence:

“The bank was full of people waiting. Mary went to the bank.”

What does “bank” mean in each case? In the first sentence, it refers to a financial institution (or a bench). In the second, it is less clear without context. A transformer solves this automatically by looking at all surrounding words. In the second sentence, seeing “Mary went”, the model better understands that it probably is a riverbank (where she goes to swim) or a financial institution (where she goes to conduct a transaction).

This ability to understand the complete meaning of a word by considering the entire sentence is called *contextualization*, and it is the heart of how modern transformers work.

F.2 The Attention Mechanism: An Analogy

The attention mechanism is the “magic trick” that allows transformers to understand context. Think of it as participating in an important conversation in a noisy room:

1. **Lots of noise:** Someone is speaking and being very important to you.
2. **You ignore the rest:** Your brain automatically focuses attention on that person, ignoring other sounds.
3. **You understand better:** By concentrating, you catch every word clearly.

The neural attention mechanism works the same way: each word “asks” how important every other word is to understanding its meaning, then “focuses” its attention on the most relevant ones.

F.3 Practical Example Step-by-Step

Let’s see how a transformer understands the word “eats” in two different contexts:

F.3.1 Context 1: “The cat eats fish”

The transformer, when processing “eats”, internally asks:

- How relevant is “The”? Little (it’s just an article). **Low attention.**
- How relevant is “cat”? Very relevant (it’s the subject, who eats). **High attention.**
- How relevant is “fish”? Very relevant (it’s what is eaten). **High attention.**

Thus, the mental representation of “eats” focuses primarily on “cat” and “fish”.

F.3.2 Context 2: “The restaurant eats into profit margins”

Here the transformer asks in a different context:

- How relevant is “restaurant”? Very relevant (it’s the subject). **High attention.**
- How relevant is “profit”? Very relevant (it’s affected). **High attention.**
- How relevant is “margins”? Very relevant (it’s what’s being consumed). **High attention.**

The word “eats” gets a completely different representation because it “attends” to different words in different contexts.

F.4 How Attention Works Mathematically (Simple Version)

Behind the attention shift is mathematics. Here is the simplified version without too much technical detail:

F.4.1 Step 1: Queries, Keys, and Values

Each word in the sentence is transformed into three versions:

- **Query:** “What information do I need to know?”
- **Key:** “What information do I have?”
- **Value:** “Here is my important information.”

Imagine in a library, each visitor is a “query”, each book is a “key”, and the content is the “value”. The librarian (attention) matches queries with the most relevant keys to access the correct value.

F.4.2 Step 2: Compatibility

The system examines how *compatible* each query is with each key. Queries similar to keys get a *high* compatibility score. This is calculated by multiplying the query by the key (mathematically, the dot product).

F.4.3 Step 3: Focus

The compatibility scores are converted into “focus weights”. High compatibilities mean “focus attention here”, and low ones mean “ignore this”. Mathematically, a function called **softmax** converts scores into percentages (like: 40% attention here, 35% here, 25% there).

F.4.4 Step 4: Combine

Finally, the system combines all values, weighted by the focus weights. If a word has 80% attention on the subject, 80% of the subject’s information gets blended into that word’s representation.

F.5 Multiple Attention Heads: Multiple Perspectives

A key feature is that transformers do not use a single attention but *multiple attention heads* in parallel. It’s as if you had 8 people analyzing the sentence *simultaneously*, each paying attention to different aspects:

- **Person 1:** “I focus on subject-verb relationships.”
- **Person 2:** “I search for the direct object.”
- **Person 3:** “I track information about verb tense.”
- **Person 4:** “I search for modifiers and adjectives.”

Each “head” learns to focus on different language patterns. Together, they capture much richer understanding than a single head.

F.6 Stacking Layers

Transformers process information in *multiple layers* (usually 12 to 48 for practical models). Imagine editing an essay:

1. **First pass:** You fix spelling and basic grammar.
2. **Second pass:** You improve clarity and sentence structure.
3. **Third pass:** You ensure consistency and narrative flow.

Transformers work the same way: each layer refines text understanding. The first layer captures basic features (simple words, genders), while later layers understand complex concepts (word relationships, abstract meanings).

F.7 Why Does It Matter?

The attention mechanism was revolutionary because:

1. **Parallelism:** It finds context for *any word with any other word*, without processing them sequentially (unlike earlier systems). This makes it very fast.

2. **Flexibility:** It learns what patterns to look for automatically from data, without you needing to program rules manually.
3. **Scalability:** It works from small texts to contexts with millions of words.
4. **General Capabilities:** The same mechanism works for translation, summarization, question-answering, text generation, computer vision, and more.

F.8 Practical Challenges

Despite the extraordinary success of transformers, they present challenges:

F.8.1 Computational Complexity

If a sentence has 1,000 words, the standard attention mechanism must compare each word with all other 1,000 words. That's $1,000 \times 1,000 = 1,000,000$ comparisons. For long documents with millions of words, this becomes prohibitively expensive in time and memory.

F.8.2 Memory Requirements

Especially during inference (when using trained models), storing the entire attention matrix can consume enormous amounts of RAM or GPU memory, limiting the sequence lengths you can process.

F.8.3 Device Efficiency

Transformers were designed for powerful TPUs and GPUs. Running them on phones or embedded devices is challenging.

F.9 Modern Solutions

To solve these challenges, research has proposed many variants:

- **Sparse Attention:** Instead of comparing each word with ALL others, it compares only with a strategic subset (nearby neighbors, periodic patterns). Reduces complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$.
- **Recurrent Models:** Like Mamba, they incorporate aspects of old recurrent neural networks but with better modern efficiency.
- **Gated Attention:** Mechanisms that selectively learn what information to pass forward, reducing storage needs.
- **Quantization:** Use smaller numbers (integers instead of decimals) to reduce memory without losing too much precision.

These innovations are what this toolkit (Frankenstein) allows you to experiment with easily.

F.10 Key Takeaways

- A **transformer** is a neural network that understands language context very well.
- The **attention mechanism** allows each word to “focus” on which other words are relevant to understanding its meaning.
- Attention works by computing **compatibility** between each word (query) and all others (keys), then combining information based on that compatibility (values).

- **Multiple heads** of attention explore different patterns in parallel.
- **Multiple layers** progressively refine understanding.
- The main challenge is that standard attention has quadratic complexity, which is expensive for long texts.
- Many **modern solutions** (sparse attention, recurrent models, quantization) address these challenges, and this toolkit lets you explore all of them.

G Annex F: Normalization, Quantization, Depth Routing, and Runtime Algorithms

This annex collects the implementation-level detail for normalization variants, BitNet-style ternary quantization, adaptive depth routing (Mixture-of-Depths), conditional memory (Engram), factorized embeddings, and the runtime algorithms (training-step stability controls, SBERT inference router) that support the configuration-driven pipeline. The main sections defer to this annex for the mathematical formulations and pseudocode; the bibliography keys cited here resolve against `docs/bibliography/`.

G.1 Normalization Variants: LayerNorm, RMSNorm, Dynamic Tanh, and Dynamic Erf

Normalization determines how activation scale is controlled across depth. In this repository, normalization is not only a modeling choice but also a schema compatibility question, because only certain values are currently accepted by `norm_type`. The schema contract is:

$$\text{norm_type} \in \{\text{layer_norm}, \text{dynamic_tanh}, \text{derf}\}$$

so `rms_norm` is **not** a valid schema value in the current contract. The three formulations most relevant to this codebase are:

G.1.1 LayerNorm (Baseline)

LayerNorm [3] is the default normalizer; it centers and rescales each token by its per-token mean and variance:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2, \quad y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

G.1.2 RMSNorm (Not Accepted by Schema)

RMSNorm removes mean-centering and only rescales by root mean square magnitude [62]:

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}, \quad y_i = \gamma_i \frac{x_i}{\text{RMS}(x)}.$$

Compared with LayerNorm, RMSNorm is computationally simpler (no subtraction of feature mean) and is often used when reducing normalization overhead is important. It is documented here for reference but is **not** an accepted `norm_type` value.

G.1.3 Dynamic Tanh (DyT)

Dynamic Tanh proposes replacing explicit normalization with a bounded elementwise map [67]:

$$\text{DyT}(x) = \tanh(\alpha x)$$

where α is learned. The core idea is that bounded nonlinear contraction can provide stable signal scaling without explicitly computing per-token normalization statistics.

G.1.4 Dynamic Erf (D erf)

D erf extends the same normalization-free direction by using an error-function based map [8]:

$$\text{D erf}(x) = \text{erf}(\alpha x + s)$$

with learnable scale/shift. Reported results in the cited work indicate stronger performance than DyT and common normalization baselines across multiple domains.

G.1.5 Comparison

Method	Formula	Stats Needed	Notes
LayerNorm	$\gamma_i(x_i - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta_i$	mean + variance	Default; full centering and rescaling [3].
RMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{d} \sum_j x_j^2 + \epsilon}$	RMS only	Lower overhead; widely used baseline; not accepted by schema [62].
Dynamic Tanh	$\tanh(\alpha x)$	none	Normalization-free bounded transform; drop-in replacement [67].
Dynamic Erf	$\text{erf}(\alpha x + s)$	none	Normalization-free alternative; improves over DyT [8].

G.2 BitNet Ternary Quantization Path

When `use_bitnet=true`, BitLinear layers replace standard `nn.Linear` with ternary weight quantization ($-1, 0, +1$) following BitNet [52]. Given weight tensor W , a practical scaling is:

$$s = \text{mean}(|W|), \quad \tilde{W} = \text{clip} \left(\text{round} \left(\frac{W}{s} \right), -1, 1 \right).$$

The packed mapping uses two bits per weight symbol for storage efficiency. This enables significant compression ($\sim 32\times$ versus FP32) but is experimental and may affect model quality; it is best suited for inference deployment.

Activation quantization for the INT8 path is:

$$q = \text{round} \left(x \cdot \frac{127}{\max(|x|) + \epsilon} \right), \quad q \in [-128, 127]$$

with dequantization $x \approx q/\alpha$. For N parameters, size estimates before metadata and packing overhead are:

$$\text{FP32 size} \approx 4N, \quad \text{FP16 size} \approx 2N, \quad \text{1.58-bit size} \approx \frac{1.58}{8}N$$

which aligns with lightweight deployment goals [43, 7].

G.3 Mixture-of-Depths (MoD) Token Routing

When `use_mixture_of_depths=true`, each transformer block scores tokens with a lightweight router. Only the top `mixture_of_depths_capacity_ratio` subset is updated; skipped tokens pass through unchanged [68]. This allocates more depth to salient tokens, reducing per-layer compute.

- 1: **Input:** hidden states $H \in \mathbb{R}^{n \times d}$, router R , capacity ratio ρ
- 2: $\text{scores} \leftarrow R(H)$ ▷ per-token router score
- 3: $\mathcal{S} \leftarrow \text{top-k}(\text{scores}, k = \lceil \rho \cdot n \rceil)$ ▷ select salient subset
- 4: $H_{\text{update}} \leftarrow \text{block}(H[\mathcal{S}])$ ▷ apply transformer block to selected tokens
- 5: $H[\mathcal{S}] \leftarrow H_{\text{update}}$ ▷ write back; non-selected tokens bypass the block
- 6: **Output:** H ▷ auxiliary load-balancing loss added from `mixture_of_depths_router_aux_loss_weight`

G.3.1 Configuration

- `use_mixture_of_depths`: bool — enable MoD token routing.
- `mixture_of_depths_capacity_ratio`: float in $(0, 1]$ — fraction of tokens updated per layer.
- `mixture_of_depths_router_aux_loss_weight`: float ≥ 0 — auxiliary loss weight for router regularization.

G.4 Engram Conditional Memory

Engram (`engram_attn`) implements conditional memory via scalable N-gram lookup [10]. It builds hash-based lookup tables for N-gram contexts from size 2 up to `engram_max_ngram_size`, with independent hash heads per N-gram order. A causal depthwise convolution (`engram_kernel_size`) provides short-range context before the N-gram lookup.

- 1: **Input:** token stream $x_{1:n}$, N-gram orders $\{2, \dots, N\}$, hash heads per order H
- 2: $h_{\text{conv}} \leftarrow \text{DepthwiseConv1d}(x_{1:n}, k = \text{engram_kernel_size})$ ▷ short-range context
- 3: **for** each N-gram order $k \in \{2, \dots, N\}$ **do**
- 4: $\text{ctx}_t^{(k)} \leftarrow x_{t-k+1:t}$ ▷ N-gram context window
- 5: $\text{lookup}_t^{(k)} \leftarrow \text{HashTable}^{(k)}[\text{hash}(\text{ctx}_t^{(k)})]$ ▷ per-head retrieval, H heads per order
- 6: **end for**
- 7: $y_t \leftarrow \sum_k \text{lookup}_t^{(k)} + h_{\text{conv},t}$ ▷ combine conditional memories
- 8: **Output:** $y_{1:n}$

G.4.1 Configuration

- `engram_max_ngram_size`: int ≥ 2 — highest N-gram order (default 3).
- `engram_n_heads_per_ngram`: int ≥ 1 — hash heads per N-gram order (default 4).
- `engram_embed_dim_per_head`: int ≥ 1 — embedding dim per hash head (default 32).
- `engram_kernel_size`: int ≥ 1 — causal depthwise conv kernel width (default 4).
- `engram_seed`: int — hash seed for reproducibility (default 42).

Total Engram hidden size = $(\text{max_ngram_size}-1) \times \text{n_heads_per_ngram} \times \text{embed_dim_per_head}$.

Algorithm 40 Schema-Driven Training Step with Stability Controls

Require: Batch stream, config C

```
1: Initialize retry counter  $r \leftarrow 0$ 
2: for each optimizer step do
3:   Accumulate gradients for  $K = C.\text{gradient\_accumulation\_steps}$  micro-batches
4:   Apply global norm clipping with  $\tau = C.\text{grad\_clip\_max\_norm}$ 
5:   if post-clip gradient exceeds  $C.\text{inf\_post\_clip\_threshold}$  or NaN/Inf detected then
6:     if  $r < C.\text{max\_nan\_retries}$  then
7:       restore safe state / skip step;  $r \leftarrow r + 1$ 
8:       continue
9:     else
10:      stop training with failure state
11:    end if
12:  end if
13:  run optimizer step selected by optimizer_class
14:  update scheduler (cosine, constant, or linear_warmup_then_constant)
15:  if step mod checkpoint_every_n_steps = 0 then
16:    save rolling checkpoint and prune to max_rolling_checkpoints
17:  end if
18:  update best checkpoints up to num_best_checkpoints
19:  emit CSV + telemetry following gradient_log_interval and telemetry_log_interval
20: end for
```

G.5 Factorized Embeddings and Embedding Convolution

G.5.1 Factorized Embeddings

When `use_factorized_embedding=true`, the embedding matrix is factorized into two smaller matrices, reducing parameters from $O(V \times H)$ to $O(V \times R + R \times H)$ where $R = \text{factorized_embedding_dim}$.

- `use_factorized_embedding`: bool — enable factorization.
- `factorized_embedding_dim`: int ≥ 1 — intermediate dimension (typical: 64–256).

G.5.2 Embedding Convolution

When `use_embedding_conv=true`, a depthwise Conv1d is applied over the embedding stream before the first transformer block, with kernel size `embedding_conv_kernel`. This provides a lightweight local-context filter on token embeddings.

G.6 Training-Step Stability Controls

The schema-driven training step applies gradient accumulation, global-norm clipping, post-clip explosion guards, and bounded NaN/Inf retries. The accumulated gradient is:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^K g_i, \quad K = \text{gradient_accumulation_steps}$$

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min \left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon} \right), \quad \tau = \text{grad_clip_max_norm}$$

then overflow guards use `inf_post_clip_threshold` and retry logic bounded by `max_nan_retries`.

Algorithm 41 SBERT Inference Mode Router

Require: mode m , model E , inputs X

```
1: if  $m = \text{similarity}$  then  
2:   return  $\cos(E(x_1), E(x_2))$   
3: else if  $m = \text{search}$  then  
4:   return top- $k$  by dot-product/cosine against corpus embeddings  
5: else if  $m = \text{cluster}$  then  
6:   return clustering labels over  $E(X)$   
7: else  
8:   return serialized embeddings  $E(X)$   
9: end if
```

G.7 SBERT Inference Mode Router

The SBERT inference dispatcher selects among similarity scoring, corpus search, clustering, and persistent encoding modes over a single shared encoder [42]. The cosine similarity and regression loss are:

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\| \|e_2\|}, \quad \mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

with $y \in [-1, 1]$ in this pipeline.